

Assignment 1 - Regression

Group 6

2026-01-03

Contents

1	Abstract	1
2	Data Preprocessing	2
2.1	Data Understanding	5
2.2	Feature Selection and Reduction Strategy	21
2.3	Data Wrangling	22
2.4	Data Anomalies	37
2.5	Summary	101
3	Training	105
3.1	Task and metric	105
3.2	Loading The preprocessed data	106
3.3	Imputation of missing values	110
3.4	Linear model	113
3.5	Random Forest	126
3.6	XGBoost	130
3.7	Cross-Validation	138
4	Conclusion	139
4.1	Lessons Learned & Critical reflection	140

1 Abstract

This project investigates the prediction of LendingClub interest rates using borrower and loan characteristics, with the goal of comparing models of increasing complexity and assessing their generalization performance. After extensive data preprocessing and exploratory analysis, three modeling approaches were evaluated: linear regression, Random Forest, and XGBoost. The linear model served as a transparent baseline but showed clear limitations due to non-linearity and heteroskedasticity. Tree-based ensemble methods substantially improved predictive accuracy by capturing complex interactions. The XGBoost achieved the strongest overall performance with a MSE of 8.78. Cross-validation and diagnostic analyses revealed stable generalization

across data splits but also highlighted systematic underestimation of extreme interest rates, reflecting inherent challenges in modeling rare, high-risk loans. Overall, the results demonstrate that while advanced ensemble methods offer meaningful improvements over linear models, predictive accuracy is fundamentally constrained by data noise and heterogeneity in consumer credit markets.

2 Data Preprocessing

In this exploratory data analysis we used following libraries:

```
library(tidyverse)
library(corrplot)
library(fastDummies)
library(dplyr)
library(ggplot2)
library(patchwork)
library(lubridate)
library(caret)
library(readr)
```

The data is loaded into a dataframe and predefined columns are dropped. With a first look at the data using the data dictionary, as well as looking at the first few rows, a dictionary for the data types is created. The data is casted into the correct data types. and a copy of the original dataframe is created for backup.

```
original <- read.csv("/Users/Jujou/Documents/Repos/GRP-6_DS-Project/DS-Project_data/LCdata.csv",
                    row.names = NULL,
                    sep = ";",
                    header = TRUE)

# Predefined columns to drop
columns_to_drop <- c("collection_recovery_fee",
                    "installment",
                    "funded_amnt",
                    "funded_amnt_inv",
                    "issue_d",
                    "last_pymnt_amnt",
                    "last_pymnt_d",
                    "loan_status",
                    "next_pymnt_d",
                    "out_prncp",
                    "out_prncp_inv",
                    "pymnt_plan",
                    "recoveries",
                    "total_pymnt",
                    "total_pymnt_inv",
                    "total_rec_int",
                    "total_rec_late_fee",
                    "initial_list_status",
                    "total_rec_prncp",
                    "url")

original_drop <- original[, -match(columns_to_drop, names(original))]
```

```

# Dtype dictionary
dtype_dict <- list(
  acc_now_delinq = "integer",
  addr_state = "factor",
  all_util = "numeric",
  annual_inc = "numeric",
  annual_inc_joint = "numeric",
  application_type = "factor",
  collections_12_mths_ex_med = "integer",
  delinq_2yrs = "integer",
  desc = "factor",
  dti = "numeric",
  dti_joint = "numeric",
  earliest_cr_line = "str", # special case, will be converted to date later
  emp_length = "factor",
  emp_title = "factor",
  home_ownership = "factor",
  id = "integer",
  il_util = "numeric",
  inq_fi = "integer",
  inq_last_12m = "integer",
  inq_last_6mths = "integer",
  int_rate = "numeric",
  last_credit_pull_d = "str", # special case, will be converted to date later
  loan_amnt = "integer",
  max_bal_bc = "numeric",
  member_id = "integer",
  mths_since_last_delinq = "integer",
  mths_since_last_major_derog = "integer",
  mths_since_last_record = "integer",
  mths_since_rcnt_il = "integer",
  open_acc = "integer",
  open_acc_6m = "integer",
  open_il_12m = "integer",
  open_il_24m = "integer",
  open_il_6m = "integer",
  open_rv_12m = "integer",
  open_rv_24m = "integer",
  policy_code = "factor",
  pub_rec = "integer",
  purpose = "factor",
  revol_bal = "integer",
  revol_util = "numeric",
  term = "factor",
  title = "factor",
  tot_coll_amt = "numeric",
  tot_cur_bal = "numeric",
  total_acc = "integer",
  total_bal_il = "numeric",
  total_cu_tl = "integer",
  total_rev_hi_lim = "numeric",
  zip_code = "factor"
)

```

```

# Cast dtypes according to dictionary
for(col_name in names(dtype_dict)) {
  dtype <- dtype_dict[[col_name]]
  if(dtype == "factor") {
    original_drop[[col_name]] <- as.factor(original_drop[[col_name]])
  } else if(dtype == "integer") {
    original_drop[[col_name]] <- as.integer(original_drop[[col_name]])
  } else if(dtype == "numeric") {
    original_drop[[col_name]] <- as.numeric(original_drop[[col_name]])
  } else if(dtype == "date") {
    original_drop[[col_name]] <- as.Date(original_drop[[col_name]], format="%b-%Y")
  }
}

str(original_drop)

```

```

## 'data.frame': 798641 obs. of 52 variables:
## $ id : int 83489 79893 79924 77792 72176 72819 89258 5764263 837813 305855
## $ member_id : int 83471 79878 79906 77764 70868 72816 80039 7196024 1047868 33138
## $ loan_amnt : int 2600 6700 7000 3000 2525 7200 2000 18000 5000 7000 ...
## $ term : Factor w/ 2 levels " 36 months"," 60 months": 1 1 1 1 1 1 1 1 1 1 ..
## $ int_rate : num 8.38 7.75 7.75 9.01 9.33 ...
## $ emp_title : Factor w/ 274363 levels "", " \tAdv Mtr Proj Fld Rep",...: 50890 1 1 2
## $ emp_length : Factor w/ 12 levels "< 1 year","1 year",...: 5 1 1 1 1 1 9 2 2 3 ...
## $ home_ownership : Factor w/ 6 levels "ANY","MORTGAGE",...: 2 3 3 6 6 6 2 2 2 ...
## $ annual_inc : num 6500 NA NA 35000 110000 95000 150000 59000 52500 36000 ...
## $ verification_status : chr "Not Verified" "Not Verified" "Not Verified" "Not Verified" ...
## $ desc : Factor w/ 112107 levels "", "\t Loan for purchase of grand piano. Pian
## $ purpose : Factor w/ 14 levels "car","credit_card",...: 10 10 10 10 10 10 3 3 9
## $ title : Factor w/ 57719 levels "", "\tcredit_card",...: 14195 37624 21574 2048
## $ zip_code : Factor w/ 931 levels "007xx","008xx",...: 758 90 90 626 90 90 60 286
## $ addr_state : Factor w/ 51 levels "AK","AL","AR",...: 6 35 35 17 35 35 7 11 25 42
## $ dti : num 6.46 1 1 10 10 ...
## $ delinq_2yrs : int NA NA NA NA NA NA 0 0 0 0 ...
## $ earliest_cr_line : chr "" "" "" "" ...
## $ inq_last_6mths : int NA NA NA NA NA NA 0 1 4 0 ...
## $ mths_since_last_delinq : int NA NA NA NA NA NA 0 66 62 NA ...
## $ mths_since_last_record : int NA NA NA NA NA NA 0 NA NA NA ...
## $ open_acc : int NA NA NA NA NA NA 7 9 8 8 ...
## $ pub_rec : int NA NA NA NA NA NA 0 0 0 0 ...
## $ revol_bal : int 0 0 0 0 0 0 150786 11608 9204 5046 ...
## $ revol_util : num NA NA NA NA NA NA 2.2 56.9 29.3 19.9 ...
## $ total_acc : int NA NA NA NA NA NA 16 19 43 28 ...
## $ last_credit_pull_d : chr "Mar-2010" "May-2007" "May-2007" "Jun-2010" ...
## $ collections_12_mths_ex_med : int NA NA NA NA NA NA NA 0 0 0 ...
## $ mths_since_last_major_derog : int NA NA NA NA NA NA NA 67 NA NA ...
## $ policy_code : Factor w/ 1 level "1": 1 1 1 1 1 1 1 1 1 1 ...
## $ application_type : Factor w/ 2 levels "INDIVIDUAL","JOINT": 1 1 1 1 1 1 1 1 1 1 ...
## $ annual_inc_joint : num NA NA NA NA NA NA NA NA NA NA ...
## $ dti_joint : num NA NA NA NA NA NA NA NA NA NA ...
## $ verification_status_joint : chr "" "" "" "" ...
## $ acc_now_delinq : int NA NA NA NA NA NA 0 0 0 0 ...
## $ tot_coll_amt : num NA NA NA NA NA NA NA 0 NA 0 ...

```



```
## $ tot_cur_bal      : num  NA NA NA NA NA ...
## $ open_acc_6m      : int   NA NA NA NA NA NA NA NA NA NA NA ...
## $ open_il_6m       : int   NA NA NA NA NA NA NA NA NA NA NA ...
## $ open_il_12m      : int   NA NA NA NA NA NA NA NA NA NA NA ...
## $ open_il_24m      : int   NA NA NA NA NA NA NA NA NA NA NA ...
## $ mths_since_rcnt_il : int   NA NA NA NA NA NA NA NA NA NA NA ...
## $ total_bal_il     : num   NA NA NA NA NA NA NA NA NA NA NA ...
## $ il_util          : num   NA NA NA NA NA NA NA NA NA NA NA ...
## $ open_rv_12m      : int   NA NA NA NA NA NA NA NA NA NA NA ...
## $ open_rv_24m      : int   NA NA NA NA NA NA NA NA NA NA NA ...
## $ max_bal_bc       : num   NA NA NA NA NA NA NA NA NA NA NA ...
## $ all_util         : num   NA NA NA NA NA NA NA NA NA NA NA ...
## $ total_rev_hi_lim  : num   NA NA NA NA NA NA NA 20400 NA 25300 ...
## $ inq_fi           : int   NA NA NA NA NA NA NA NA NA NA NA ...
## $ total_cu_tl      : int   NA NA NA NA NA NA NA NA NA NA NA ...
## $ inq_last_12m     : int   NA NA NA NA NA NA NA NA NA NA NA ...
```

```
# Make Copy to work with
```

```
data <- original_drop
```

A predefined set of variables was excluded at the beginning of the preprocessing pipeline based on the project specification and the provided data dictionary. In addition, url is removed immediately after loading the raw data. Together with id, and member_id these variables uniquely identify loans or borrowers but do not contain predictive information and are not available or meaningful for new loan applications. The id and member_id are dropped later after checking for duplicates. Dropping them at the outset prevents accidental use in downstream preprocessing or modeling steps and ensures that all subsequent transformations operate exclusively on potentially informative features.

2.1 Data Understanding

2.1.1 Dimensions

For a first understanding of the first few rows of the dataframe, we use the head() function. We check the dimensions of the dataframe using the dim() function, and we use the str() function to get an overview of the data types of each column. Furthermore we look at the summary statistics of the dataset. One could discuss whether one should look at the statistics before a train-test-split or after, since this leads to certain bias. In this case we decided to look at the statistics before the split, as learned in class.

```
# Display the first few rows of the dataset
print("First few rows of the dataset:")
```

```
## [1] "First few rows of the dataset:"
```

```
print(head(data))
```

```
##      id member_id loan_amnt      term int_rate      emp_title emp_length
## 1 83489     83471     2600 36 months    8.38 College Pro Painters      3 years
## 2 79893     79878     6700 36 months    7.75                    < 1 year
## 3 79924     79906     7000 36 months    7.75                    < 1 year
## 4 77792     77764     3000 36 months    9.01      Tanks Tavern    < 1 year
```

```

## 5 72176      70868      2525 36 months      9.33      < 1 year
## 6 72819      72816      7200 36 months     10.28      < 1 year
##  home_ownership annual_inc verification_status
## 1      MORTGAGE      6500      Not Verified
## 2      NONE      NA      Not Verified
## 3      NONE      NA      Not Verified
## 4      RENT      35000      Not Verified
## 5      RENT      110000      Not Verified
## 6      RENT      95000      Not Verified
##
## 1
## 2
## 3
## 4
## 5
## 6 I am consolidating 2 loans that carry 14.5% and 16% interest rate respectively. The first loan has
##  purpose      title zip_code addr_state      dti
## 1  other      Credit card refinancing      806xx      CO      6.46
## 2  other      Moving expenses and security deposit      100xx      NY      1.00
## 3  other      Education      100xx      NY      1.00
## 4  other      Dep4774      665xx      KS      10.00
## 5  other      Car repair bill      100xx      NY      10.00
## 6  other      Consolidate 2 high-interest rate loans      100xx      NY      10.00
##  delinq_2yrs earliest_cr_line inq_last_6mths mths_since_last_delinq
## 1      NA      NA      NA
## 2      NA      NA      NA
## 3      NA      NA      NA
## 4      NA      NA      NA
## 5      NA      NA      NA
## 6      NA      NA      NA
##  mths_since_last_record open_acc pub_rec revol_bal revol_util total_acc
## 1      NA      NA      NA      0      NA      NA
## 2      NA      NA      NA      0      NA      NA
## 3      NA      NA      NA      0      NA      NA
## 4      NA      NA      NA      0      NA      NA
## 5      NA      NA      NA      0      NA      NA
## 6      NA      NA      NA      0      NA      NA
##  last_credit_pull_d collections_12_mths_ex_med mths_since_last_major_derog
## 1      Mar-2010      NA      NA
## 2      May-2007      NA      NA
## 3      May-2007      NA      NA
## 4      Jun-2010      NA      NA
## 5      May-2007      NA      NA
## 6      May-2007      NA      NA
##  policy_code application_type annual_inc_joint dti_joint
## 1      1      INDIVIDUAL      NA      NA
## 2      1      INDIVIDUAL      NA      NA
## 3      1      INDIVIDUAL      NA      NA
## 4      1      INDIVIDUAL      NA      NA
## 5      1      INDIVIDUAL      NA      NA
## 6      1      INDIVIDUAL      NA      NA
##  verification_status_joint acc_now_delinq tot_coll_amt tot_cur_bal open_acc_6m
## 1      NA      NA      NA      NA
## 2      NA      NA      NA      NA

```

```
## 3      NA      NA      NA      NA
## 4      NA      NA      NA      NA
## 5      NA      NA      NA      NA
## 6      NA      NA      NA      NA
##   open_il_6m open_il_12m open_il_24m mths_since_rcnt_il total_bal_il il_util
## 1      NA      NA      NA      NA      NA      NA
## 2      NA      NA      NA      NA      NA      NA
## 3      NA      NA      NA      NA      NA      NA
## 4      NA      NA      NA      NA      NA      NA
## 5      NA      NA      NA      NA      NA      NA
## 6      NA      NA      NA      NA      NA      NA
##   open_rv_12m open_rv_24m max_bal_bc all_util total_rev_hi_lim inq_fi
## 1      NA      NA      NA      NA      NA      NA
## 2      NA      NA      NA      NA      NA      NA
## 3      NA      NA      NA      NA      NA      NA
## 4      NA      NA      NA      NA      NA      NA
## 5      NA      NA      NA      NA      NA      NA
## 6      NA      NA      NA      NA      NA      NA
##   total_cu_tl inq_last_12m
## 1      NA      NA
## 2      NA      NA
## 3      NA      NA
## 4      NA      NA
## 5      NA      NA
## 6      NA      NA
```

```
# Get the dimensions of the dataset
dim(data)
```

```
## [1] 798641    52
```

```
print(paste("The number of rows is:", nrow(data))) # 798'641
```

```
## [1] "The number of rows is: 798641"
```

```
print(paste("The number of columns is:", ncol(data))) # 52
```

```
## [1] "The number of columns is: 52"
```

```
# Get the structure of the dataset
str(data)
```

```
## 'data.frame':    798641 obs. of  52 variables:
##  $ id              : int  83489 79893 79924 77792 72176 72819 89258 5764263 837813 305855
##  $ member_id       : int  83471 79878 79906 77764 70868 72816 80039 7196024 1047868 33138
##  $ loan_amnt       : int  2600 6700 7000 3000 2525 7200 2000 18000 5000 7000 ...
##  $ term            : Factor w/ 2 levels " 36 months"," 60 months": 1 1 1 1 1 1 1 1 1 1 ..
##  $ int_rate        : num  8.38 7.75 7.75 9.01 9.33 ...
##  $ emp_title       : Factor w/ 274363 levels "", " \tAdv Mtr Proj Fld Rep",...: 50890 1 1 2
##  $ emp_length      : Factor w/ 12 levels "< 1 year","1 year",...: 5 1 1 1 1 1 9 2 2 3 ...
##  $ home_ownership  : Factor w/ 6 levels "ANY","MORTGAGE",...: 2 3 3 6 6 6 2 2 2 2 ...
```

```

## $ annual_inc           : num  6500 NA NA 35000 110000 95000 150000 59000 52500 36000 ...
## $ verification_status  : chr   "Not Verified" "Not Verified" "Not Verified" "Not Verified" ...
## $ desc                 : Factor w/ 112107 levels "", "\t Loan for purchase of grand piano. Pia
## $ purpose              : Factor w/ 14 levels "car","credit_card",...: 10 10 10 10 10 10 3 3 9 1
## $ title                : Factor w/ 57719 levels "", "\tcredit_card",...: 14195 37624 21574 2048
## $ zip_code             : Factor w/ 931 levels "007xx","008xx",...: 758 90 90 626 90 90 60 286
## $ addr_state           : Factor w/ 51 levels "AK","AL","AR",...: 6 35 35 17 35 35 7 11 25 42
## $ dti                  : num    6.46 1 1 10 10 ...
## $ delinq_2yrs          : int    NA NA NA NA NA NA NA 0 0 0 0 ...
## $ earliest_cr_line     : chr     "" "" "" "" ...
## $ inq_last_6mths       : int    NA NA NA NA NA NA NA 0 1 4 0 ...
## $ mths_since_last_delinq : int    NA NA NA NA NA NA NA 0 66 62 NA ...
## $ mths_since_last_record : int    NA NA NA NA NA NA NA 0 NA NA NA ...
## $ open_acc             : int    NA NA NA NA NA NA NA 7 9 8 8 ...
## $ pub_rec              : int    NA NA NA NA NA NA NA 0 0 0 0 ...
## $ revol_bal            : int    0 0 0 0 0 0 150786 11608 9204 5046 ...
## $ revol_util           : num    NA NA NA NA NA NA NA 2.2 56.9 29.3 19.9 ...
## $ total_acc            : int    NA NA NA NA NA NA NA 16 19 43 28 ...
## $ last_credit_pull_d   : chr     "Mar-2010" "May-2007" "May-2007" "Jun-2010" ...
## $ collections_12_mths_ex_med : int    NA NA NA NA NA NA NA NA 0 0 0 ...
## $ mths_since_last_major_derog: int    NA NA NA NA NA NA NA NA 67 NA NA ...
## $ policy_code          : Factor w/ 1 level "1": 1 1 1 1 1 1 1 1 1 1 ...
## $ application_type     : Factor w/ 2 levels "INDIVIDUAL","JOINT": 1 1 1 1 1 1 1 1 1 1 ...
## $ annual_inc_joint     : num    NA NA NA NA NA NA NA NA NA NA NA ...
## $ dti_joint            : num    NA NA NA NA NA NA NA NA NA NA NA ...
## $ verification_status_joint : chr     "" "" "" "" ...
## $ acc_now_delinq       : int    NA NA NA NA NA NA NA 0 0 0 0 ...
## $ tot_coll_amt         : num    NA NA NA NA NA NA NA NA 0 NA 0 ...
## $ tot_cur_bal          : num    NA NA NA NA NA ...
## $ open_acc_6m          : int    NA NA NA NA NA NA NA NA NA NA NA ...
## $ open_il_6m           : int    NA NA NA NA NA NA NA NA NA NA NA ...
## $ open_il_12m          : int    NA NA NA NA NA NA NA NA NA NA NA ...
## $ open_il_24m          : int    NA NA NA NA NA NA NA NA NA NA NA ...
## $ mths_since_rcnt_il   : int    NA NA NA NA NA NA NA NA NA NA NA ...
## $ total_bal_il         : num    NA NA NA NA NA NA NA NA NA NA NA ...
## $ il_util              : num    NA NA NA NA NA NA NA NA NA NA NA ...
## $ open_rv_12m          : int    NA NA NA NA NA NA NA NA NA NA NA ...
## $ open_rv_24m          : int    NA NA NA NA NA NA NA NA NA NA NA ...
## $ max_bal_bc           : num    NA NA NA NA NA NA NA NA NA NA NA ...
## $ all_util             : num    NA NA NA NA NA NA NA NA NA NA NA ...
## $ total_rev_hi_lim     : num    NA NA NA NA NA NA NA NA 20400 NA 25300 ...
## $ inq-fi              : int    NA NA NA NA NA NA NA NA NA NA NA ...
## $ total_cu_tl          : int    NA NA NA NA NA NA NA NA NA NA NA ...
## $ inq_last_12m         : int    NA NA NA NA NA NA NA NA NA NA NA ...

```

```

# Summary statistics of the dataset
summary(data)

```

```

##           id           member_id           loan_amnt           term
## Min.      : 54734   Min.      : 70473   Min.      : 500   36 months:559045
## 1st Qu.: 9207230   1st Qu.:10877939   1st Qu.: 8000   60 months:239596
## Median :34433372   Median :37095300   Median :13000
## Mean      :32463636   Mean      :35000265   Mean      :14754
## 3rd Qu.:54900100   3rd Qu.:58470266   3rd Qu.:20000

```

```

## Max. :68617057 Max. :73544841 Max. :35000
##
## int_rate emp_title emp_length home_ownership
## Min. : 5.32 : 46354 10+ years:262361 ANY : 2
## 1st Qu.: 9.99 Teacher : 12106 2 years : 70953 MORTGAGE:399151
## Median :12.99 Manager : 10114 < 1 year : 63483 NONE : 45
## Mean :13.24 Registered Nurse: 4996 3 years : 62992 OTHER : 155
## 3rd Qu.:16.20 Owner : 4845 1 year : 51547 OWN : 78789
## Max. :28.99 RN : 4804 5 years : 50157 RENT :320499
## (Other) :715422 (Other) :237148
## annual_inc verification_status
## Min. : 0 Length:798641
## 1st Qu.: 45000 Class :character
## Median : 65000 Mode :character
## Mean : 75014
## 3rd Qu.: 90000
## Max. :9500000
## NA's :4
## desc
## :685241
## : 219
## Debt Consolidation : 13
## Borrower added on 03/17/14 > Debt consolidation<br>: 11
## Borrower added on 01/29/14 > Debt consolidation<br>: 8
## Borrower added on 02/19/14 > Debt consolidation<br>: 8
## (Other) :113141
## purpose title zip_code
## debt_consolidation:471937 Debt consolidation :372740 945xx : 8801
## credit_card :185491 Credit card refinancing:147903 750xx : 8532
## home_improvement : 46543 Home improvement : 36035 112xx : 8399
## other : 38720 Other : 28754 606xx : 7763
## major_purchase : 15579 Debt Consolidation : 14115 300xx : 7334
## small_business : 9372 Major purchase : 10880 100xx : 6866
## (Other) : 30999 (Other) :188214 (Other):750946
## addr_state dti delinq_2yrs earliest_cr_line
## CA :116619 Min. : 0.00 Min. : 0.0000 Length:798641
## NY : 66812 1st Qu.: 11.91 1st Qu.: 0.0000 Class :character
## TX : 64003 Median : 17.66 Median : 0.0000 Mode :character
## FL : 54824 Mean : 18.16 Mean : 0.3145
## IL : 31885 3rd Qu.: 23.95 3rd Qu.: 0.0000
## NJ : 30001 Max. :9999.00 Max. :39.0000
## (Other):434497 NA's :25
## inq_last_6mths mths_since_last_delinq mths_since_last_record
## Min. : 0.0000 Min. : 0.00 Min. : 0.00
## 1st Qu.: 0.0000 1st Qu.: 15.00 1st Qu.: 51.00
## Median : 0.0000 Median : 31.00 Median : 70.00
## Mean : 0.6947 Mean : 34.07 Mean : 70.12
## 3rd Qu.: 1.0000 3rd Qu.: 50.00 3rd Qu.: 92.00
## Max. :33.0000 Max. :188.00 Max. :129.00
## NA's :25 NA's :408818 NA's :675190
## open_acc pub_rec revol_bal revol_util
## Min. : 0.00 Min. : 0.0000 Min. : 0 Min. : 0.00
## 1st Qu.: 8.00 1st Qu.: 0.0000 1st Qu.: 6443 1st Qu.: 37.70
## Median :11.00 Median : 0.0000 Median : 11876 Median : 56.00

```

```

## Mean :11.55 Mean : 0.1953 Mean : 16930 Mean : 55.05
## 3rd Qu.:14.00 3rd Qu.: 0.0000 3rd Qu.: 20839 3rd Qu.: 73.50
## Max. :90.00 Max. :63.0000 Max. :2904836 Max. :892.30
## NA's :25 NA's :25 NA's :2 NA's :454
## total_acc last_credit_pull_d collections_12_mths_ex_med
## Min. : 1.00 Length:798641 Min. : 0.00000
## 1st Qu.: 17.00 Class :character 1st Qu.: 0.00000
## Median : 24.00 Mode :character Median : 0.00000
## Mean : 25.27 Mean : 0.01447
## 3rd Qu.: 32.00 3rd Qu.: 0.00000
## Max. :169.00 Max. :20.00000
## NA's :25 NA's :126
## mths_since_last_major_derog policy_code application_type annual_inc_joint
## Min. : 0.0 1:798641 INDIVIDUAL:798181 Min. : 17950
## 1st Qu.: 27.0 JOINT : 460 1st Qu.: 76167
## Median : 44.0 Median :101886
## Mean : 44.1 Mean :110745
## 3rd Qu.: 61.0 3rd Qu.:133000
## Max. :188.0 Max. :500000
## NA's :599107 NA's :798181
## dti_joint verification_status_joint acc_now_delinq
## Min. : 3.00 Length:798641 Min. : 0.000000
## 1st Qu.:13.32 Class :character 1st Qu.: 0.000000
## Median :17.66 Mode :character Median : 0.000000
## Mean :18.37 Mean : 0.005026
## 3rd Qu.:22.65 3rd Qu.: 0.000000
## Max. :43.86 Max. :14.000000
## NA's :798183 NA's :25
## tot_coll_amt tot_cur_bal open_acc_6m open_il_6m
## Min. : 0.0 Min. : 0 Min. : 0.0 Min. : 0.00
## 1st Qu.: 0.0 1st Qu.: 29861 1st Qu.: 0.0 1st Qu.: 1.00
## Median : 0.0 Median : 80647 Median : 1.0 Median : 2.00
## Mean : 228.3 Mean : 139508 Mean : 1.1 Mean : 2.92
## 3rd Qu.: 0.0 3rd Qu.: 208229 3rd Qu.: 2.0 3rd Qu.: 4.00
## Max. :9152545.0 Max. :8000078 Max. :14.0 Max. :33.00
## NA's :63276 NA's :63276 NA's :779525 NA's :779525
## open_il_12m open_il_24m mths_since_rcnt_il total_bal_il
## Min. : 0.00 Min. : 0.00 Min. : 0.00 Min. : 0
## 1st Qu.: 0.00 1st Qu.: 0.00 1st Qu.: 6.00 1st Qu.: 10164
## Median : 0.00 Median : 1.00 Median : 12.00 Median : 24544
## Mean : 0.76 Mean : 1.67 Mean : 21.09 Mean : 36428
## 3rd Qu.: 1.00 3rd Qu.: 2.00 3rd Qu.: 23.00 3rd Qu.: 47640
## Max. :12.00 Max. :19.00 Max. :363.00 Max. :878459
## NA's :779525 NA's :779525 NA's :780030 NA's :779525
## il_util open_rv_12m open_rv_24m max_bal_bc
## Min. : 0.00 Min. : 0.00 Min. : 0.00 Min. : 0
## 1st Qu.: 58.40 1st Qu.: 0.00 1st Qu.: 1.00 1st Qu.: 2406
## Median : 74.85 Median : 1.00 Median : 2.00 Median : 4502
## Mean : 71.49 Mean : 1.38 Mean : 2.97 Mean : 5878
## 3rd Qu.: 87.70 3rd Qu.: 2.00 3rd Qu.: 4.00 3rd Qu.: 7774
## Max. :223.30 Max. :22.00 Max. :43.00 Max. :83047
## NA's :782007 NA's :779525 NA's :779525 NA's :779525
## all_util total_rev_hi_lim inq-fi total_cu_tl
## Min. : 0.00 Min. : 0 Min. : 0.00 Min. : 0.00

```

```
## 1st Qu.: 47.60    1st Qu.: 13900    1st Qu.: 0.00    1st Qu.: 0.00
## Median : 61.90    Median : 23700    Median : 0.00    Median : 0.00
## Mean   : 60.83    Mean   : 32093    Mean   : 0.95    Mean   : 1.53
## 3rd Qu.: 75.20    3rd Qu.: 39800    3rd Qu.: 1.00    3rd Qu.: 2.00
## Max.   :151.40    Max.   :9999999    Max.   :16.00    Max.   :35.00
## NA's   :779525    NA's   :63276     NA's   :779525    NA's   :779525
## inq_last_12m
## Min.    :-4.00
## 1st Qu.: 0.00
## Median : 2.00
## Mean    : 1.98
## 3rd Qu.: 3.00
## Max.    :32.00
## NA's    :779525
```

General observations: - Many numeric columns show very large ranges, some with maximum values far beyond plausible real-world limits. - Several variables contain placeholder values such as 999, 9999, or -1 indicating missing or unknown information. - A number of columns show repeated NA counts, such as NA's: 25 appearing across unrelated features, suggesting systematic missingness. - Some categorical columns include unusual entries like (Other) or long HTML-like strings.

Loan amount variables (loan_amnt, funded_amnt, funded_amnt_inv) show values ranging from \$500 to tens of millions, which is unrealistic for consumer lending and likely erroneous. Such extreme values need investigation and removal. Interest rate values are within expected ranges. Employment fields contain high-cardinality free-text entries and placeholders. Home ownership includes standard entries but also rare placeholders such as ANY or NONE. Annual income includes extreme values exceeding several million dollars, which are likely incorrect and should be capped or treated as NA. Zip codes and state entries contain unusual placeholders such as Other. Debt-to-income ratio includes 999 or 999.99 as clear placeholders and should be converted to NA. Delinquency-related variables contain many missing values and some extreme maxima that may be unrealistic. Revolving utilization contains impossible values greater than 100 percent, indicating data entry errors. Other credit variables show suspiciously high counts or balances that do not fit typical borrower profiles. Common placeholder patterns detected include 999, 999.0, 999.99, 9999, or textual placeholders like Other. These should be replaced with NA during cleaning. Repeated NA counts (e.g., NA's: 25) across certain variables suggest missing information originating from the same borrower rows. These relationships should be examined. Columns containing URLs or HTML-like descriptions contribute no analytical value and should be removed unless needed for text analysis. Issues to fix include extreme outliers, placeholder values requiring conversion to NA, text cleanup for free-text fields, consistent NA handling across affected rows, and parsing of date fields. Irrelevant columns such as url or verbose descriptions can be dropped. After cleaning, a new summary should be generated to confirm improvements.

```
# vector of placeholder values to replace
placeholders <- c(999, 9999.00, 9999999)

data <- data %>%
  mutate(across(where(is.numeric),
    ~ ifelse(. %in% placeholders, NA, .)))
```

2.1.2 Numerical and Categorical Data

To differentiate between numerical and categorical data, we can use the `supply()` function to check the class of each column in the dataframe. Numerical columns are typically of class “integer” or “numeric”, while categorical columns are of class “factor” or “character”. We can then create separate lists for numerical and categorical columns.

```
# Identify numerical and categorical columns
numerical_cols <- names(data)[sapply(data, function(col) is.numeric(col) ||
                                     is.integer(col))]
categorical_cols <- names(data)[sapply(data, function(col) is.factor(col) ||
                                     is.character(col))]
print("Numerical Columns:")
```

```
## [1] "Numerical Columns:"
```

```
print(numerical_cols)
```

```
## [1] "id" "member_id"
## [3] "loan_amnt" "int_rate"
## [5] "annual_inc" "dti"
## [7] "delinq_2yrs" "inq_last_6mths"
## [9] "mths_since_last_delinq" "mths_since_last_record"
## [11] "open_acc" "pub_rec"
## [13] "revol_bal" "revol_util"
## [15] "total_acc" "collections_12_mths_ex_med"
## [17] "mths_since_last_major_derog" "annual_inc_joint"
## [19] "dti_joint" "acc_now_delinq"
## [21] "tot_coll_amt" "tot_cur_bal"
## [23] "open_acc_6m" "open_il_6m"
## [25] "open_il_12m" "open_il_24m"
## [27] "mths_since_rcnt_il" "total_bal_il"
## [29] "il_util" "open_rv_12m"
## [31] "open_rv_24m" "max_bal_bc"
## [33] "all_util" "total_rev_hi_lim"
## [35] "inq_fi" "total_cu_tl"
## [37] "inq_last_12m"
```

```
print("Categorical Columns:")
```

```
## [1] "Categorical Columns:"
```

```
print(categorical_cols)
```

```
## [1] "term" "emp_title"
## [3] "emp_length" "home_ownership"
## [5] "verification_status" "desc"
## [7] "purpose" "title"
## [9] "zip_code" "addr_state"
## [11] "earliest_cr_line" "last_credit_pull_d"
## [13] "policy_code" "application_type"
## [15] "verification_status_joint"
```

2.1.2.1 Categorical Data Analysis Overview of the categorical columns


```
categorical_cols <- names(data)[
  sapply(data, function(x) is.factor(x) || is.character(x))
]
```

```
categorical_cols
```

```
## [1] "term" "emp_title"
## [3] "emp_length" "home_ownership"
## [5] "verification_status" "desc"
## [7] "purpose" "title"
## [9] "zip_code" "addr_state"
## [11] "earliest_cr_line" "last_credit_pull_d"
## [13] "policy_code" "application_type"
## [15] "verification_status_joint"
```

```
num_categories <- sapply(
  data[categorical_cols],
  function(x) length(unique(na.omit(x)))
)
```

```
num_categories
```

```
##           term           emp_title      emp_length
##           2          274363           12
##   home_ownership verification_status      desc
##           6              3          112107
##           purpose           title      zip_code
##          14          57719           931
##       addr_state earliest_cr_line last_credit_pull_d
##          51             695           103
##       policy_code application_type verification_status_joint
##           1              2              4
```

```
table(data$term)
```

```
##
## 36 months 60 months
##   559045    239596
```

```
# Convert term to numeric loan duration in months
data$term <- trimws(as.character(data$term)) # remove whitespace and drop factor
data$term <- ifelse(grepl("^36", data$term), 36, 60)
data$term <- as.numeric(data$term)
```

```
str(data$term)
```

```
## num [1:798641] 36 36 36 36 36 36 36 36 36 36 ...
```

```
table(data$term)
```

```
##
##      36      60
## 559045 239596
```

Although the variable term has only two distinct values, it is represented as a numeric feature indicating the loan duration in months. This representation reflects the inherent quantitative nature of the variable, as the difference between a 36-month and a 60-month loan corresponds to a real and meaningful difference in exposure time and repayment horizon. Encoding term numerically preserves both the natural ordering and the magnitude of this difference, rather than merely distinguishing between categories. In addition, treating term as a numeric variable results in a more parsimonious model specification and allows us to avoid unnecessary categorical encoding.

Ordinal variables were encoded as ordered factors to preserve their ordinal structure without imposing unjustified metric assumptions. While some factor levels correspond to linear numeric scales, others do not. Therefore, a uniform numeric encoding would introduce inconsistent and potentially misleading distance assumptions. Although this encoding must be reapplied after exporting and reloading the preprocessed CSV file in the training and testing script (we recognized that ordering is not stored in the csv)

```
table(data$emp_length)
```

```
##
## < 1 year    1 year 10+ years    2 years    3 years    4 years    5 years    6 years
##    63483     51547    262361    70953     62992     47330     50157     38621
##    7 years    8 years    9 years      n/a
##    40059     39559     31216     40363
```

```
data$emp_length[data$emp_length == "n/a"] <- NA
data$emp_length <- factor(
  data$emp_length,
  levels = c(
    "< 1 year",
    "1 year",
    "2 years",
    "3 years",
    "4 years",
    "5 years",
    "6 years",
    "7 years",
    "8 years",
    "9 years",
    "10+ years"
  ),
  ordered = TRUE
)
```

```
table(data$emp_length)
```

```
##
## < 1 year    1 year    2 years    3 years    4 years    5 years    6 years    7 years
##    63483     51547     70953     62992     47330     50157     38621     40059
##    8 years    9 years   10+ years
##    39559     31216     262361
```

The variable `emp_length` is treated as an ordered categorical feature representing the borrower's employment tenure. Entries labeled as "n/a" are recoded as missing values, as they indicate the absence of reliable information rather than a valid employment duration. The remaining values are converted into an ordered factor with a logically increasing sequence from less than one year to ten or more years of employment. This encoding preserves the natural ordinal structure of employment tenure without imposing a strictly linear or continuous relationship, allowing the model to exploit relative ordering while avoiding unwarranted assumptions about the magnitude of differences between categories

```
table(data$home_ownership)
```

```
##
##      ANY MORTGAGE      NONE      OTHER      OWN      RENT
##      2    399151      45      155    78789    320499
```

all rows with ANY, NONE and OTHER are dropped due to missing information about these columns and the l

```
data <- data %>%
  filter(!home_ownership %in% c("ANY", "NONE", "OTHER"))
```

drop unused levels

```
data$home_ownership <- droplevels(data$home_ownership)
```

introduce order

```
data$home_ownership <- factor(
  data$home_ownership,
  levels = c("RENT", "MORTGAGE", "OWN"),
  ordered = TRUE
)
table(data$home_ownership)
```

```
##
##      RENT MORTGAGE      OWN
##    320499    399151    78789
```

The variable `home_ownership` is restricted to the categories RENT, MORTGAGE, and OWN, as the remaining categories (ANY, NONE, and OTHER) lack clear semantic interpretation in the context of borrower housing status and occur only infrequently in the dataset. Retaining these categories would introduce ambiguity and unnecessary noise without providing reliable predictive information. After removing the affected observations, unused factor levels are dropped, and the remaining categories are encoded as an ordered categorical variable reflecting increasing levels of housing stability.

```
table(data$purpose)
```

```
##
##      car      credit_card debt_consolidation      educational
##      7921      185426      471871      375
##  home_improvement      house      major_purchase      medical
##      46531      3318      15572      7656
##      moving      other      renewable_energy      small_business
##      4867      38698      505      9360
##      vacation      wedding
##      4242      2097
```

```

data$purpose <- factor(
  data$purpose,
  levels = c(
    "car",
    "credit_card",
    "debt_consolidation",
    "educational",
    "home_improvement",
    "house",
    "major_purchase",
    "medical",
    "moving",
    "other",
    "renewable_energy",
    "small_business",
    "vacation",
    "wedding"
  )
)

```

The variable `purpose` is treated as a nominal categorical feature representing the stated reason for the loan. All observed categories are explicitly defined as factor levels to enforce a consistent category schema and prevent unintended level ordering or implicit recoding during downstream processing. No ordinal relationship is assumed between categories, as loan purposes represent qualitatively different use cases rather than a ranked scale.

```

knitr::kable(
  as.data.frame(table(data$purpose)),
  col.names = c("Purpose", "Count"),
  caption = "Distribution of loan purpose"
)

```

Table 1: Distribution of loan purpose

Purpose	Count
car	7921
credit_card	185426
debt_consolidation	471871
educational	375
home_improvement	46531
house	3318
major_purchase	15572
medical	7656
moving	4867
other	38698
renewable_energy	505
small_business	9360
vacation	4242
wedding	2097

```
# drop row for educational purpose so we can collapse one level
data <- data[data$purpose != "educational", ]
data$purpose <- droplevels(data$purpose)
```

The loan purpose category educational is excluded from further analysis due to its extreme rarity in the dataset after preprocessing. After preprocessing, only a single observation is associated with this category, making it statistically uninformative and unsuitable for reliable model estimation. Retaining such an isolated category would introduce unnecessary noise, increase model complexity, and risk unstable parameter estimates without providing meaningful predictive value. Given the negligible sample size and the absence of generalizable patterns, dropping this category improves model robustness and aligns with the principle of Occam's razor by favoring a simpler and more reliable feature representation.

```
table(data$addr_state)
```

The variable addr_state is excluded from the dataset due to its high cardinality and extreme imbalance across the different categories. While the feature contains geographic information, several states are represented by only a handful of observations (IA with 2 records and ID with 3 records), where others dominate the dataset (CA, NY, TX). This imbalance leads to sparse and unstable category representations, increases model complexity through a large number of dummy variables, and raises the risk of overfitting without guaranteeing meaningful performance gains. Dropping this feature therefore improves model stability and reduces dimensionality later (encoding).

```
data <- data[, !(names(data) %in% "addr_state")]
```

```
table(data$verification_status)
```

```
##
##      Not Verified Source Verified      Verified
##      239979      296600      261485
```

```
data$verification_status <- factor(
  data$verification_status,
  levels = c(
    "Not Verified",
    "Source Verified",
    "Verified"
  ),
  ordered = TRUE
)
```

The variable verification_status is treated as an ordered categorical feature because its categories represent increasing levels of income and information verification. The sequence from Not Verified to Source Verified to Verified reflects a monotonic increase in the reliability of borrower-reported information and, consequently, a decreasing level of informational uncertainty.

A data quality review was performed on the variables **earliest_cr_line** and **last_credit_pull_d** to ensure correct formatting and logical consistency. These variables describe the borrower's credit history timeline and are relevant for credit risk modeling and feature engineering.

```
str(data[, c("earliest_cr_line", "last_credit_pull_d")])
```

```
## 'data.frame': 798064 obs. of 2 variables:
## $ earliest_cr_line : chr "" "" "" "" ...
## $ last_credit_pull_d: chr "Mar-2010" "Jun-2010" "May-2007" "May-2007" ...
```

Both columns were inspected and identified as character strings containing values structured in a “Mon-YYYY” format (e.g., “Oct-2004”).

```
data <- data %>%
  mutate(
    earliest_cr_line_date = my(earliest_cr_line),      # convert Mon-YYYY
    last_credit_pull_d_date = my(last_credit_pull_d)   # convert Mon-YYYY
  )
```

```
str(data[, c("earliest_cr_line_date", "last_credit_pull_d_date")])
```

```
## 'data.frame': 798064 obs. of 2 variables:
## $ earliest_cr_line_date : Date, format: NA NA ...
## $ last_credit_pull_d_date: Date, format: "2010-03-01" "2010-06-01" ...
```

```
summary(data[, c("earliest_cr_line_date", "last_credit_pull_d_date")])
```

```
## earliest_cr_line_date last_credit_pull_d_date
## Min. :1944-01-01 Min. :2007-05-01
## 1st Qu.:1994-06-01 1st Qu.:2016-01-01
## Median :1999-09-01 Median :2016-01-01
## Mean :1998-03-07 Mean :2015-10-26
## 3rd Qu.:2003-05-01 3rd Qu.:2016-01-01
## Max. :2012-11-01 Max. :2016-01-01
## NA's :21 NA's :45
```

```
data %>%
  filter(is.na(earliest_cr_line_date)) %>%
  count(earliest_cr_line) %>%
  arrange(desc(n))
```

```
## earliest_cr_line n
## 1 21
```

```
data %>%
  filter(is.na(earliest_cr_line_date)) %>%
  select(earliest_cr_line, last_credit_pull_d, earliest_cr_line_date, last_credit_pull_d_date) %>%
  head(30)
```

```
## earliest_cr_line last_credit_pull_d earliest_cr_line_date
## 1 Mar-2010 <NA>
## 2 Jun-2010 <NA>
## 3 May-2007 <NA>
## 4 May-2007 <NA>
## 5 Jun-2007 <NA>
## 6 Feb-2015 <NA>
## 7 Aug-2009 <NA>
```

## 8	Feb-2009	<NA>
## 9	May-2007	<NA>
## 10	May-2007	<NA>
## 11		<NA>
## 12	Sep-2014	<NA>
## 13	Apr-2014	<NA>
## 14	May-2007	<NA>
## 15	Jan-2016	<NA>
## 16	Sep-2014	<NA>
## 17	Apr-2010	<NA>
## 18	Jan-2016	<NA>
## 19	Jun-2010	<NA>
## 20	Jun-2007	<NA>
## 21	Feb-2013	<NA>
##	last_credit_pull_d_date	
## 1	2010-03-01	
## 2	2010-06-01	
## 3	2007-05-01	
## 4	2007-05-01	
## 5	2007-06-01	
## 6	2015-02-01	
## 7	2009-08-01	
## 8	2009-02-01	
## 9	2007-05-01	
## 10	2007-05-01	
## 11	<NA>	
## 12	2014-09-01	
## 13	2014-04-01	
## 14	2007-05-01	
## 15	2016-01-01	
## 16	2014-09-01	
## 17	2010-04-01	
## 18	2016-01-01	
## 19	2010-06-01	
## 20	2007-06-01	
## 21	2013-02-01	

Date fields must be correctly formatted and logically consistent to support downstream analysis. Incorrect or contradictory dates can lead to inaccurate derived features and unreliable modeling outcomes. In this context, it is expected that the earliest recorded credit line cannot occur after the most recent credit pull. Therefore, the relationship: **earliest_cr_line_date** **last_credit_pull_d_date** should hold for all valid observations.

```
logic_check <- data %>%
  mutate(
    row_id = row_number(),
    chk_earliest_le_pull = ifelse(
      !is.na(earliest_cr_line_date) & !is.na(last_credit_pull_d_date),
      earliest_cr_line_date <= last_credit_pull_d_date,
      NA
    )
  )

logic_summary <- logic_check %>%
```

```

summarise(
  n_rows      = n(),
  n_pairs     = sum(!is.na(chk_earliest_le_pull)),
  violations  = sum(chk_earliest_le_pull == FALSE, na.rm = TRUE),
  prop_viol   = ifelse(n_pairs == 0, NA_real_, violations / n_pairs)
)

logic_summary

```

```

##   n_rows n_pairs violations prop_viol
## 1 798064 797999          0          0

```

A logical rule comparison was applied to validate the chronological relationship between the two dates. The results indicate that all records with available values satisfy the expected chronological order. No inconsistencies were detected.

```

data <- data %>% select(-earliest_cr_line, -last_credit_pull_d)
names(data)

```

```

## [1] "id"                "member_id"
## [3] "loan_amnt"         "term"
## [5] "int_rate"          "emp_title"
## [7] "emp_length"        "home_ownership"
## [9] "annual_inc"        "verification_status"
## [11] "desc"              "purpose"
## [13] "title"             "zip_code"
## [15] "dti"               "delinq_2yrs"
## [17] "inq_last_6mths"    "mths_since_last_delinq"
## [19] "mths_since_last_record" "open_acc"
## [21] "pub_rec"           "revol_bal"
## [23] "revol_util"        "total_acc"
## [25] "collections_12_mths_ex_med" "mths_since_last_major_derog"
## [27] "policy_code"       "application_type"
## [29] "annual_inc_joint"  "dti_joint"
## [31] "verification_status_joint" "acc_now_delinq"
## [33] "tot_coll_amt"      "tot_cur_bal"
## [35] "open_acc_6m"       "open_il_6m"
## [37] "open_il_12m"       "open_il_24m"
## [39] "mths_since_rcnt_il" "total_bal_il"
## [41] "il_util"           "open_rv_12m"
## [43] "open_rv_24m"       "max_bal_bc"
## [45] "all_util"          "total_rev_hi_lim"
## [47] "inq_fi"            "total_cu_tl"
## [49] "inq_last_12m"      "earliest_cr_line_date"
## [51] "last_credit_pull_d_date"

```

The date variables `earliest_cr_line_date` and `last_credit_pull_d_date` are excluded from the modeling feature set because they are not suitable for prediction at loan application time. While `earliest_cr_line_date` reflects historical information, it introduces temporal information that would require careful transformation to avoid unintended leakage and is largely captured by more interpretable summary variables such as credit history length. The variable `last_credit_pull_d_date` is explicitly post-application and depends on operational processes that occur after the loan request is initiated, making it unavailable for new incoming

applications. Retaining these raw date features would therefore risk semantic leakage, increase preprocessing complexity, and reduce robustness in the unseen evaluation setting.

```
data <- data %>% select(-earliest_cr_line_date, -last_credit_pull_d_date)

logic_emp <- data %>%
  mutate(
    row_id = row_number(),

    # Cases where emp_length is present but emp_title is missing/empty
    inconsistency_emp_missing_title = (emp_title == "" | is.na(emp_title)) & !is.na(emp_length),

    # Cases where emp_title is present but emp_length is missing
    inconsistency_missing_length = !(emp_title == "" | is.na(emp_title)) & is.na(emp_length)
  )

# Summary
logic_emp %>%
  summarise(
    inconsistencies_title_missing = sum(inconsistency_emp_missing_title, na.rm = TRUE),
    inconsistencies_length_missing = sum(inconsistency_missing_length, na.rm = TRUE)
  )

##   inconsistencies_title_missing inconsistencies_length_missing
## 1                               6314                          355
```

The consistency check compares `emp_title` (free-text job title) with `emp_length` (standardized tenure). We find 6,314 observations where `emp_length` is provided but `emp_title` is missing, and 355 observations showing the opposite pattern. This indicates that missingness in these variables is not deterministic and likely driven by different reporting behaviors (non-disclosure, incomplete application fields, or data ingestion gaps). Because there is no defensible rule to reconstruct the missing field from the present one without introducing bias, we do not correct or drop observations based on this inconsistency. Instead, we treat this as a feature-quality consideration. `emp_title` is later removed due to high cardinality and limited interpretability for regression, while `emp_length` is retained as an ordered factor because it is low-dimensional and semantically meaningful.

```
#show unique values of policy code
print(unique(data$policy_code))
```

Policy code only has one value (1). Therefore it is dropped later.

2.2 Feature Selection and Reduction Strategy

Before implementing data cleaning steps, we define explicit criteria for keeping or excluding variables. Each later removal decision is justified by at least one of the following criteria:

1. **Feature semantics / application-time availability (leakage avoidance).** Variables that are not available at loan-application time or that contain post-origination operational outcomes are excluded to avoid leakage and to ensure compatibility with the unseen “reality check” dataset.
2. **Data quality (“too bad to fix”).** Variables with extreme missingness, systematic corruption, implausible ranges, or inconsistent semantics that cannot be repaired reliably are removed.

3. **Preprocessing effort vs. expected benefit.** High-cardinality categorical variables and free-text fields require substantial encoding and normalization effort and can increase overfitting risk. Following an iterative “low-hanging-fruit” approach, we exclude these unless there is clear evidence that they improve performance.
4. **Redundancy and multicollinearity risk.** If multiple variables represent the same underlying concept, we keep the most reliable and interpretable one to reduce noise and prevent unstable model estimates.
5. **Population homogeneity** Subpopulations with fundamentally different feature semantics and extremely small sample sizes (e.g., joint loan applications) are excluded to maintain a homogeneous modeling population. This avoids mixing structurally incompatible observations that would otherwise introduce instability and bias.

These criteria are applied consistently in the subsequent preprocessing steps.

2.3 Data Wrangling

Data wrangling is the process of cleaning and transforming raw data into a format that is suitable for analysis. This includes handling missing values, correcting duplicate entries, and addressing any incorrect values in the dataset. The first chapters of this project are per definition also part of data wrangling, but were separated for better clarity in the code. Cleaning and transforming the data is an iterative process that requires careful consideration of the specific characteristics of the dataset and the goals of the analysis. The better the cleaning and transforming is done, the more reliable and accurate the results of the analysis will be.

2.3.1 Duplicated Values

Duplicated values can be entire rows which are duplicated or, in this case, duplication of the id and/or member_id columns. If these rows can be identified, the duplicated row should be dropped. If multiple duplicated rows are kept in the dataset, the model would learn to overfit on these rows, leading to a worse performance on unseen data.

```
# Check for duplicate rows
duplicate_rows <- data[duplicated(data), ]
print(paste("Number of duplicate rows:", nrow(duplicate_rows)))
```

```
## [1] "Number of duplicate rows: 0"
```

```
# Check for duplicate IDs
duplicate_ids <- data[duplicated(data$id)]
print(paste("Number of duplicate IDs:", sum(duplicate_ids)))
```

```
## [1] "Number of duplicate IDs: 0"
```

```
# Check for duplicated member IDs
duplicate_member_ids <- data[duplicated(data$member_id)]
print(paste("Number of duplicate member IDs:", sum(duplicate_member_ids)))
```

```
## [1] "Number of duplicate member IDs: 0"
```

There are no duplicated rows to take care of.

purpose and title have a similar meaning defined in the dictionary. To get an overview of these two columns, they are checked against each other.

```
purpose_chr <- as.character(data$purpose)
title_chr   <- as.character(data$title)

# Exact Match
purpose_title_exact_match <- mean(purpose_chr == title_chr, na.rm = TRUE)

# Fuzzy Match
purpose_title_fuzzy_match <- mean(
  str_to_lower(purpose_chr) == str_to_lower(title_chr),
  na.rm = TRUE
)

purpose_title_exact_match
```

```
## [1] 0.001078861
```

```
purpose_title_fuzzy_match
```

```
## [1] 0.04209938
```

```
unique_purpose <- length(unique(purpose_chr))
unique_title  <- length(unique(title_chr))

unique_purpose
```

```
## [1] 13
```

```
unique_title
```

```
## [1] 57335
```

```
# Show all unique values of purpose and the count
table(data$purpose)
```

```
##
##           car           credit_card debt_consolidation  home_improvement
##           7921           185426           471871           46531
##           house      major_purchase           medical           moving
##           3318           15572           7656           4867
##           other  renewable_energy      small_business           vacation
##           38698           505           9360           4242
##           wedding
##           2097
```

Since purpose contains more clear classifications, title was dropped. As a free text field, title does not provide a clear purpose. (“My New Car”, “CAR”, “Buy a car”, “Car loan!!”, “Auto”, “Car debt consolidation”)

```
data <- data %>%
  select(-title)
names(data)
```

```
## [1] "id" "member_id"
## [3] "loan_amnt" "term"
## [5] "int_rate" "emp_title"
## [7] "emp_length" "home_ownership"
## [9] "annual_inc" "verification_status"
## [11] "desc" "purpose"
## [13] "zip_code" "dti"
## [15] "delinq_2yrs" "inq_last_6mths"
## [17] "mths_since_last_delinq" "mths_since_last_record"
## [19] "open_acc" "pub_rec"
## [21] "revol_bal" "revol_util"
## [23] "total_acc" "collections_12_mths_ex_med"
## [25] "mths_since_last_major_derog" "policy_code"
## [27] "application_type" "annual_inc_joint"
## [29] "dti_joint" "verification_status_joint"
## [31] "acc_now_delinq" "tot_coll_amt"
## [33] "tot_cur_bal" "open_acc_6m"
## [35] "open_il_6m" "open_il_12m"
## [37] "open_il_24m" "mths_since_rcnt_il"
## [39] "total_bal_il" "il_util"
## [41] "open_rv_12m" "open_rv_24m"
## [43] "max_bal_bc" "all_util"
## [45] "total_rev_hi_lim" "inq_fi"
## [47] "total_cu_tl" "inq_last_12m"
```

2.3.2 Missing Values

In the summary, we can see that there are several columns with a high number of missing values. Depending on the percentage of missing values, we can decide to either drop the column or impute the missing values. For columns with a low percentage of missing values, we can use techniques such as mean/median imputation for numerical columns or mode imputation for categorical columns. For columns with a high percentage of missing values, it may be better to drop the column altogether, as imputing too many missing values can introduce bias into the dataset.

```
# Calculate the amount and percentage of missing values for each column
missing_values <- data.frame(
  column = names(data),
  missing_count = sapply(data, function(col) sum(is.na(col))),
  missing_percentage = round(sapply(data, function(col) mean(is.na(col)) * 100), 2)
)

knitr::kable(
  missing_values,
  caption = "Missing values per column after preprocessing"
)
```

Table 2: Missing values per column after preprocessing

	column	missing_count	missing_percentage
id	id	0	0.00
member_id	member_id	0	0.00
loan_amnt	loan_amnt	0	0.00
term	term	0	0.00
int_rate	int_rate	0	0.00
emp_title	emp_title	0	0.00
emp_length	emp_length	40352	5.06
home_ownership	home_ownership	0	0.00
annual_inc	annual_inc	0	0.00
verification_status	verification_status	0	0.00
desc	desc	0	0.00
purpose	purpose	0	0.00
zip_code	zip_code	0	0.00
dti	dti	2	0.00
delinq_2yrs	delinq_2yrs	21	0.00
inq_last_6mths	inq_last_6mths	21	0.00
mths_since_last_delinq	mths_since_last_delinq	408502	51.19
mths_since_last_record	mths_since_last_record	674687	84.54
open_acc	open_acc	21	0.00
pub_rec	pub_rec	21	0.00
revol_bal	revol_bal	67	0.01
revol_util	revol_util	446	0.06
total_acc	total_acc	21	0.00
collections_12_mths_ex_med	collections_12_mths_ex_med	112	0.01
mths_since_last_major_derog	mths_since_last_major_derog	598557	75.00
policy_code	policy_code	0	0.00
application_type	application_type	0	0.00
annual_inc_joint	annual_inc_joint	797604	99.94
dti_joint	dti_joint	797606	99.94
verification_status_joint	verification_status_joint	0	0.00
acc_now_delinq	acc_now_delinq	21	0.00
tot_coll_amt	tot_coll_amt	62799	7.87
tot_cur_bal	tot_cur_bal	62795	7.87
open_acc_6m	open_acc_6m	778948	97.60
open_il_6m	open_il_6m	778948	97.60
open_il_12m	open_il_12m	778948	97.60
open_il_24m	open_il_24m	778948	97.60
mths_since_rcnt_il	mths_since_rcnt_il	779453	97.67
total_bal_il	total_bal_il	778949	97.60
il_util	il_util	781430	97.92
open_rv_12m	open_rv_12m	778948	97.60
open_rv_24m	open_rv_24m	778948	97.60
max_bal_bc	max_bal_bc	778952	97.61
all_util	all_util	778948	97.60
total_rev_hi_lim	total_rev_hi_lim	62784	7.87
inq_fi	inq_fi	778948	97.60
total_cu_tl	total_cu_tl	778948	97.60
inq_last_12m	inq_last_12m	778948	97.60

Especially for high columns with high NA-counts, one should contact the data manager to understand the

logic behind the missing values. It is possible that there was a mistake in the SQL-output or that there is a specific reason why these values are missing. Since this is not possible in this project, we have to make assumptions based on the data dictionary and the summary statistics. NA-values can therefore either be imputed using different imputation methods or dropped.

We have to be careful to drop all rows or columns with NA-values, since sometimes there is a logic behind a missing value. For example, in the column “mths_since_last_delinq”, a missing value could mean that the person has never been delinquent. Dropping these rows would lead to a loss of valuable information. Therefore, we have to carefully analyze each column with missing values and decide on a case-by-case basis how to handle them.

- delinq_2yrs: There are 21 missing values, which is a very small percentage of the total data. These values could be imputed or dropped after further analysis.
- inq_last_6mths: There are 21 missing values. These values could be imputed or dropped after further analysis.
- open_acc: There are 21 missing values. These values could be imputed or dropped after further analysis.
- pub_rec: There are 21 missing values. These values could be imputed or dropped after further analysis.
- total_acc: There are 21 missing values. These values could be imputed or dropped after further analysis.
- acc_now_delinq: There are 21 missing values. These values could be imputed or dropped after further analysis.
- mths_since_last_delinq: There are 408'818 (51.19%) missing values. These missing values likely indicate that the borrower has never been delinquent, yet further in-depth analysis is needed. Possible options for imputation in the latter case would be imputation with -1 or to change the datatype to a categorical variable with levels.
- mths_since_last_record: There are 675'190 (84.54%) missing values. These missing values likely indicate that the borrower has no public records, yet further in-depth analysis is needed. Possible options for imputation in the latter case would be imputation with -1 or to change the datatype to a categorical variable with levels.
- revol_bal: There are 2 missing values. These values could be imputed or dropped after further analysis.
- revol_util: There are 454 missing values. These values could be imputed or dropped after further analysis.
- collections_12_mths_ex_med: There are 126 missing values. These values could be imputed or dropped after further analysis.
- mths_since_last_major_derog: There are 599'107 (75.02%) missing values. These missing values likely indicate that the borrower has no major derogatory marks, yet further in-depth analysis is needed. Possible options for imputation in the latter case would be imputation with -1 or to change the datatype to a categorical variable with levels.
- annual_inc_joint, dti_joint, verification_status_joint: These columns have higher NA-counts due to the fact that they are dependent on the application_type being “JOINT”. Dropping these columns would lead to a loss of valuable information for joint applications. A comparison model would have to be made to check whether or not the model gets better when keeping this information for prediction or not.
- tot_coll_amt: There are 63'276 (7.92%) missing values. These values could be imputed or dropped after further analysis.
- tot_cur_bal: There are 63'276 (7.92%) missing values. These values could be imputed or dropped after further analysis.
- total_rev_hi_lim: There are 63'276 (7.92%) missing values. These values could be imputed or dropped after further analysis.
- open_acc_6m, open_il_6m, open_il_12m, open_il_24m, mths_since_rcnt_il, total_bal_il, il_util, open_rv_12m, open_rv_24m, max_bal_bc, all_util, inq_fi, total_cu_tl, inq_last_12m: All of these columns have a high number of missing values (97.61%). Imputing these missing values would likely introduce bias into the dataset. Therefore, it is better to drop these columns altogether.

We can filter out the columns with >95% missing values. Imputing these will introduce bias into the dataset. Therefore we will drop these columns.

```
# Columns to drop due to high percentage of missing values
columns_to_drop_na <- c("open_acc_6m",
                        "open_il_6m",
                        "open_il_12m",
                        "open_il_24m",
                        "mths_since_rcnt_il",
                        "total_bal_il",
                        "il_util",
                        "open_rv_12m",
                        "open_rv_24m",
                        "max_bal_bc",
                        "all_util",
                        "inq-fi",
                        "total_cu_tl",
                        "inq_last_12m")

# Drop the columns from the dataset
cleaned_data <- data[, -match(columns_to_drop_na, names(data))]
dim(cleaned_data)
```

```
## [1] 798064    34
```

It is noticeable that there are exactly 25 rows with NA-values in multiple columns. If these 25 rows are always the same, it would make sense to drop these rows altogether instead of imputing the missing values.

```
# Identify rows with NA values in the specified columns
na_rows <- data %>%
  filter(is.na(delinq_2yrs) |
         is.na(inq_last_6mths) |
         is.na(open_acc) |
         is.na(pub_rec) |
         is.na(total_acc) |
         is.na(acc_now_delinq))
print("Rows with NA values in specified columns:")
```

```
## [1] "Rows with NA values in specified columns:"
```

```
print(na_rows)
```

```
##      id member_id loan_amnt term int_rate
## 1  83489     83471     2600   36     8.38
## 2  77792     77764     3000   36     9.01
## 3  72176     70868     2525   36     9.33
## 4  72819     72816     7200   36    10.28
## 5  83979     83974     3000   36     7.43
## 6  70686     70681     5000   36     7.75
## 7  84670     79576     5000   36     7.75
## 8  73003     70651     3200   36     9.96
## 9  91023     70879     1900   36     9.64
## 10 77757     70626     3000   36     9.33
```

## 11	71823	70473	4350	36	8.07
## 12	85781	85727	1400	36	10.91
## 13	83185	83132	1000	36	7.12
## 14	72197	72194	3900	36	9.33
## 15	74323	74301	6500	36	9.64
## 16	72998	72992	1000	36	9.64
## 17	85818	85802	5000	36	11.22
## 18	76629	76623	1275	36	12.49
## 19	85675	85667	1000	36	14.07
## 20	81085	80973	10500	36	11.22
## 21	73582	73096	3500	36	10.28
##				emp_title	emp_length home_ownership
## 1				College Pro Painters	3 years MORTGAGE
## 2				Tanks Tavern	< 1 year RENT
## 3					< 1 year RENT
## 4					< 1 year RENT
## 5				NC	1 year MORTGAGE
## 6				Homemaker	10+ years MORTGAGE
## 7				Diamond Management and Technology Consultants	10+ years MORTGAGE
## 8					< 1 year MORTGAGE
## 9				Scheduall	1 year MORTGAGE
## 10					< 1 year OWN
## 11					< 1 year RENT
## 12				Stanford University Libraries, LOCKSS Project	< 1 year RENT
## 13				Mana Products	6 years RENT
## 14					< 1 year RENT
## 15				Air Force	< 1 year RENT
## 16				Halping hands company inc.	< 1 year RENT
## 17				Compensation Solutions	< 1 year RENT
## 18				Infinitely law group	1 year RENT
## 19				Macy's	10+ years RENT
## 20				Town of Plainville	3 years RENT
## 21					< 1 year RENT
##	annual_inc	verification_status			
## 1	6500	Not Verified			
## 2	35000	Not Verified			
## 3	110000	Not Verified			
## 4	95000	Not Verified			
## 5	80000	Not Verified			
## 6	70000	Not Verified			
## 7	300000	Not Verified			
## 8	150000	Not Verified			
## 9	100000	Not Verified			
## 10	20000	Not Verified			
## 11	120000	Not Verified			
## 12	40000	Not Verified			
## 13	50000	Not Verified			
## 14	98000	Not Verified			
## 15	20000	Not Verified			
## 16	12000	Not Verified			
## 17	28000	Not Verified			
## 18	40000	Not Verified			
## 19	25000	Not Verified			
## 20	60000	Not Verified			


```

## 21      180000      Not Verified
##
## 1
## 2
## 3
## 4      I am consolidating 2 loans that carry 1
## 5
## 6
## 7
## 8      Hi, I am looking to renovate my bathroom as in the next year I want to sell my home. They say y
## 9
## 10
## 11
## 12      I'm starting full-time wo
## 13
## 14
## 15 I'm in the military and before I came in I racked up a large credit card debt. With all the differ
## 16
## 17
## 18
## 19
## 20
## 21      I am getting marrie
##
##      purpose zip_code   dti delinq_2yrs inq_last_6mths
## 1      other      806xx  6.46           NA           NA
## 2      other      665xx 10.00           NA           NA
## 3      other      100xx 10.00           NA           NA
## 4      other      100xx 10.00           NA           NA
## 5      other      070xx  0.39           NA           NA
## 6      other      068xx  8.81           NA           NA
## 7      other      701xx  5.38           NA           NA
## 8      other      019xx 10.00           NA           NA
## 9 debt_consolidation 333xx 10.00           NA           NA
## 10     other      024xx 10.00           NA           NA
## 11     other      024xx  4.00           NA           NA
## 12     other      652xx  8.61           NA           NA
## 13     other      100xx  1.10           NA           NA
## 14     other      100xx 10.00           NA           NA
## 15     other      064xx 10.00           NA           NA
## 16     other      021xx 10.00           NA           NA
## 17     other      078xx  3.51           NA           NA
## 18     other      021xx 10.00           NA           NA
## 19     other      080xx 16.27           NA           NA
## 20     other      061xx 19.50           NA           NA
## 21     other      100xx 10.00           NA           NA
##
##      mths_since_last_delinq mths_since_last_record open_acc pub_rec revol_bal
## 1      NA                  NA                  NA      NA      0
## 2      NA                  NA                  NA      NA      0
## 3      NA                  NA                  NA      NA      0
## 4      NA                  NA                  NA      NA      0
## 5      NA                  NA                  NA      NA      0
## 6      NA                  NA                  NA      NA      0
## 7      NA                  NA                  NA      NA      0
## 8      NA                  NA                  NA      NA      0

```

## 9		NA		NA	NA	NA	0
## 10		NA		NA	NA	NA	0
## 11		NA		NA	NA	NA	0
## 12		NA		NA	NA	NA	0
## 13		NA		NA	NA	NA	0
## 14		NA		NA	NA	NA	0
## 15		NA		NA	NA	NA	0
## 16		NA		NA	NA	NA	0
## 17		NA		NA	NA	NA	0
## 18		NA		NA	NA	NA	0
## 19		NA		NA	NA	NA	0
## 20		NA		NA	NA	NA	0
## 21		NA		NA	NA	NA	0
##	revol_util	total_acc	collections_12_mths_ex_med	mths_since_last_major_derog			
## 1	NA	NA	NA	NA			
## 2	NA	NA	NA	NA			
## 3	NA	NA	NA	NA			
## 4	NA	NA	NA	NA			
## 5	NA	NA	NA	NA			
## 6	NA	NA	NA	NA			
## 7	NA	NA	NA	NA			
## 8	NA	NA	NA	NA			
## 9	NA	NA	NA	NA			
## 10	NA	NA	NA	NA			
## 11	NA	NA	NA	NA			
## 12	NA	NA	NA	NA			
## 13	NA	NA	NA	NA			
## 14	NA	NA	NA	NA			
## 15	NA	NA	NA	NA			
## 16	NA	NA	NA	NA			
## 17	NA	NA	NA	NA			
## 18	NA	NA	NA	NA			
## 19	NA	NA	NA	NA			
## 20	NA	NA	NA	NA			
## 21	NA	NA	NA	NA			
##	policy_code	application_type	annual_inc_joint	dti_joint			
## 1	1	INDIVIDUAL	NA	NA			
## 2	1	INDIVIDUAL	NA	NA			
## 3	1	INDIVIDUAL	NA	NA			
## 4	1	INDIVIDUAL	NA	NA			
## 5	1	INDIVIDUAL	NA	NA			
## 6	1	INDIVIDUAL	NA	NA			
## 7	1	INDIVIDUAL	NA	NA			
## 8	1	INDIVIDUAL	NA	NA			
## 9	1	INDIVIDUAL	NA	NA			
## 10	1	INDIVIDUAL	NA	NA			
## 11	1	INDIVIDUAL	NA	NA			
## 12	1	INDIVIDUAL	NA	NA			
## 13	1	INDIVIDUAL	NA	NA			
## 14	1	INDIVIDUAL	NA	NA			
## 15	1	INDIVIDUAL	NA	NA			
## 16	1	INDIVIDUAL	NA	NA			
## 17	1	INDIVIDUAL	NA	NA			
## 18	1	INDIVIDUAL	NA	NA			

## 19	1	INDIVIDUAL		NA	NA
## 20	1	INDIVIDUAL		NA	NA
## 21	1	INDIVIDUAL		NA	NA
##	verification_status_joint acc_now_delinq tot_coll_amt tot_cur_bal				
## 1			NA	NA	NA
## 2			NA	NA	NA
## 3			NA	NA	NA
## 4			NA	NA	NA
## 5			NA	NA	NA
## 6			NA	NA	NA
## 7			NA	NA	NA
## 8			NA	NA	NA
## 9			NA	NA	NA
## 10			NA	NA	NA
## 11			NA	NA	NA
## 12			NA	NA	NA
## 13			NA	NA	NA
## 14			NA	NA	NA
## 15			NA	NA	NA
## 16			NA	NA	NA
## 17			NA	NA	NA
## 18			NA	NA	NA
## 19			NA	NA	NA
## 20			NA	NA	NA
## 21			NA	NA	NA
##	open_acc_6m open_il_6m open_il_12m open_il_24m mths_since_rcnt_il				
## 1	NA	NA	NA	NA	NA
## 2	NA	NA	NA	NA	NA
## 3	NA	NA	NA	NA	NA
## 4	NA	NA	NA	NA	NA
## 5	NA	NA	NA	NA	NA
## 6	NA	NA	NA	NA	NA
## 7	NA	NA	NA	NA	NA
## 8	NA	NA	NA	NA	NA
## 9	NA	NA	NA	NA	NA
## 10	NA	NA	NA	NA	NA
## 11	NA	NA	NA	NA	NA
## 12	NA	NA	NA	NA	NA
## 13	NA	NA	NA	NA	NA
## 14	NA	NA	NA	NA	NA
## 15	NA	NA	NA	NA	NA
## 16	NA	NA	NA	NA	NA
## 17	NA	NA	NA	NA	NA
## 18	NA	NA	NA	NA	NA
## 19	NA	NA	NA	NA	NA
## 20	NA	NA	NA	NA	NA
## 21	NA	NA	NA	NA	NA
##	total_bal_il il_util open_rv_12m open_rv_24m max_bal_bc all_util				
## 1	NA	NA	NA	NA	NA
## 2	NA	NA	NA	NA	NA
## 3	NA	NA	NA	NA	NA
## 4	NA	NA	NA	NA	NA
## 5	NA	NA	NA	NA	NA
## 6	NA	NA	NA	NA	NA

```
## 7      NA      NA      NA      NA      NA      NA
## 8      NA      NA      NA      NA      NA      NA
## 9      NA      NA      NA      NA      NA      NA
## 10     NA      NA      NA      NA      NA      NA
## 11     NA      NA      NA      NA      NA      NA
## 12     NA      NA      NA      NA      NA      NA
## 13     NA      NA      NA      NA      NA      NA
## 14     NA      NA      NA      NA      NA      NA
## 15     NA      NA      NA      NA      NA      NA
## 16     NA      NA      NA      NA      NA      NA
## 17     NA      NA      NA      NA      NA      NA
## 18     NA      NA      NA      NA      NA      NA
## 19     NA      NA      NA      NA      NA      NA
## 20     NA      NA      NA      NA      NA      NA
## 21     NA      NA      NA      NA      NA      NA
##      total_rev_hi_lim inq_fi total_cu_tl inq_last_12m
## 1      NA      NA      NA      NA
## 2      NA      NA      NA      NA
## 3      NA      NA      NA      NA
## 4      NA      NA      NA      NA
## 5      NA      NA      NA      NA
## 6      NA      NA      NA      NA
## 7      NA      NA      NA      NA
## 8      NA      NA      NA      NA
## 9      NA      NA      NA      NA
## 10     NA      NA      NA      NA
## 11     NA      NA      NA      NA
## 12     NA      NA      NA      NA
## 13     NA      NA      NA      NA
## 14     NA      NA      NA      NA
## 15     NA      NA      NA      NA
## 16     NA      NA      NA      NA
## 17     NA      NA      NA      NA
## 18     NA      NA      NA      NA
## 19     NA      NA      NA      NA
## 20     NA      NA      NA      NA
## 21     NA      NA      NA      NA
```

As seen in the output, the 25 rows with NA-values are indeed always the same. Therefore we filter them out of the cleaned dataset.

```
# Remove rows with id values found in na_rows
cleaned_data <- cleaned_data %>%
  filter(!id %in% na_rows$id)
#dim(cleaned_data)
```

A similar trend can be seen in tot_call_amt, tot_cur_bal and total_rev_hi_lim, where exactly 63'276 rows have NA-values in all three columns. We check if these rows are the same and filter them out of the cleaned dataset if so.

```
# Count how many rows have ALL THREE columns missing
na_all3 <- cleaned_data %>%
  filter(
```

```

    is.na(tot_coll_amt) &
    is.na(tot_cur_bal) &
    is.na(total_rev_hi_lim)
  )

# Print how many rows have all 3 NA
cat("Rows where ALL THREE (tot_coll_amt, tot_cur_bal, total_rev_hi_lim) are NA:",
    nrow(na_all3), "\n")

```

```
## Rows where ALL THREE (tot_coll_amt, tot_cur_bal, total_rev_hi_lim) are NA: 62758
```

```

# Drop these rows from the dataset
cleaned_data <- cleaned_data %>%
  filter(
    !(is.na(tot_coll_amt) &
      is.na(tot_cur_bal) &
      is.na(total_rev_hi_lim))
  )

# Show how many rows remain
cat("Remaining rows after dropping these:", nrow(cleaned_data), "\n")

```

```
## Remaining rows after dropping these: 735285
```

The variables `tot_coll_amt`, `tot_cur_bal`, and `total_rev_hi_lim` represent a borrower's total collection amount, total current balances, and total revolving credit limit. These three features describe the borrower's overall credit utilization profile, which is essential for assessing creditworthiness. In the cleaned dataset, 63'251 rows had all three of these variables missing simultaneously. Such rows were removed because: - They contain no usable information about the borrower's credit balance. - With all three fields missing, the borrower's credit exposure cannot be approximated or imputed reliably. - The missingness is structural, not random. - Borrowers with all three fields missing typically reflect incomplete or corrupted credit bureau data, not normal variation. - Imputation is not appropriate when the entire feature set is missing. - Imputing values would inject arbitrary numbers into critical financial variables, degrading model validity. - They represent a very small portion of the overall dataset, so dropping them does not meaningfully reduce sample size or introduce bias. For these reasons, rows where `tot_coll_amt`, `tot_cur_bal`, and `total_rev_hi_lim` were all NA were excluded from further analysis.

2.3.3 Joint accounts

```

unique(cleaned_data$application_type)

## [1] INDIVIDUAL JOINT
## Levels: INDIVIDUAL JOINT

sum(cleaned_data$application_type == "JOINT", na.rm = TRUE)

## [1] 460

```

```
mean(cleaned_data$application_type == "JOINT", na.rm = TRUE) * 100
```

```
## [1] 0.06256078
```

In the dataset, only 460 rows are labeled as joint applications, corresponding to 0.06% of the total available observations. These records were removed, along with all joint-specific variables, for the following reasons:

2.3.3.1 Extremely Small Sample Size Joint applications represent only 460 observations out of the full dataset.

With such a small proportion (0.06%), this subgroup is too small to:

- learn stable statistical patterns,
- contribute meaningfully to model performance,
- or support reliable parameter estimation.

Including a subgroup of this size would introduce noise rather than useful signal.

2.3.3.2 Structural Differences Between Joint and Individual Applications Joint applications reflect the combined financial profile of two borrowers.

This leads to fundamental structural differences compared with single-applicant data:

- income fields may represent combined household income,
- debt-to-income ratios follow different calculations,
- credit history may aggregate information from two individuals,
- and several features are exclusive to joint borrowers (e.g., `annual_inc_joint`, `dti_joint`, `verification_status_joint`).

These differences produce incompatible distributions that violate the assumption of data homogeneity needed for model training.

2.3.3.3 Joint-Specific Columns Are Not Usable for Modeling Variables such as `annual_inc_joint`, `dti_joint`, and `verification_status_joint` apply only to joint applications.

For over 99.9% of rows, these fields are always NA, making them:

- unsuitable for imputation,
- non-informative for the main model,
- and a source of unnecessary sparsity and complexity.

Removing these columns simplifies preprocessing while avoiding misleading predictors.

2.3.3.4 Including Joint Records Reduces Model Stability Joint applications follow different underwriting rules and show distinct patterns of missingness, income distribution, and credit utilisation.

Mixing these with single applicants can distort:

- learned feature relationships,
- decision tree splits,
- regression coefficients,
- and probability calibration.

Excluding them leads to a more stable and generalizable model for the primary population of interest.

2.3.3.5 Why Not Build a Separate Model for Joint Applications? Although joint applications differ structurally, creating a standalone model is not feasible for the following reasons:

- The sample size ($n = 460$) is far too small to support:
 - model training,
 - validation,
 - meaningful performance evaluation,
 - or hyperparameter tuning.
- A separate model would suffer from:
 - overfitting,
 - unstable parameter estimates,
 - unreliable predictions,
 - and inability to generalize to new joint borrowers.
- Joint applications are too rare to justify maintaining a dedicated modeling pipeline, monitoring process, or deployment flow.

For these reasons, a separate joint-application model cannot be developed in a reliable or operationally meaningful way.

```
# List of joint-specific columns (from data dictionary)
joint_cols <- c(
  "annual_inc_joint",
  "dti_joint",
  "verification_status_joint"
)

# Only drop columns that actually exist in the dataset
cols_to_drop <- intersect(joint_cols, names(cleaned_data))

# Drop joint-specific columns
cleaned_data <- cleaned_data %>%
  select(-all_of(cols_to_drop))

# Drop rows where application_type == "Joint"
cleaned_data <- cleaned_data %>%
  filter(application_type != "JOINT")

# Print remaining number of rows
cat("Rows remaining after removing joint applications:", nrow(cleaned_data), "\n")
```

```
## Rows remaining after removing joint applications: 734825
```

```
# drop column application_type as it now only contains one value
cleaned_data <- cleaned_data %>%
  select(-application_type)
dim(cleaned_data) # 734'863 rows and 34 columns now
```

```
## [1] 734825      30
```

```

# Calculate the amount and percentage of missing values for each column
missing_values <- data.frame(
  column = names(cleaned_data),
  missing_count = sapply(cleaned_data, function(col) sum(is.na(col))),
  missing_percentage = round(sapply(cleaned_data, function(col) mean(is.na(col)) * 100), 2)
)

print("Missing values per column:")

```

```
## [1] "Missing values per column:"
```

```
print(missing_values)
```

```
##                                column missing_count
## id                                id              0
## member_id                        member_id        0
## loan_amnt                        loan_amnt         0
## term                             term             0
## int_rate                         int_rate          0
## emp_title                        emp_title          0
## emp_length                       emp_length      38452
## home_ownership                   home_ownership     0
## annual_inc                      annual_inc         0
## verification_status              verification_status 0
## desc                             desc              0
## purpose                          purpose           0
## zip_code                        zip_code            0
## dti                              dti               0
## delinq_2yrs                     delinq_2yrs         0
## inq_last_6mths                  inq_last_6mths        0
## mths_since_last_delinq          mths_since_last_delinq 367842
## mths_since_last_record          mths_since_last_record 615508
## open_acc                        open_acc            0
## pub_rec                         pub_rec             0
## revol_bal                       revol_bal           62
## revol_util                      revol_util          349
## total_acc                      total_acc            0
## collections_12_mths_ex_med      collections_12_mths_ex_med 0
## mths_since_last_major_derog     mths_since_last_major_derog 535447
## policy_code                    policy_code          0
## acc_now_delinq                 acc_now_delinq        0
## tot_coll_amt                   tot_coll_amt         20
## tot_cur_bal                    tot_cur_bal          16
## total_rev_hi_lim               total_rev_hi_lim       5
##                                missing_percentage
## id                                0.00
## member_id                        0.00
## loan_amnt                        0.00
## term                             0.00
## int_rate                         0.00
## emp_title                        0.00
## emp_length                       5.23
```



```
## home_ownership          0.00
## annual_inc              0.00
## verification_status     0.00
## desc                    0.00
## purpose                 0.00
## zip_code                0.00
## dti                     0.00
## delinq_2yrs             0.00
## inq_last_6mths          0.00
## mths_since_last_delinq  50.06
## mths_since_last_record  83.76
## open_acc                0.00
## pub_rec                 0.00
## revol_bal               0.01
## revol_util              0.05
## total_acc               0.00
## collections_12_mths_ex_med 0.00
## mths_since_last_major_derog 72.87
## policy_code             0.00
## acc_now_delinq          0.00
## tot_coll_amt            0.00
## tot_cur_bal             0.00
## total_rev_hi_lim        0.00
```

2.4 Data Anomalies

To check for data anomalies one should know the logical boundaries of each column. For example, a negative annual income would be an anomaly. Since we do not have this information, we can only check for obvious anomalies such as negative values in columns that should only contain positive values. Furthermore, we can check for outliers using boxplots or z-scores. Outliers can be defined as values that are more than 3 standard deviations away from the mean. These outliers can be investigated further to determine if they are valid data points or if they should be removed from the dataset. Next, one can check for inconsistent data entries. For example, if a column contains categorical data, one can check for typos or inconsistent capitalization. These inconsistencies can be corrected to ensure that the data is consistent throughout the dataset. Finally, one can check for logical inconsistencies in the data. For example, if a person has no delinquencies in the last 24 months, but has a high number of delinquencies in the last 2 years, this would be a logical inconsistency. These inconsistencies can be investigated further to determine if they are valid data points or if they should be corrected or removed from the dataset.

```
# Check for negative values in numerical columns
numerical_cols <- names(cleaned_data)[sapply(cleaned_data, is.numeric)]
negative_values <- sapply(cleaned_data[numerical_cols], function(col) sum(col < 0, na.rm = TRUE))
print("Negative values in numerical columns:")
```

```
## [1] "Negative values in numerical columns:"
```

```
print(negative_values)
```

```
##          id          member_id
##          0              0
## loan_amnt          term
##          0              0
```

```
##          int_rate          annual_inc
##          0              0
##          dti            delinq_2yrs
##          0              0
##          inq_last_6mths    mths_since_last_delinq
##          0              0
##          mths_since_last_record    open_acc
##          0              0
##          pub_rec          revol_bal
##          0              0
##          revol_util        total_acc
##          0              0
##          collections_12_mths_ex_med    mths_since_last_major_derog
##          0              0
##          acc_now_delinq          tot_coll_amt
##          0              0
##          tot_cur_bal          total_rev_hi_lim
##          0              0
```

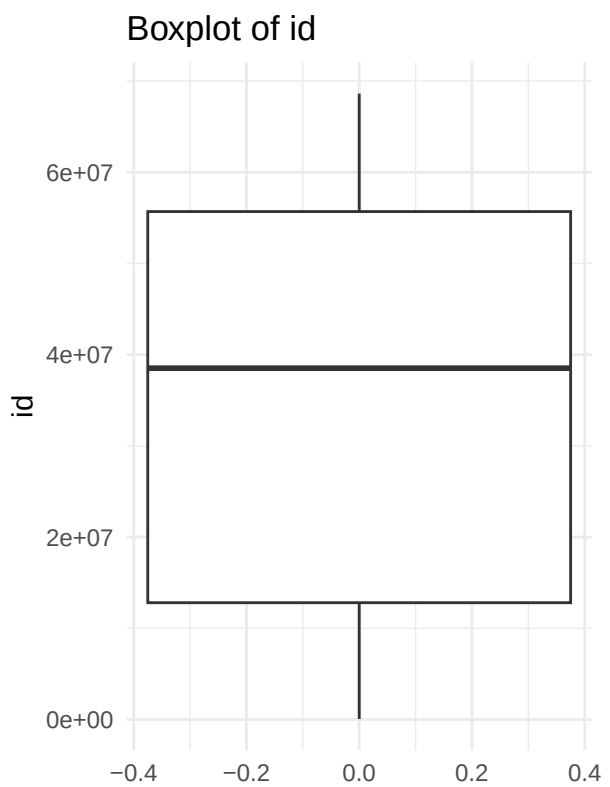
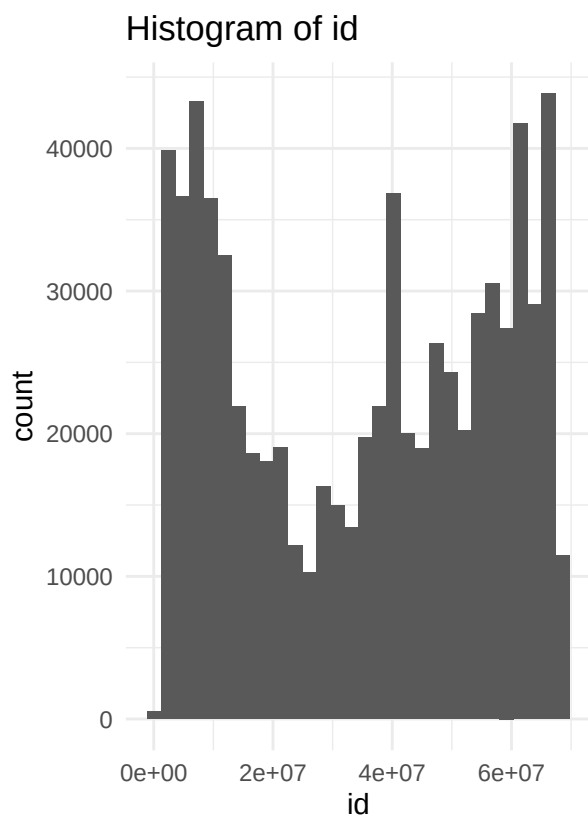
2.4.1 Outlier-Analysis

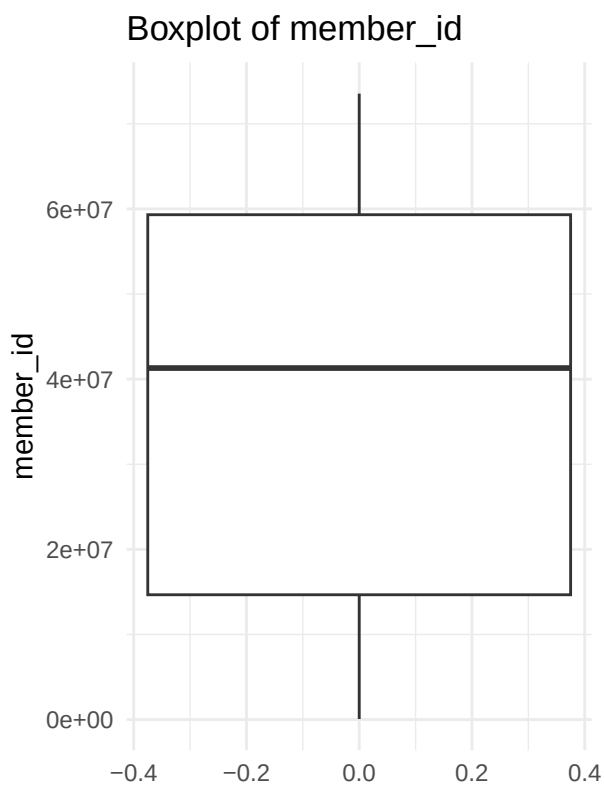
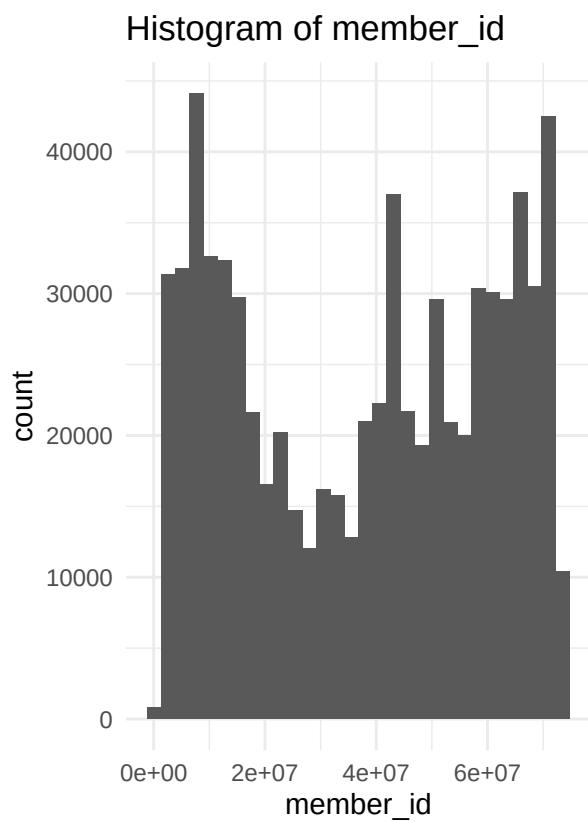
```
# Boxplot & Histogram for numerical columns to visualize outliers with ggplot2
for (col_name in numerical_cols) {

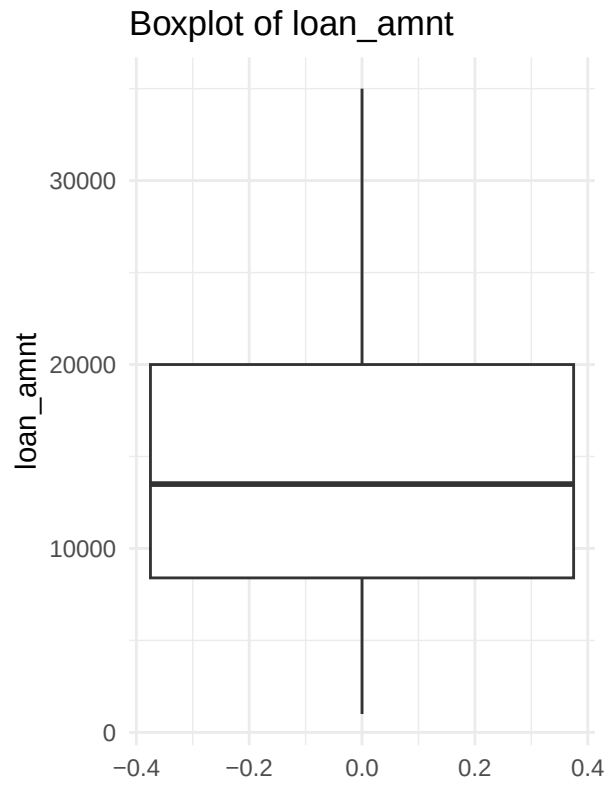
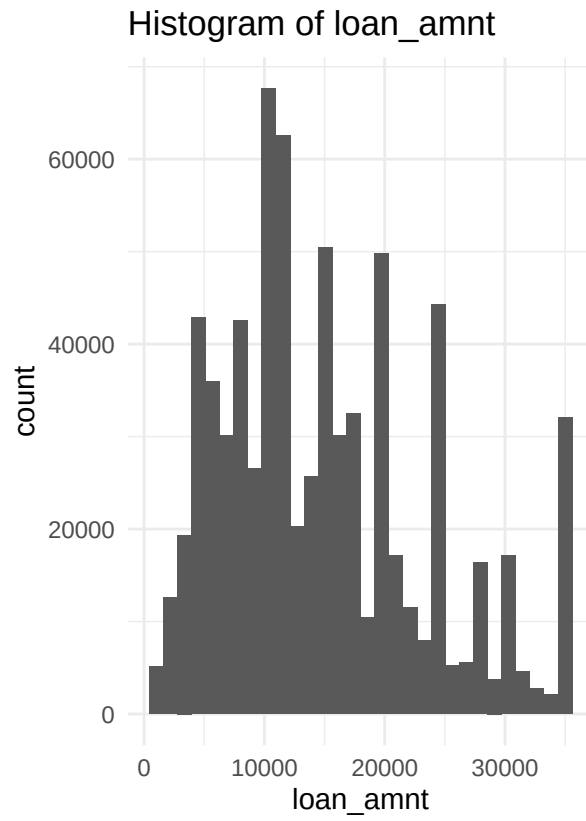
  p1 <- ggplot(cleaned_data, aes_string(x = col_name)) +
    geom_histogram(bins = 30) +
    ggtitle(paste("Histogram of", col_name)) +
    theme_minimal()

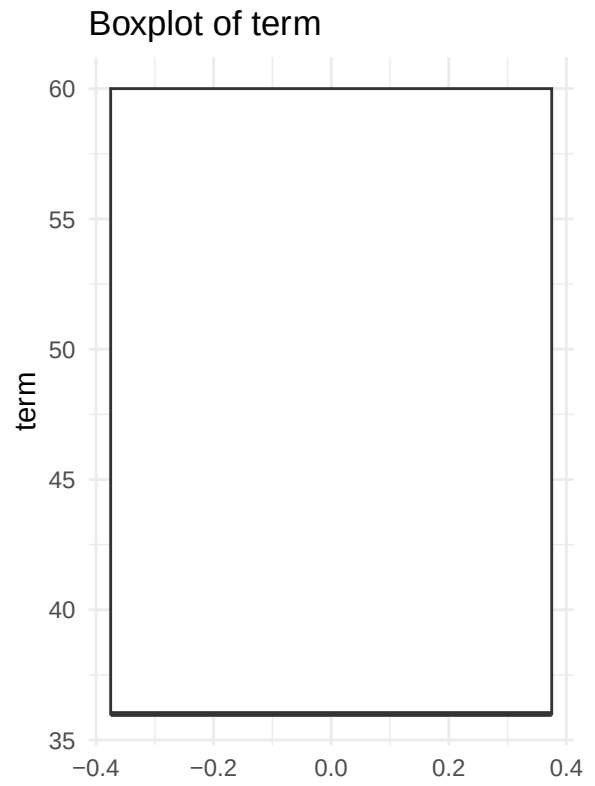
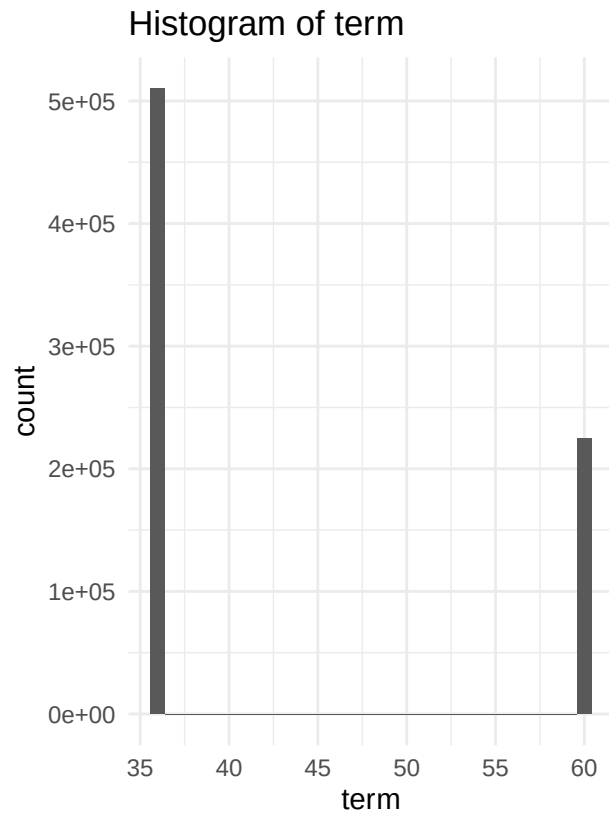
  p2 <- ggplot(cleaned_data, aes_string(y = col_name)) +
    geom_boxplot() +
    ggtitle(paste("Boxplot of", col_name)) +
    theme_minimal()

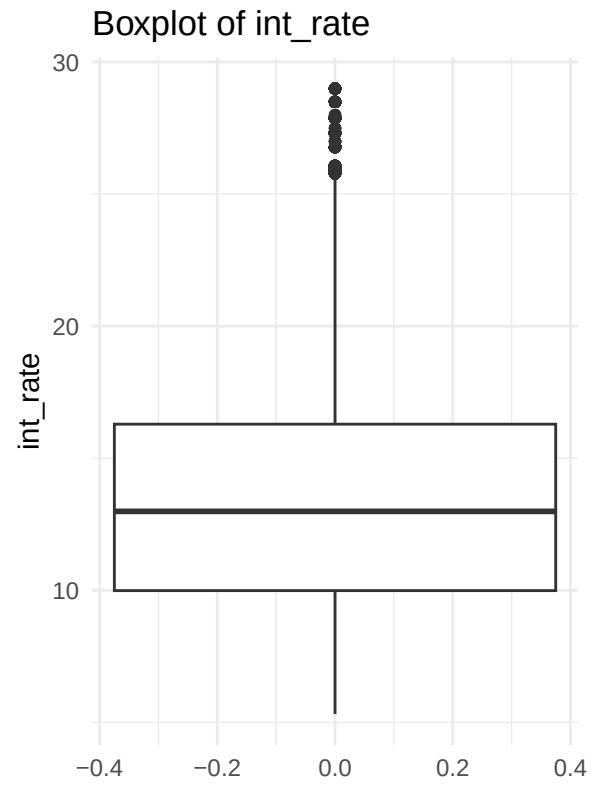
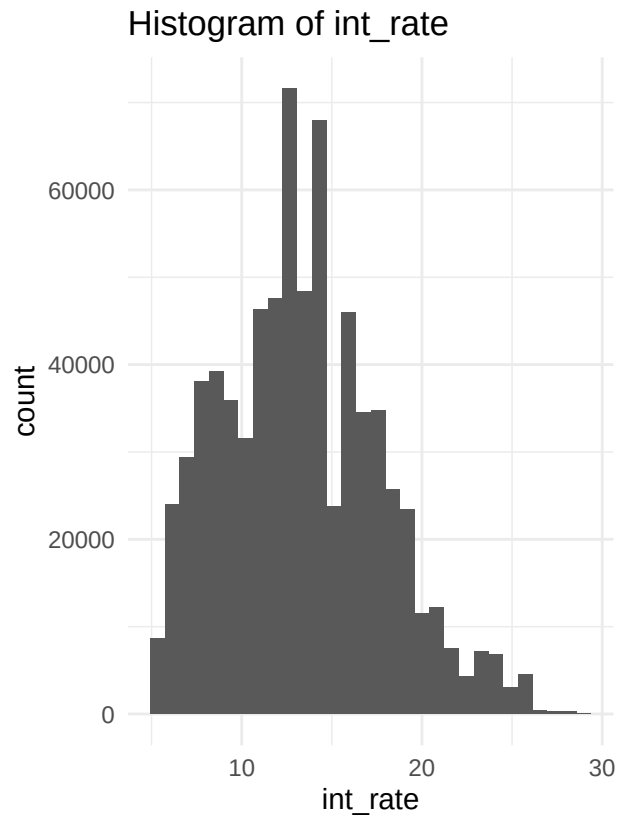
  # Combine side-by-side
  print(p1 + p2)
}
```

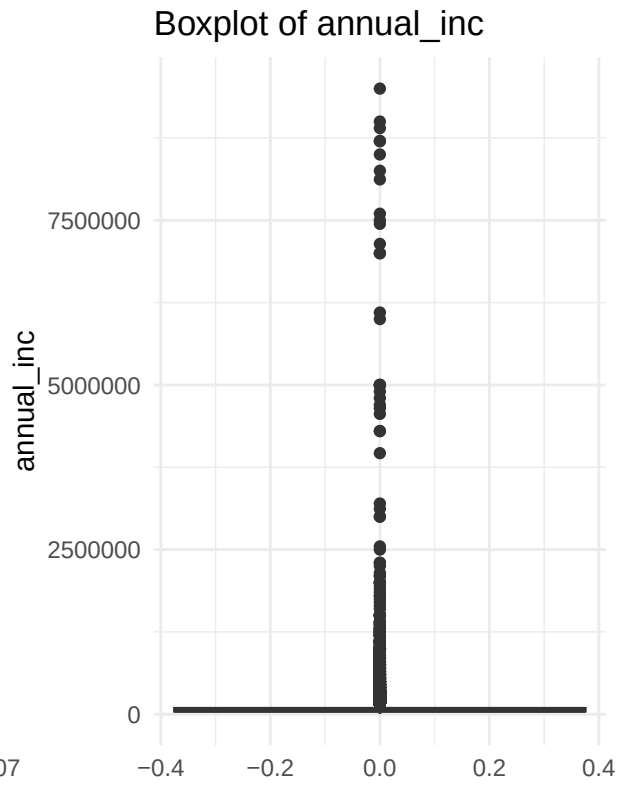
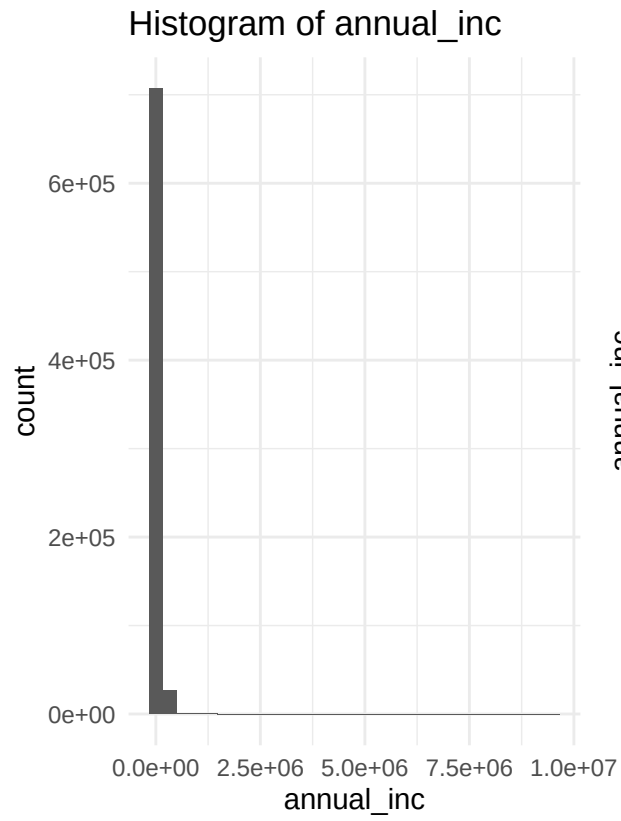


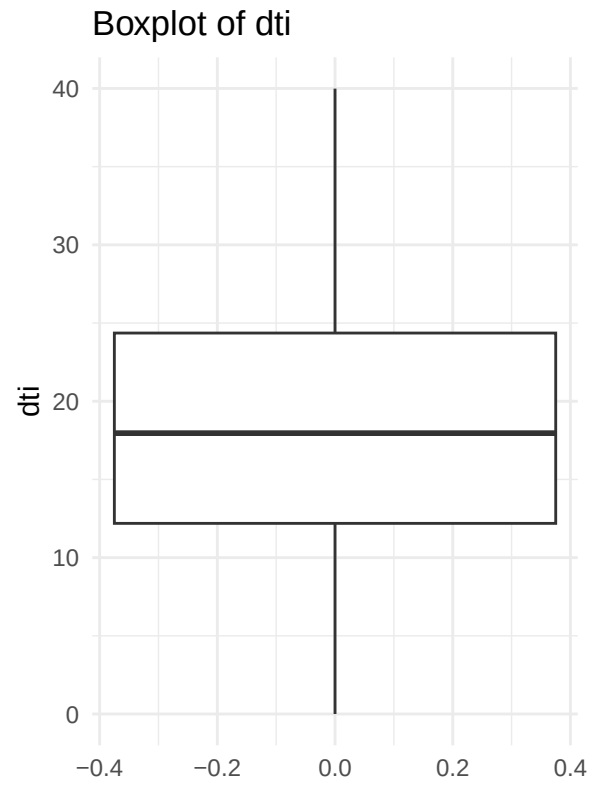
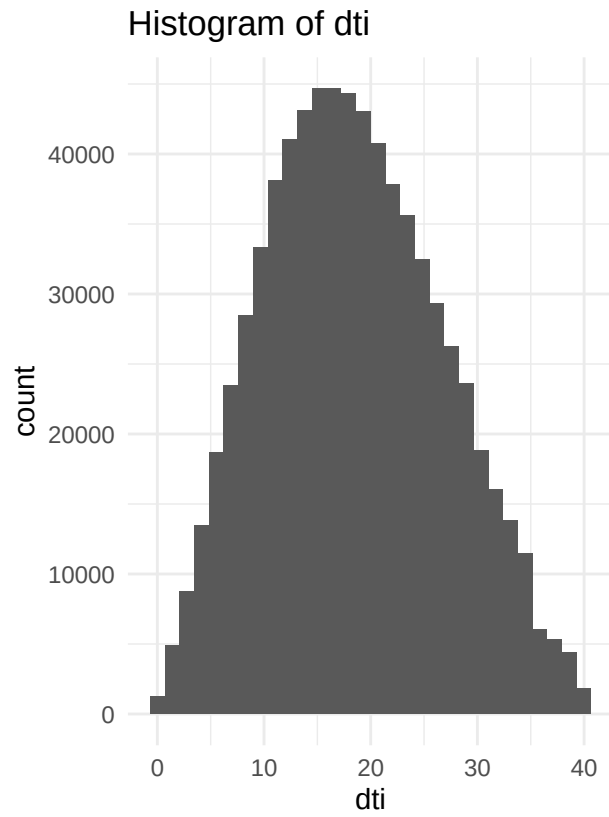


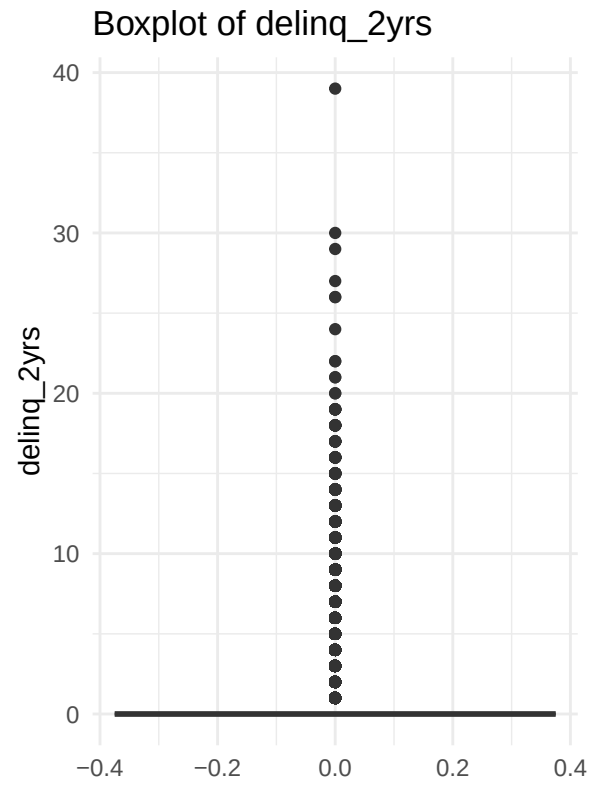
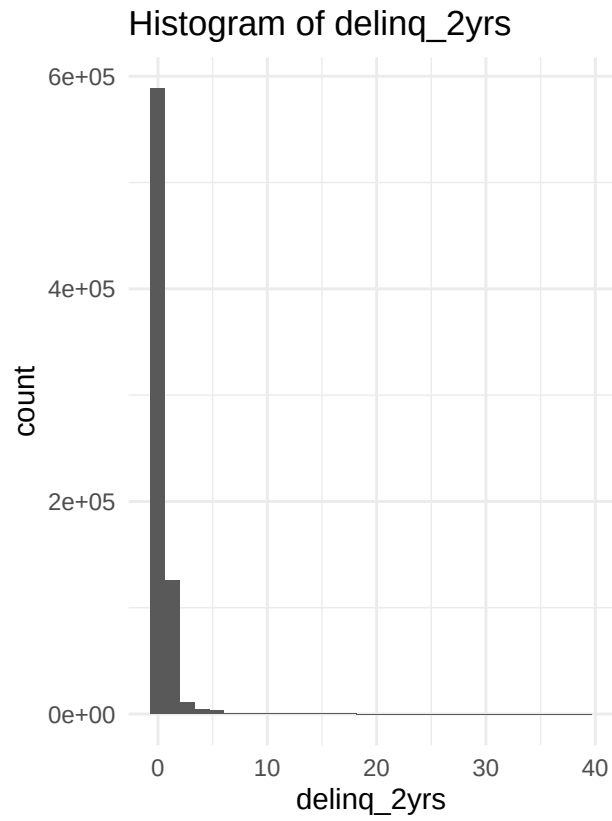


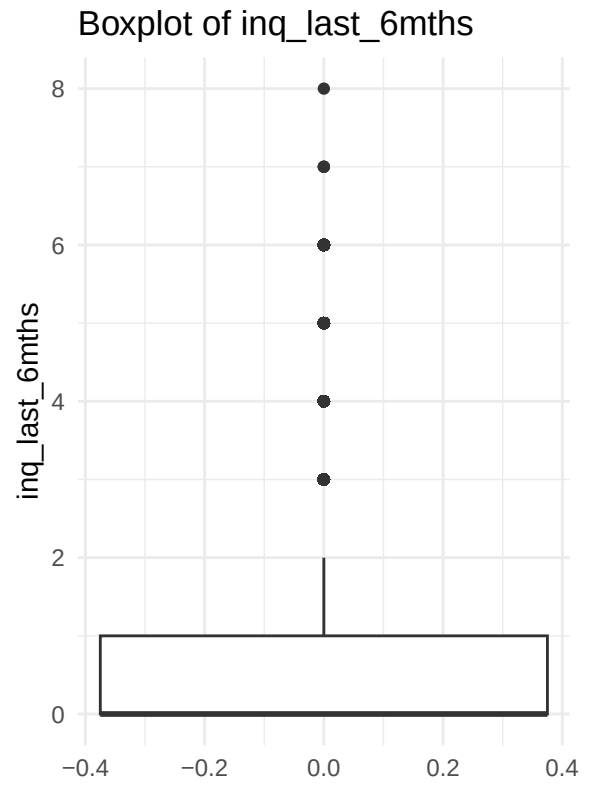
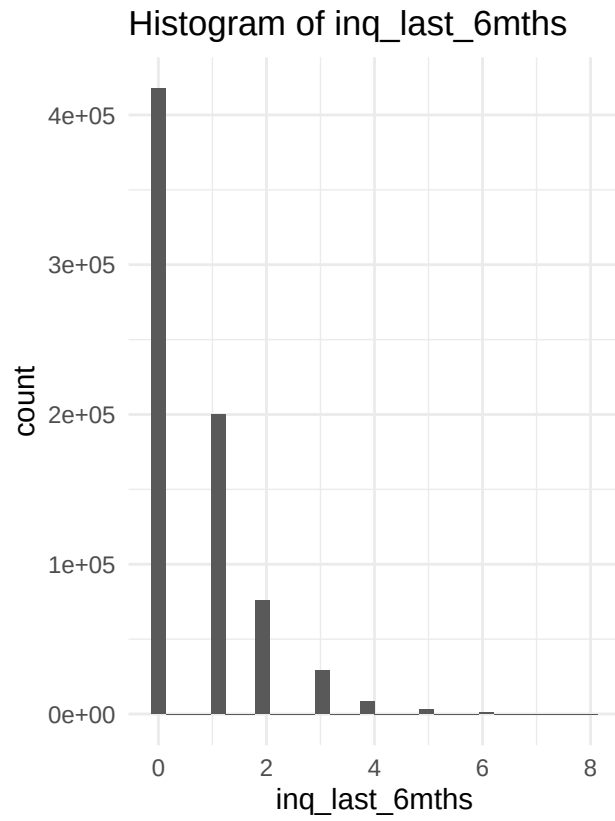


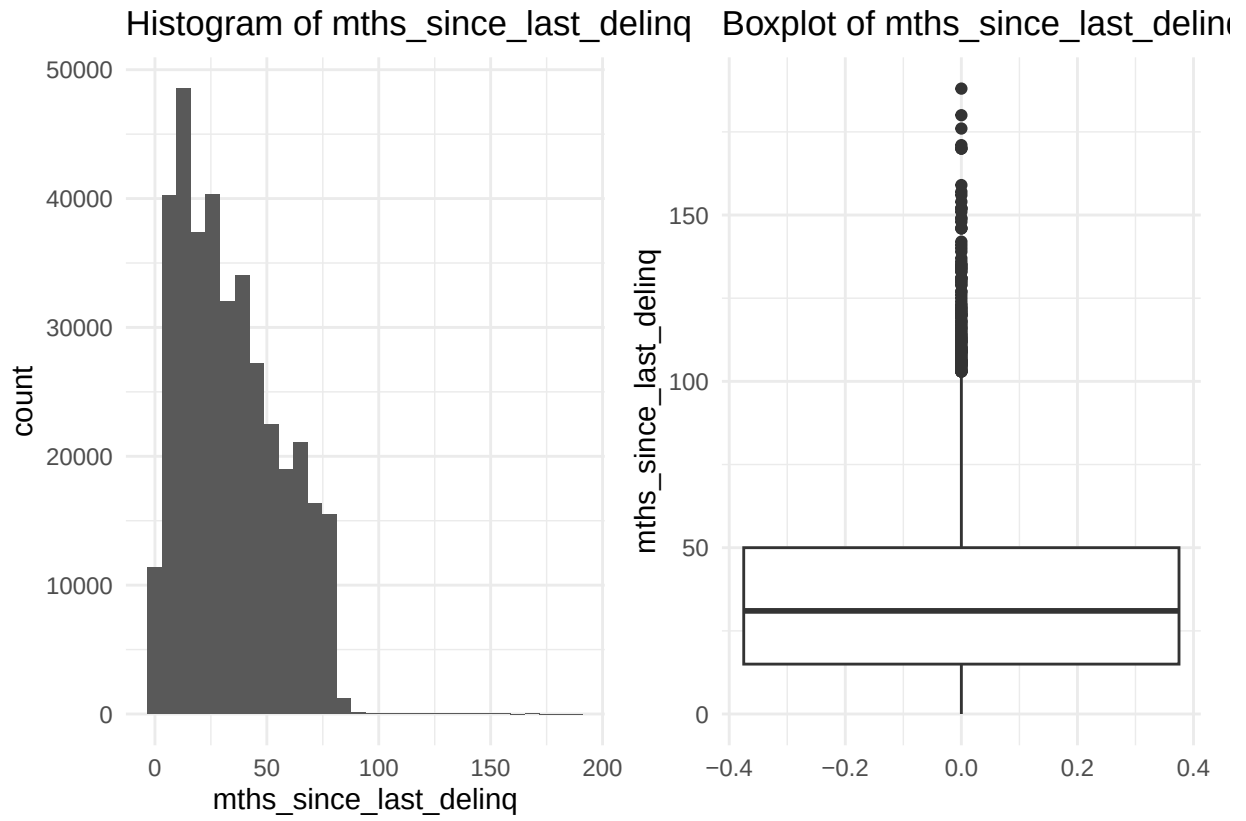




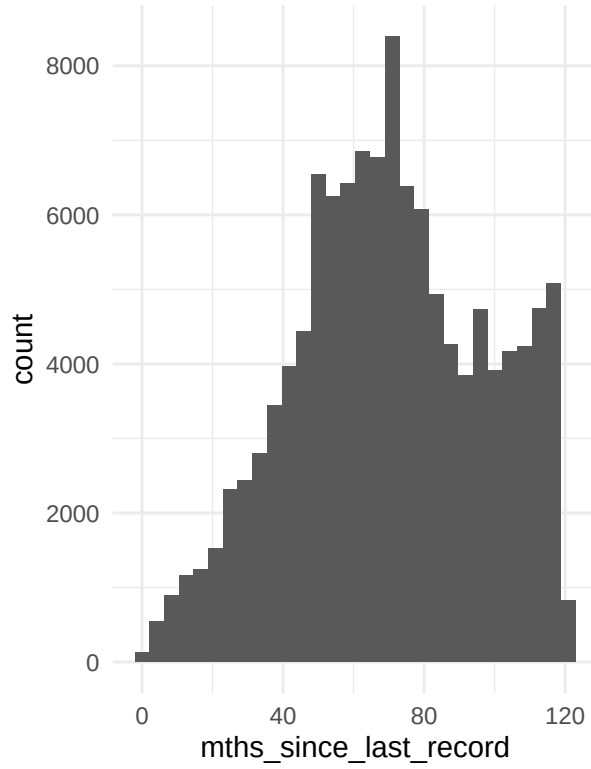




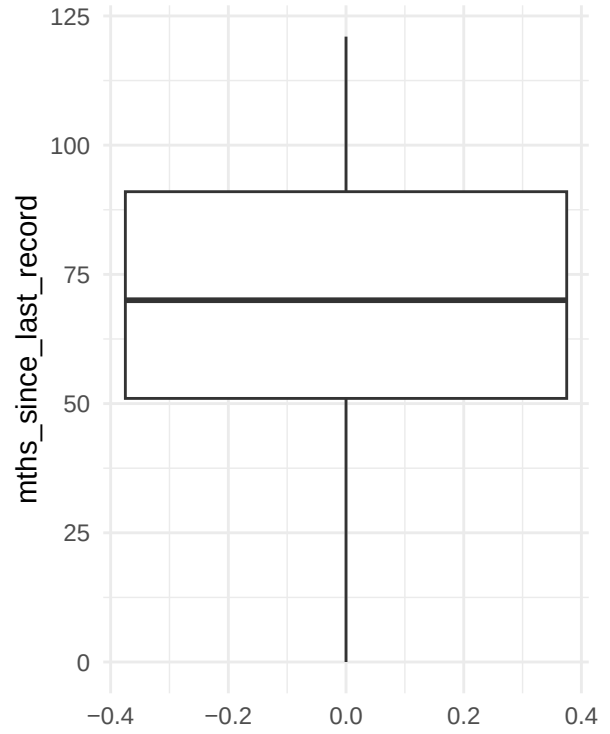


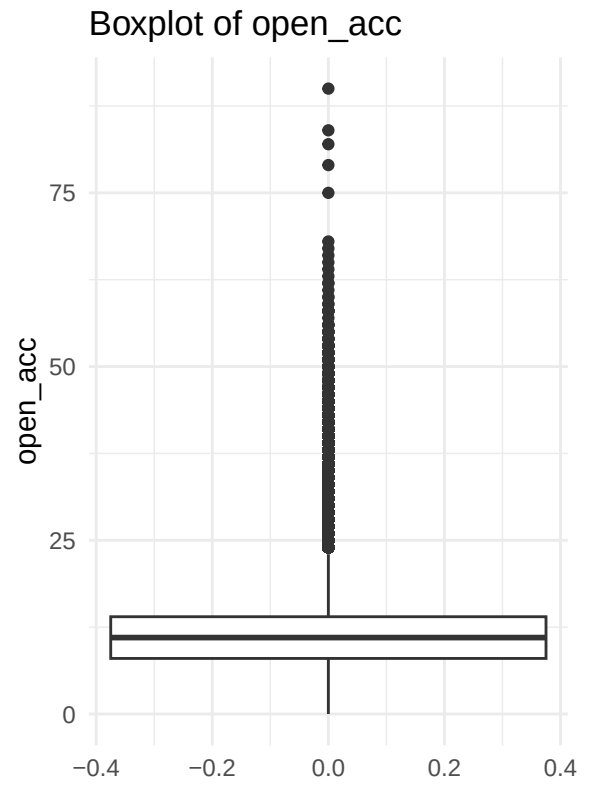
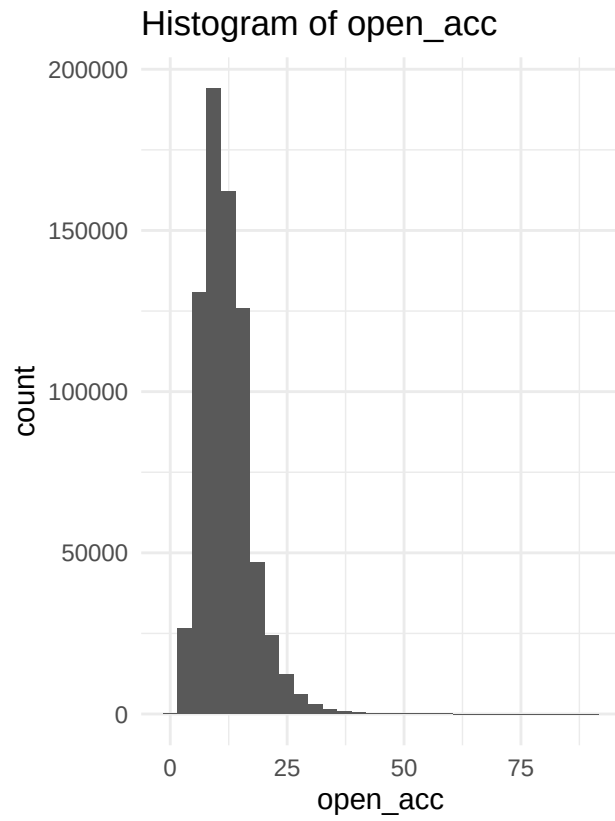


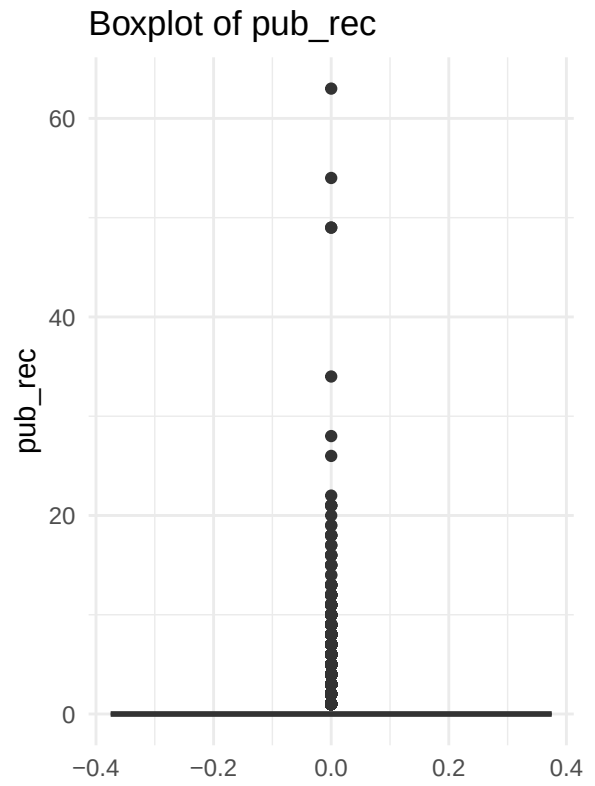
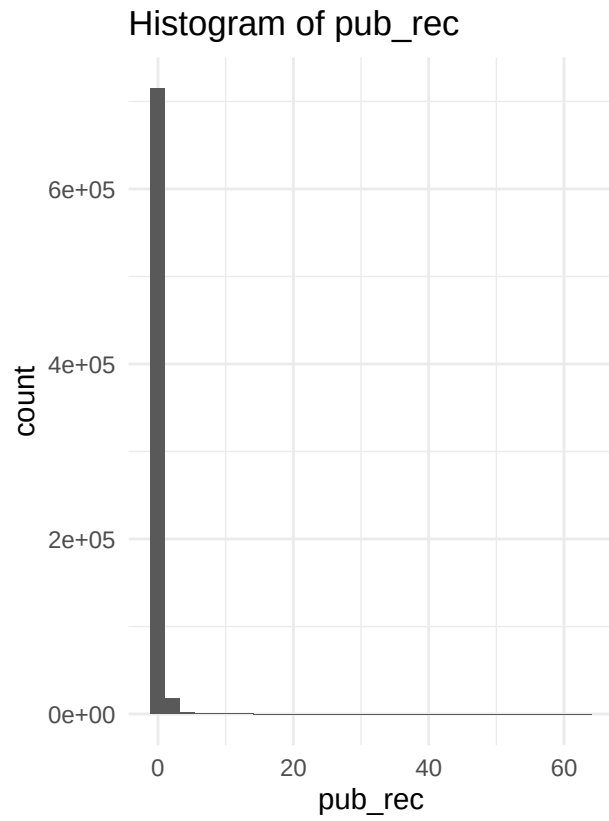
Histogram of mths_since_last_record

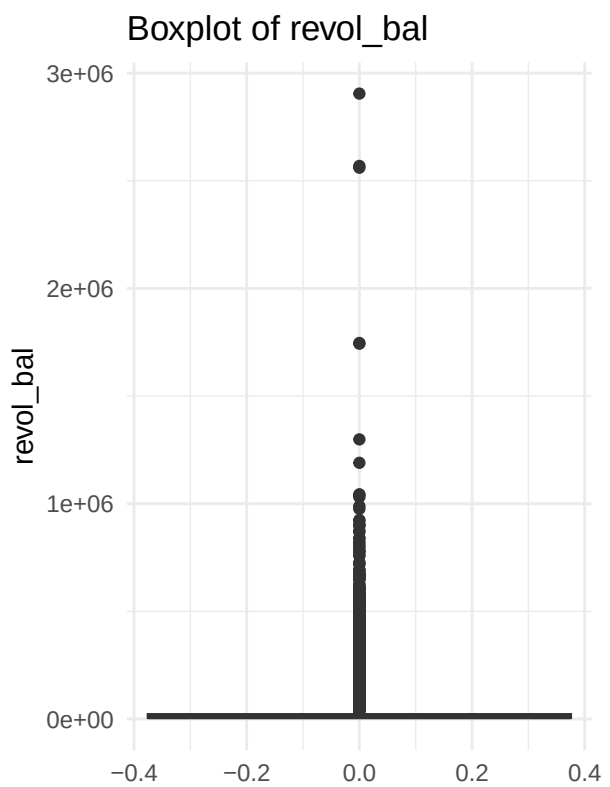
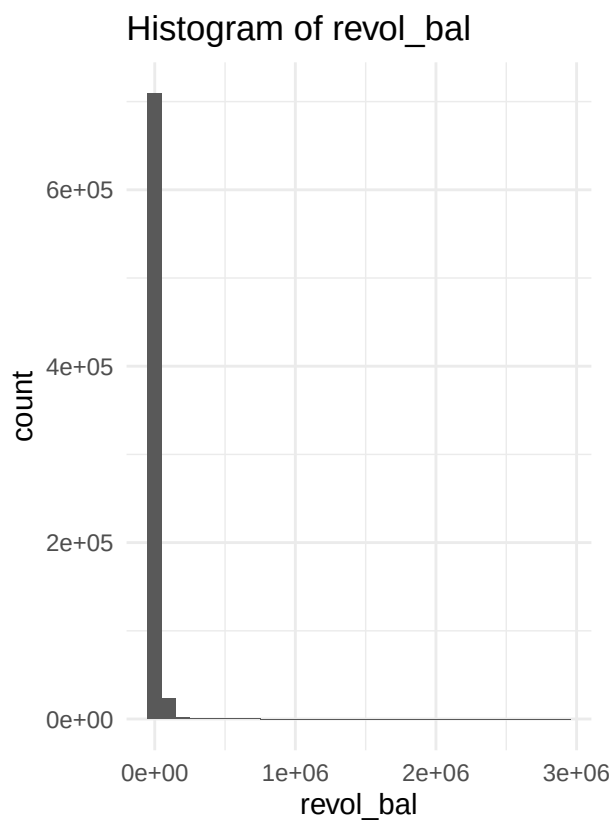


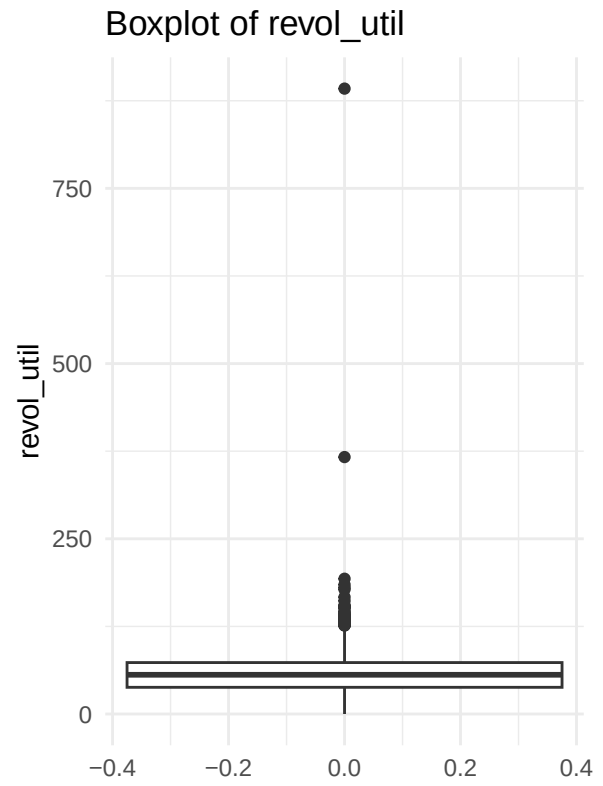
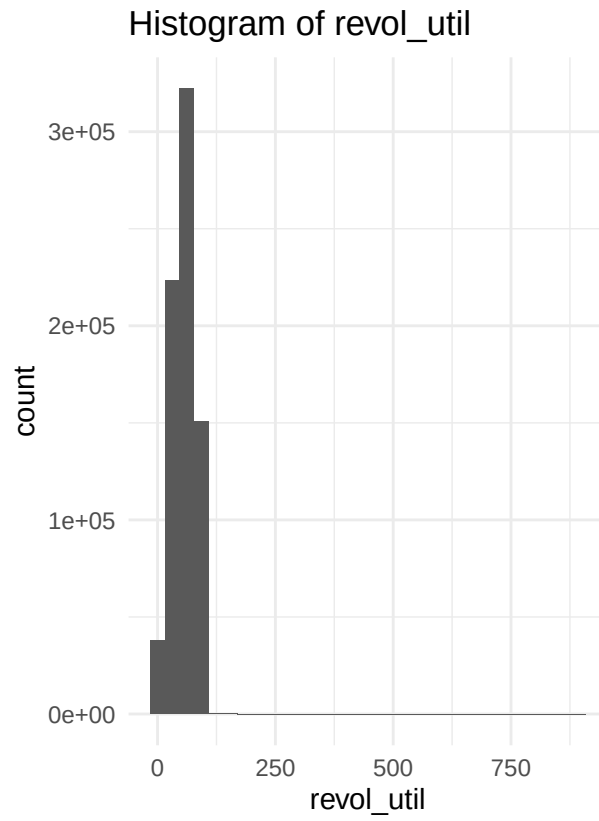
Boxplot of mths_since_last_record

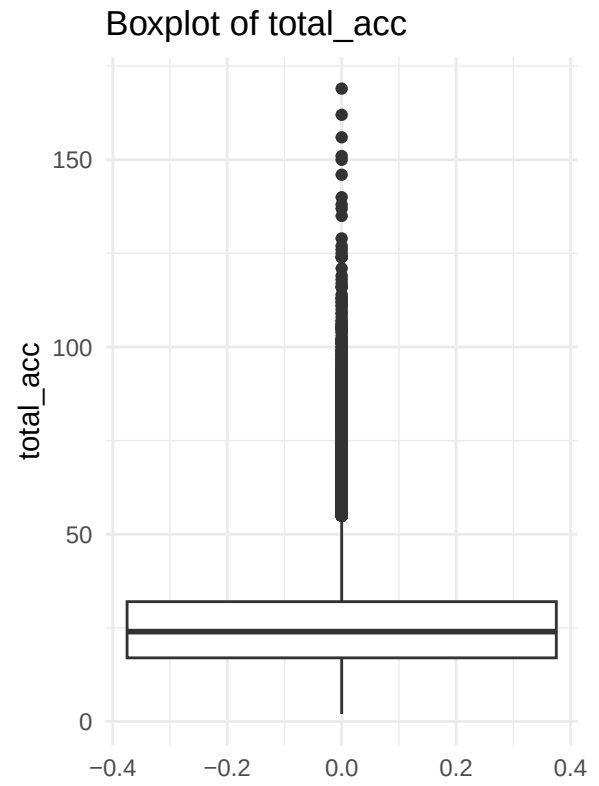
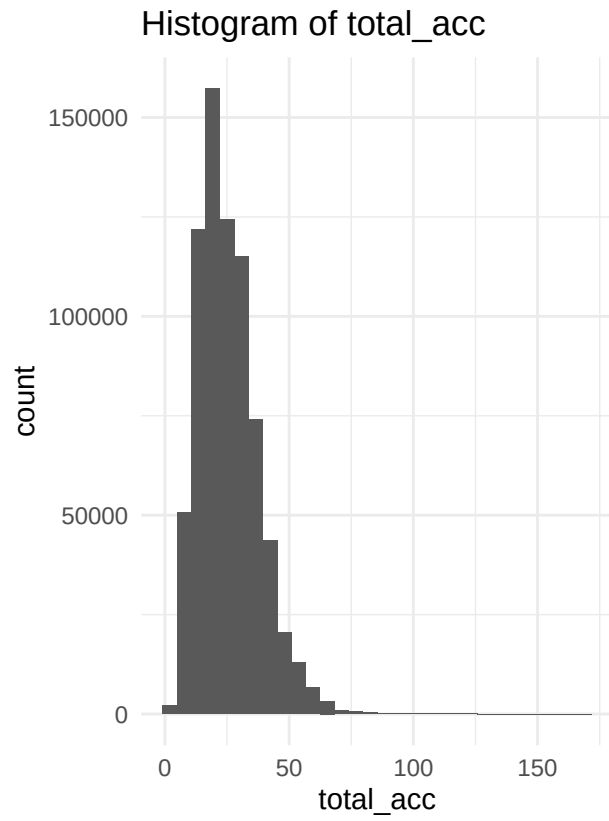


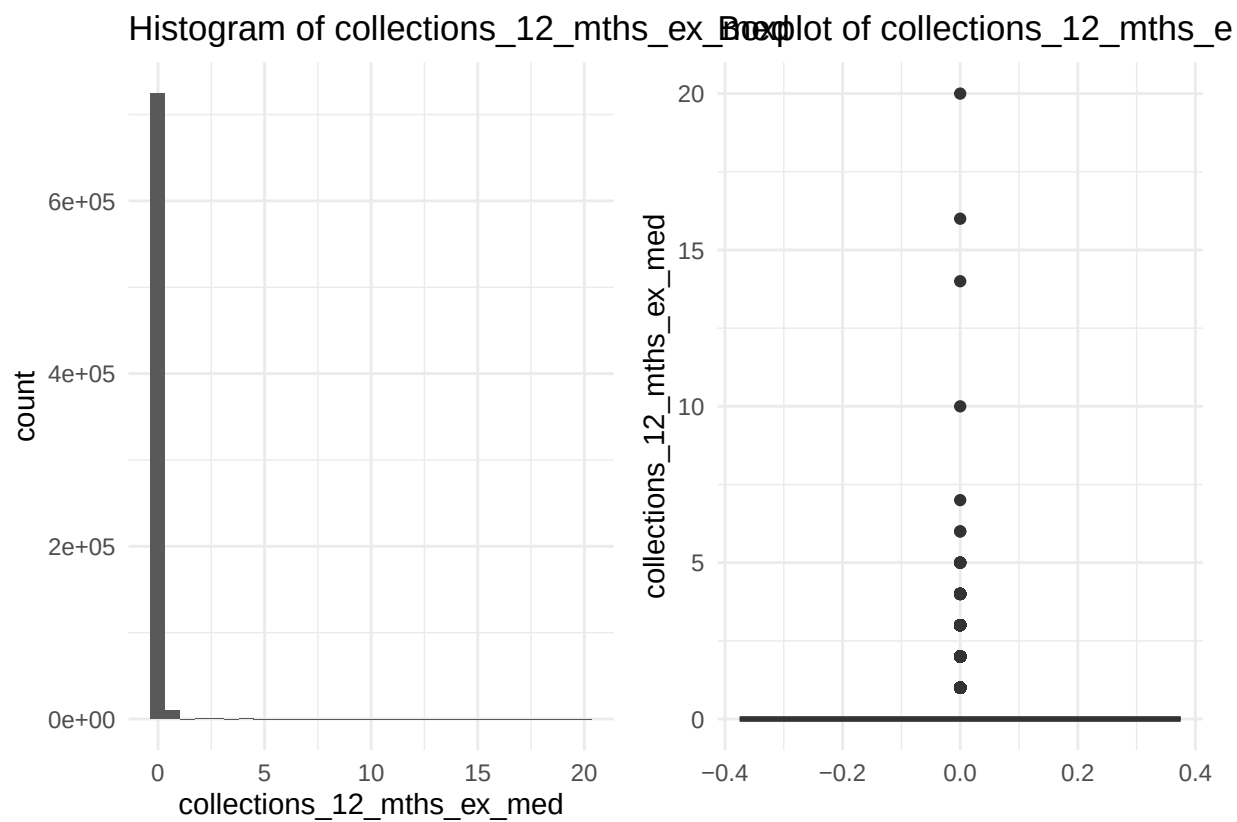


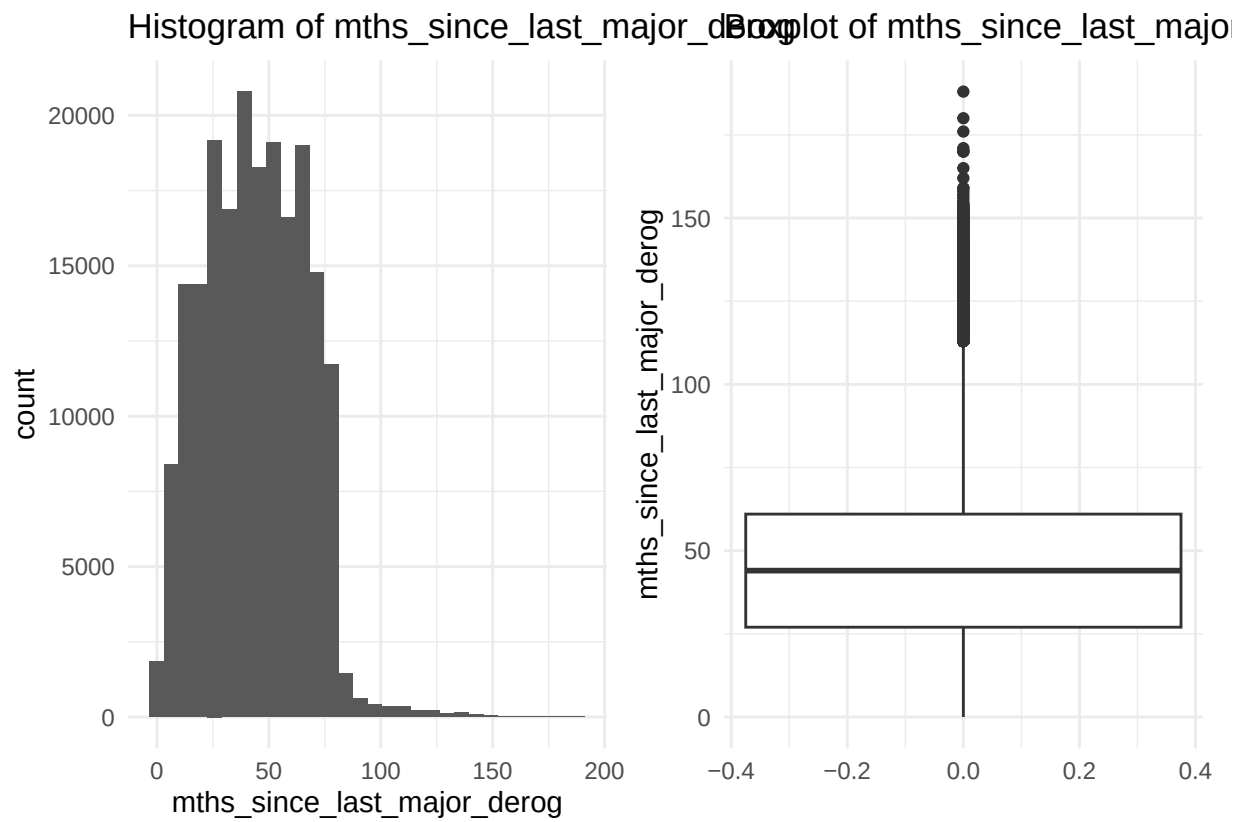


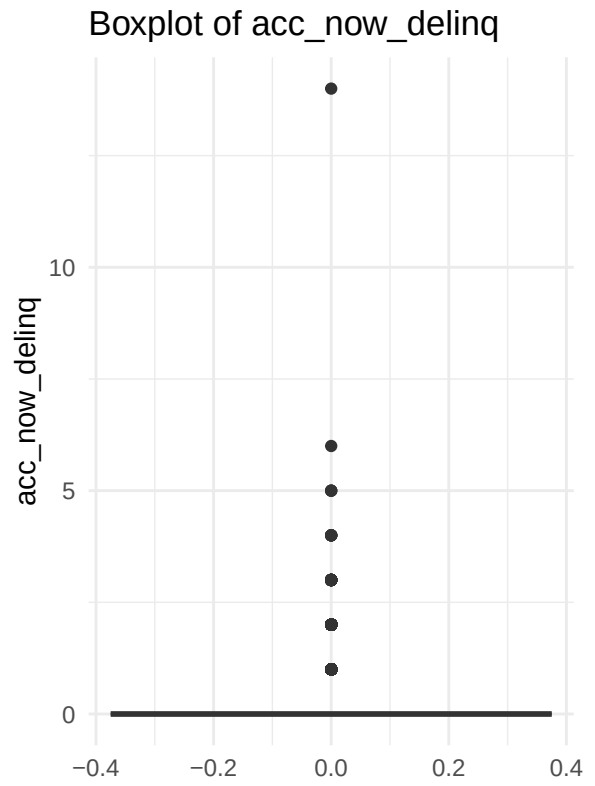
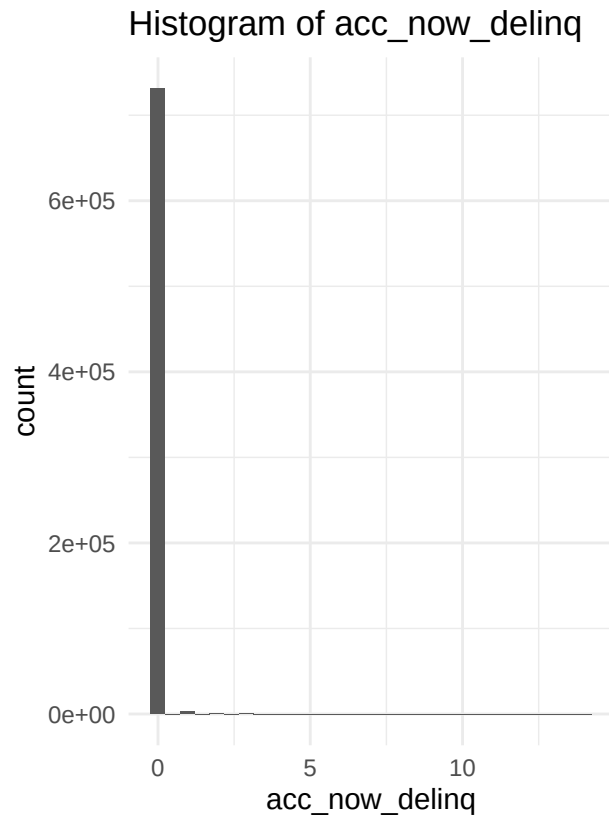


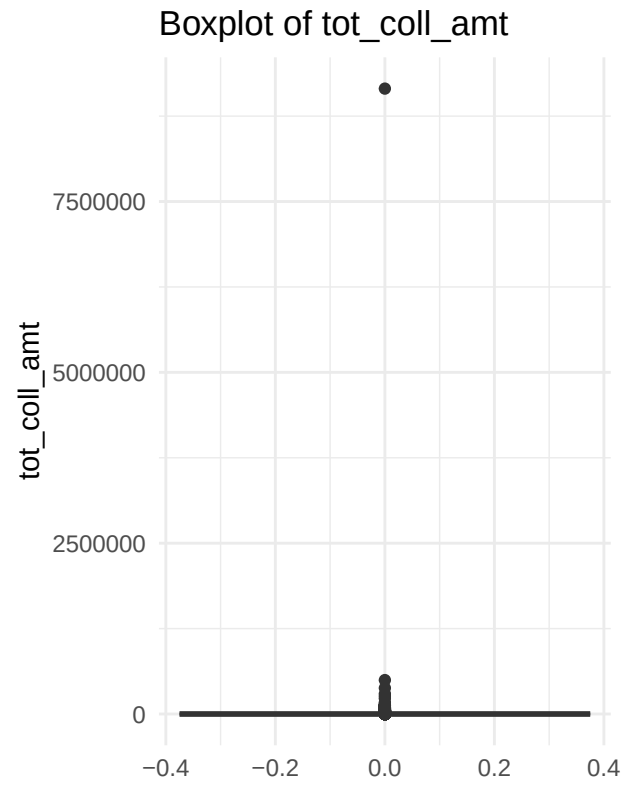
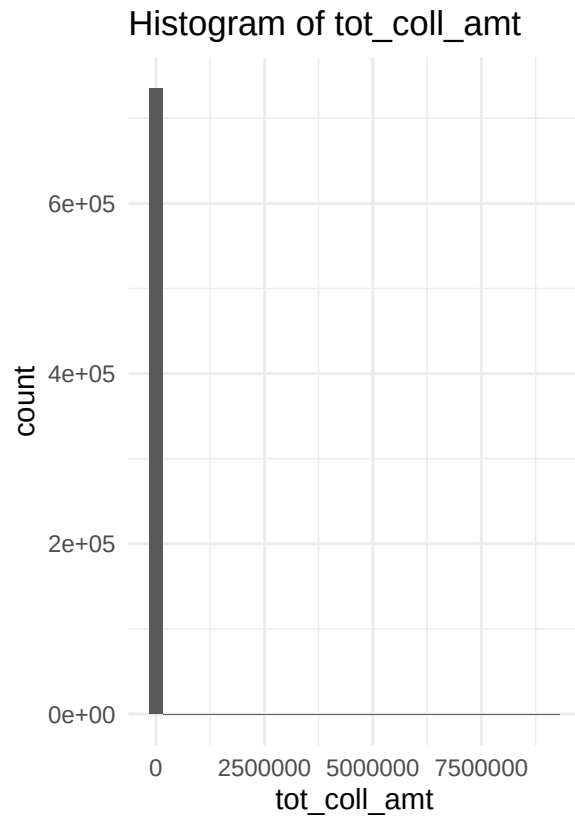


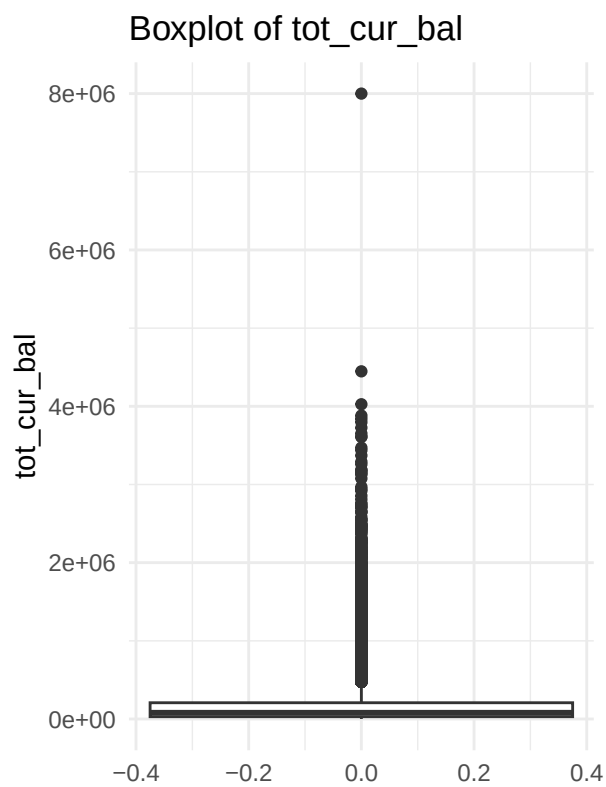
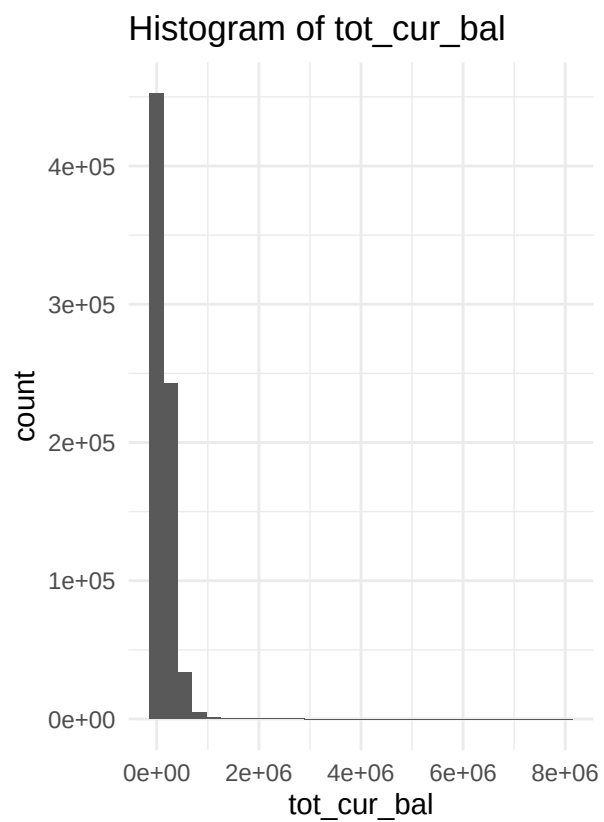


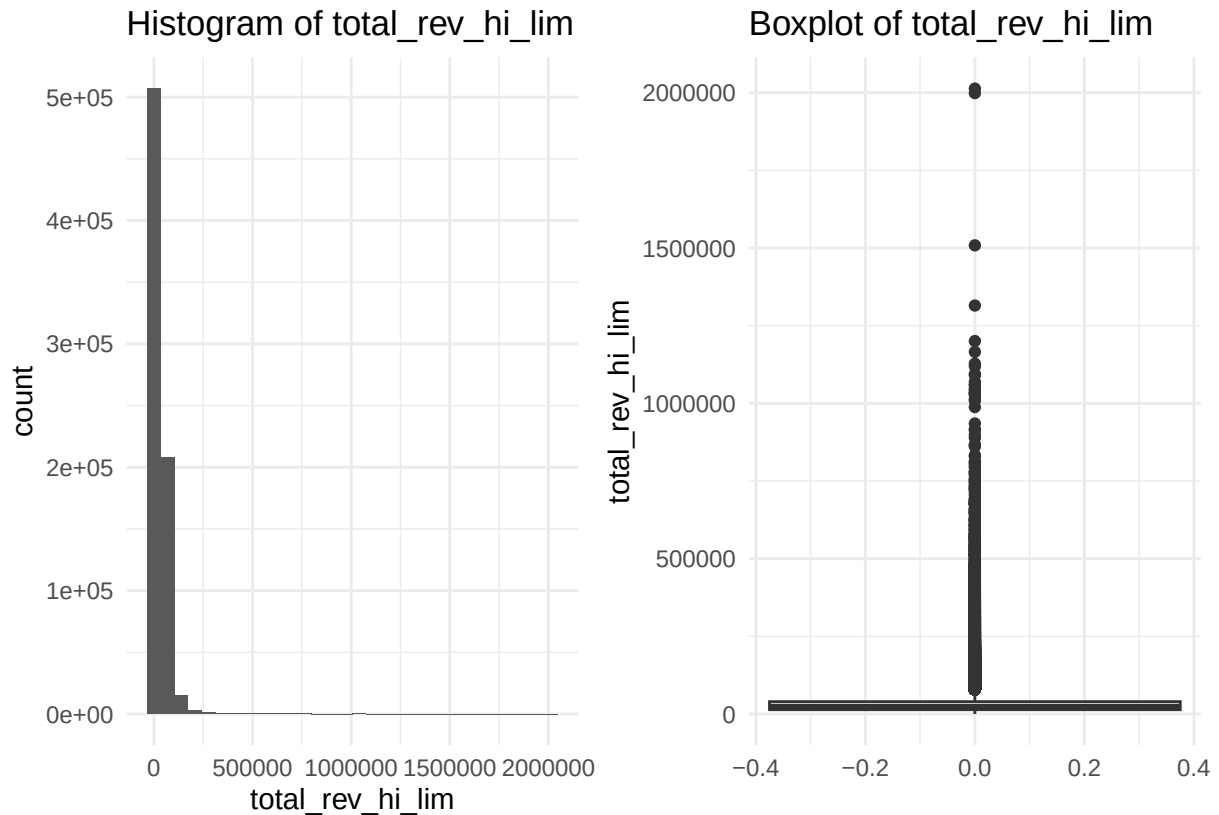












In the following section we will go through all columns and check for anomalies and outliers. If any are found, we will decide on a case-by-case basis how to handle them.

```
# Find the top 10 most frequent loan amounts
top_loan_amounts <- cleaned_data %>%
  # Filter for loans in the upper range (e.g., above $30,000)
  filter(loan_amnt >= 30000) %>%
  count(loan_amnt, sort = TRUE) %>%
  head(20)

print(top_loan_amounts) # Lending cap is found at 35'000, be aware of this in modelling
```

2.4.1.1 Loan amount (loan_amnt)

```
##      loan_amnt      n
## 1      35000 31329
## 2      30000 14991
## 3      32000  2363
## 4      33000   898
## 5      34000   501
## 6      31000   428
## 7      33600   345
## 8      32500   329
```



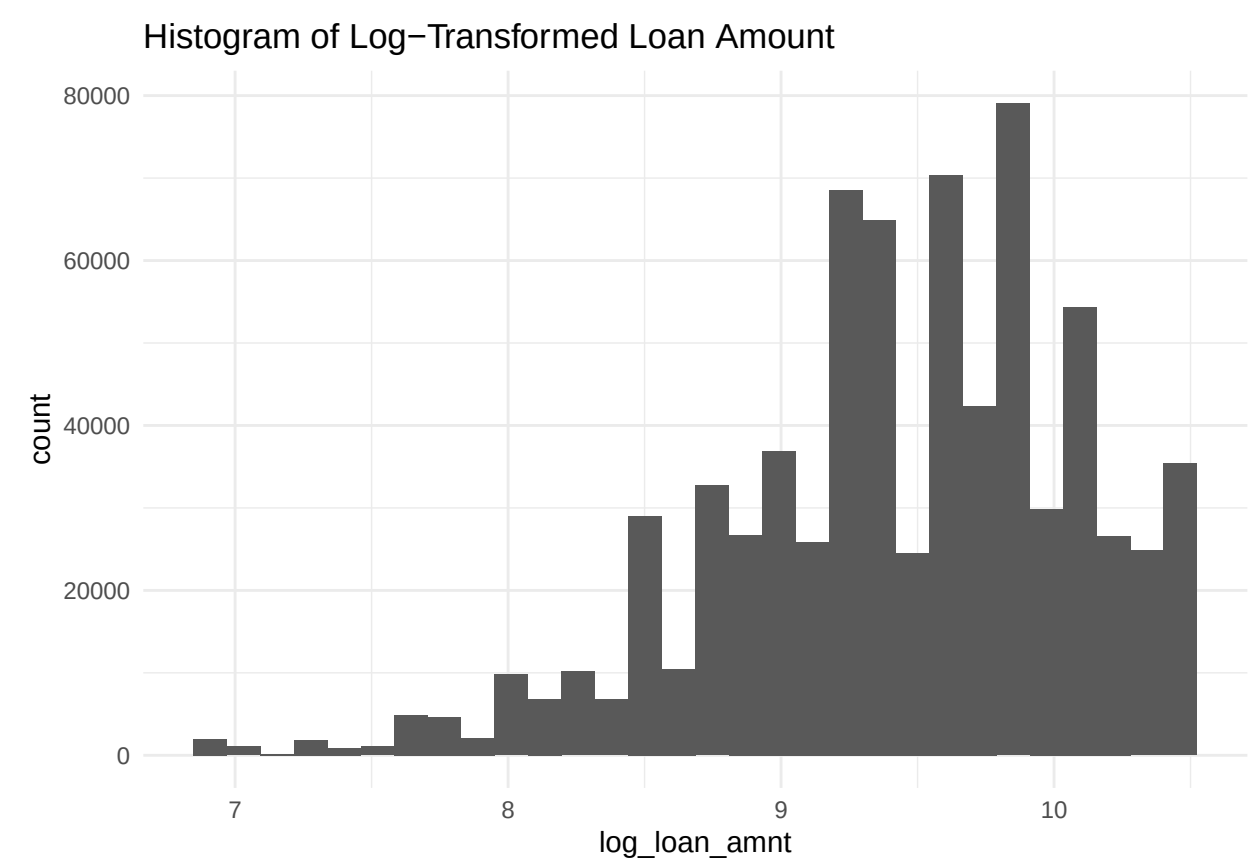
```
## 9      31500    325
## 10     31300    256
## 11     33425    234
## 12     31200    206
## 13     32350    203
## 14     30225    194
## 15     30750    183
## 16     34475    163
## 17     30800    152
## 18     32400    149
## 19     33950    143
## 20     33575    137
```

The distribution shows a strong spike at \$35,000, suggesting an effective upper cap in this dataset. This spike is therefore treated as a data boundary rather than an outlier.

Due to the big range from minimum to maximum loan amount, one could consider to scale this feature before modelling. A log-transformation could also be considered. A log-transformation column of loan amount is added to the cleaned dataset.

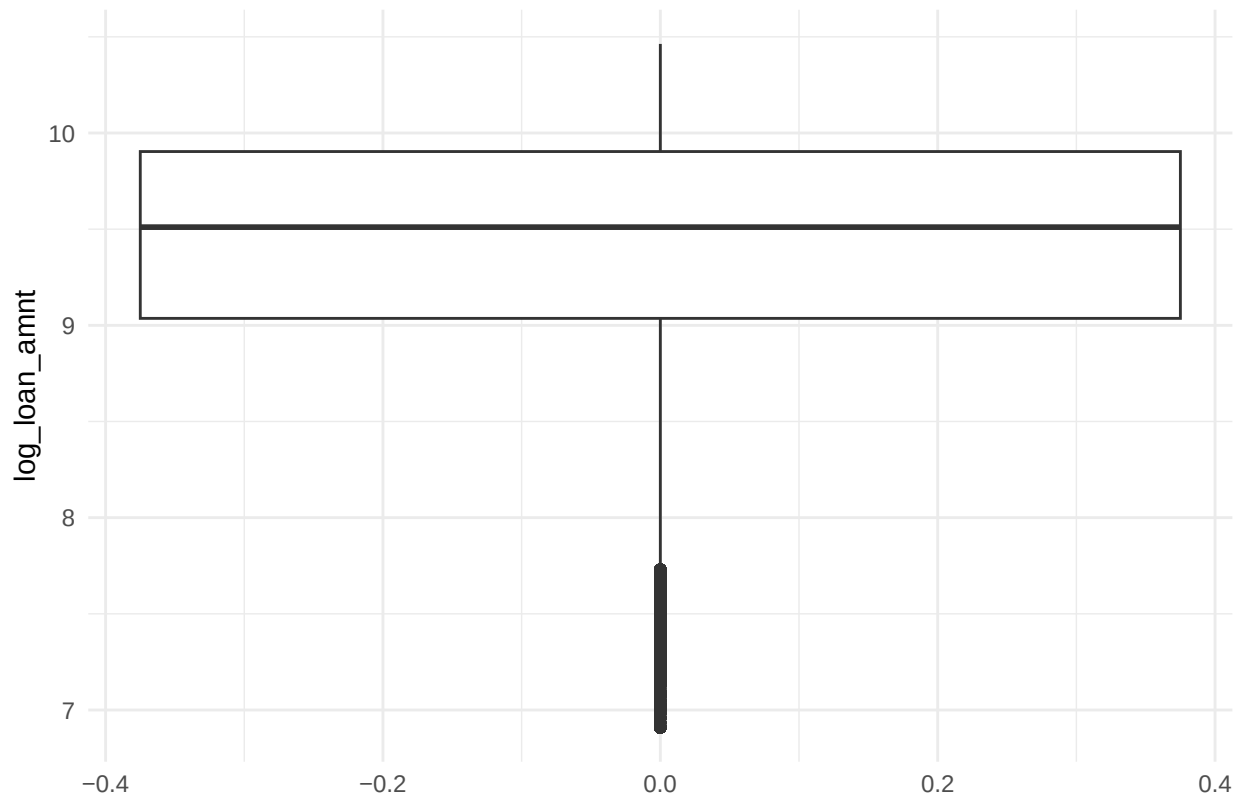
```
# Add log-transformed loan amount column
cleaned_data <- cleaned_data %>%
  mutate(log_loan_amnt = log(loan_amnt + 1)) # Adding

# Check histogram and boxplot of log-transformed loan amount
ggplot(cleaned_data, aes(x = log_loan_amnt)) +
  geom_histogram(bins = 30) +
  ggtitle("Histogram of Log-Transformed Loan Amount") +
  theme_minimal()
```



```
ggplot(cleaned_data, aes(y = log_loan_amnt)) +  
  geom_boxplot() +  
  ggtitle("Boxplot of Log-Transformed Loan Amount") +  
  theme_minimal()
```

Boxplot of Log-Transformed Loan Amount



```
# Show the lowest 5 and highest 5 annual incomes. Write as function so it can be used in later analyses
show_extreme_values <- function(data, column_name, n = 5) {
  sorted_data <- data %>%
    arrange(!!sym(column_name))

  lowest_values <- head(sorted_data, n)
  highest_values <- tail(sorted_data, n)

  list(lowest = lowest_values, highest = highest_values)
}

extreme_annual_inc <- show_extreme_values(cleaned_data, "annual_inc", n = 5)
print("Lowest 5 annual incomes:")
```

2.4.1.2 Annual income (annual_inc)

```
## [1] "Lowest 5 annual incomes:"
```

```
print(extreme_annual_inc$lowest)
```

```
##           id member_id loan_amnt term int_rate           emp_title emp_length
## 1 46275208 49382946      1000   36   13.99                <NA>
```

```

## 2 37731860 40494874 1000 36 17.14 <NA>
## 3 67898750 72757489 1125 36 17.86 Head Building Manager 10+ years
## 4 24114836 26517746 1300 36 14.49 <NA>
## 5 35083870 37767178 1500 36 19.99 Maintenance 2 years
## home_ownership annual_inc verification_status desc purpose
## 1 RENT 3000 Verified other
## 2 RENT 3600 Verified debt_consolidation
## 3 MORTGAGE 3800 Verified moving
## 4 OWN 4000 Source Verified debt_consolidation
## 5 RENT 4000 Not Verified credit_card
## zip_code dti delinq_2yrs inq_last_6mths mths_since_last_delinq
## 1 622xx 24.00 0 1 NA
## 2 927xx 28.67 0 0 NA
## 3 851xx 34.81 0 0 52
## 4 290xx 15.02 0 0 NA
## 5 117xx 39.34 0 1 29
## mths_since_last_record open_acc pub_rec revol_bal revol_util total_acc
## 1 NA 5 0 1013 7.7 5
## 2 NA 6 0 2876 37.4 6
## 3 47 4 1 3604 82.0 13
## 4 66 15 1 1131 7.0 42
## 5 101 7 1 3611 65.7 13
## collections_12_mths_ex_med mths_since_last_major_derog policy_code
## 1 0 NA 1
## 2 0 NA 1
## 3 0 44 1
## 4 0 NA 1
## 5 0 NA 1
## acc_now_delinq tot_coll_amt tot_cur_bal total_rev_hi_lim log_loan_amnt
## 1 0 0 28513 13200 6.908755
## 2 0 0 10876 7700 6.908755
## 3 0 0 3604 9009 7.026427
## 4 0 0 52807 16200 7.170888
## 5 0 0 3611 5500 7.313887

```

```
print("Highest 5 annual incomes:")
```

```
## [1] "Highest 5 annual incomes:"
```

```
print(extreme_annual_inc$highest)
```

```

## id member_id loan_amnt term int_rate emp_title
## 734821 66605376 71331231 14000 60 10.64 Customer Representative
## 734822 39429853 42233601 8000 36 13.66 Correctional Sgt.
## 734823 55140826 58711625 10550 36 15.61 commercial driver
## 734824 65945334 70640084 11000 36 6.89 registered nurse
## 734825 54067210 57607924 24000 60 7.89 reg nurse
## emp_length home_ownership annual_inc verification_status desc
## 734821 10+ years MORTGAGE 8700000 Source Verified
## 734822 10+ years MORTGAGE 8706582 Source Verified
## 734823 10+ years RENT 8900060 Source Verified
## 734824 10+ years MORTGAGE 9000000 Source Verified
## 734825 10+ years MORTGAGE 9500000 Source Verified

```

```

##           purpose zip_code  dti delinq_2yrs inq_last_6mths
## 734821 debt_consolidation  292xx 0.18          0          0
## 734822 credit_card        629xx 0.11          0          1
## 734823 debt_consolidation  338xx 0.09          0          1
## 734824 debt_consolidation  330xx 0.08          0          1
## 734825 credit_card        906xx 0.12          0          0
##           mths_since_last_delinq mths_since_last_record open_acc pub_rec revol_bal
## 734821                NA                NA          20      0      12664
## 734822                NA                NA          11      0      16449
## 734823                NA                NA           9      0       5311
## 734824                43                NA          15      0      14165
## 734825                NA                NA          12      0      16854
##           revol_util total_acc collections_12_mths_ex_med
## 734821          44.9         32                0
## 734822          54.1         18                0
## 734823         102.1         11                0
## 734824          33.8         29                0
## 734825          22.0         31                0
##           mths_since_last_major_derog policy_code acc_now_delinq tot_coll_amt
## 734821                NA                1            0          348
## 734822                NA                1            0            0
## 734823                NA                1            0            0
## 734824                43                1            0            0
## 734825                NA                1            0            0
##           tot_cur_bal total_rev_hi_lim log_loan_amnt
## 734821          89277          28200      9.546884
## 734822          79135          30400      8.987322
## 734823          21988           5200      9.263976
## 734824         324692          41900      9.305741
## 734825         256653          84200     10.085851

```

```
summary(cleaned_data$annual_inc)
```

```

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      3000  46000   65000   75557   90000 9500000

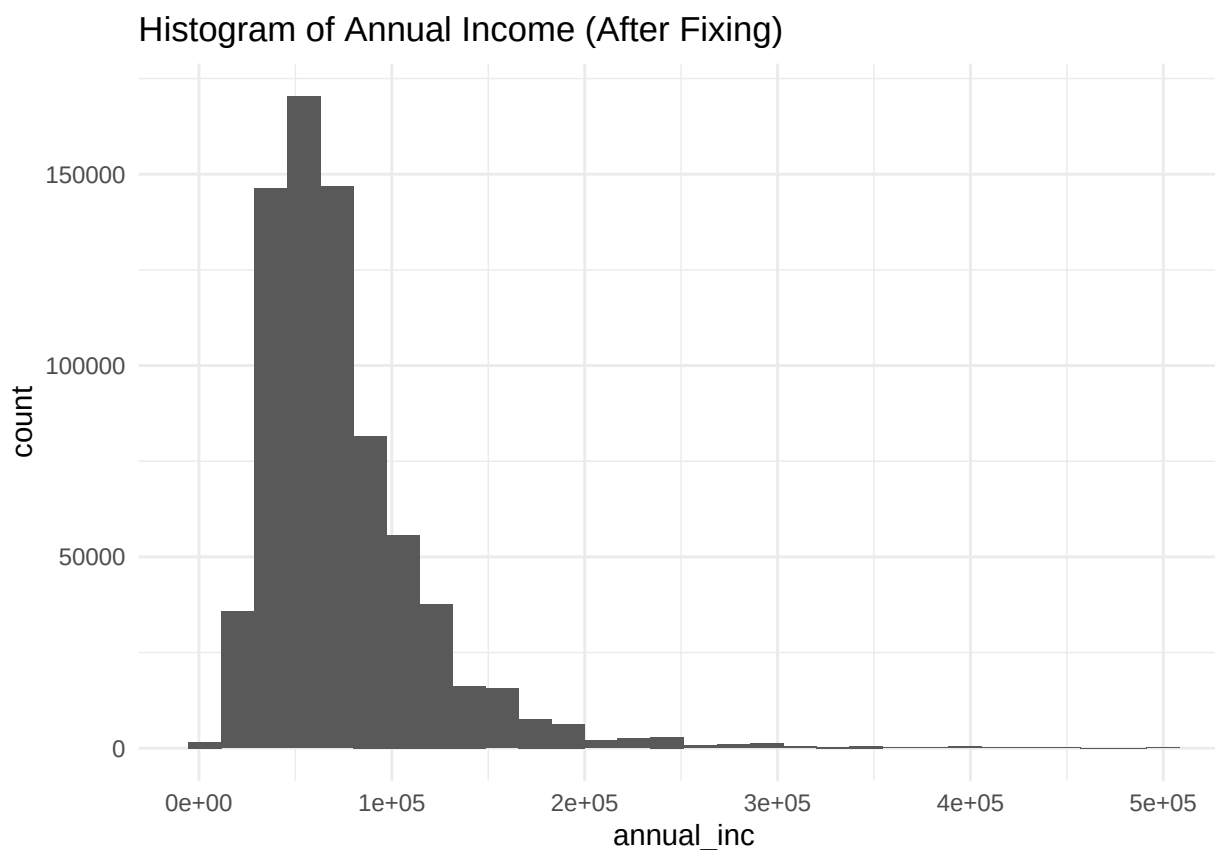
```

The variable `annual_inc` showed a highly skewed distribution with a mean of approximately \$75'556, a median of \$65,000, and a 75th percentile of \$90,000. These values are consistent with typical income patterns in US LendingClub borrower populations, where most applicants earn between \$40,000 and \$120,000. The standard deviation of roughly \$65,000 suggests that even extreme values should rarely exceed \$300,000–\$350,000, and this aligns with known domain expectations: in comparable datasets, the 99th percentile is typically around \$250,000 to \$300,000. Despite this, the dataset contained a number of extremely high incomes, including values between \$500,000 and several million dollars, with a maximum of \$9,500,000. Inspection of the associated employment titles revealed clear contradictions: applicants reporting incomes in the millions frequently listed occupations such as “Driver,” “Registered Nurse,” “Cashier,” or other roles incompatible with such earnings. This strongly suggests data entry or import errors, likely caused by misplaced decimals or dropped separators during ingestion (for example, entering 75,500 as 7,550,000). Based on both the statistical distribution and domain knowledge, \$500,000 was chosen as a conservative upper limit above which values are implausible. This threshold lies well beyond the expected upper tail of genuine borrower incomes and ensures that legitimate high-earning roles such as executives or surgeons are not incorrectly altered. For all observations with `annual_inc` greater than \$500,000, the income value was divided by 100, reflecting the most likely underlying data error pattern. After this correction, the adjusted values integrated smoothly into the natural income distribution, providing additional support for the validity of the transformation.

Because dti is directly dependent on income, the dti values for these corrected rows were set to NA, as their original values were derived from corrupted income entries. This approach preserves realistic data structure, corrects clear inconsistencies, and avoids removing potentially informative observations.

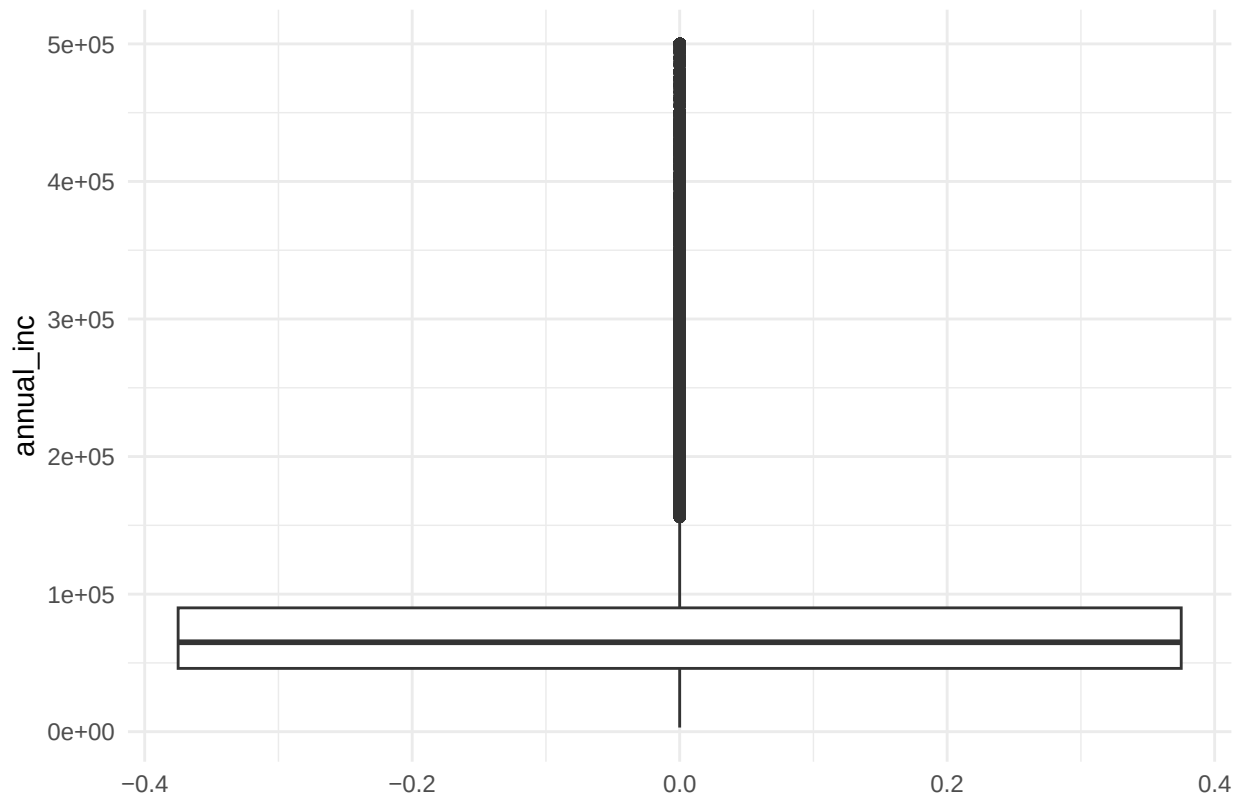
```
# Fix annual income values greater than 500'000 by dividing by 100, for the same rows, set dti to NA
cleaned_data <- cleaned_data %>%
  mutate(dti = ifelse(annual_inc > 500000, NA, dti),
         annual_inc = ifelse(annual_inc > 500000, annual_inc / 100, annual_inc)
  )

# Show distribution and boxplot of annual income after fixing
ggplot(cleaned_data, aes(x = annual_inc)) +
  geom_histogram(bins = 30) +
  ggtitle("Histogram of Annual Income (After Fixing)") +
  theme_minimal()
```



```
ggplot(cleaned_data, aes(y = annual_inc)) +
  geom_boxplot() +
  ggtitle("Boxplot of Annual Income (After Fixing)") +
  theme_minimal()
```

Boxplot of Annual Income (After Fixing)



```
# Show highest values of annual income after fixing
extreme_annual_inc_after <- show_extreme_values(cleaned_data, "annual_inc", n = 10)
print("Highest 10 annual incomes after fixing:")
```

```
## [1] "Highest 10 annual incomes after fixing:"
```

```
print(extreme_annual_inc_after$highest)
```

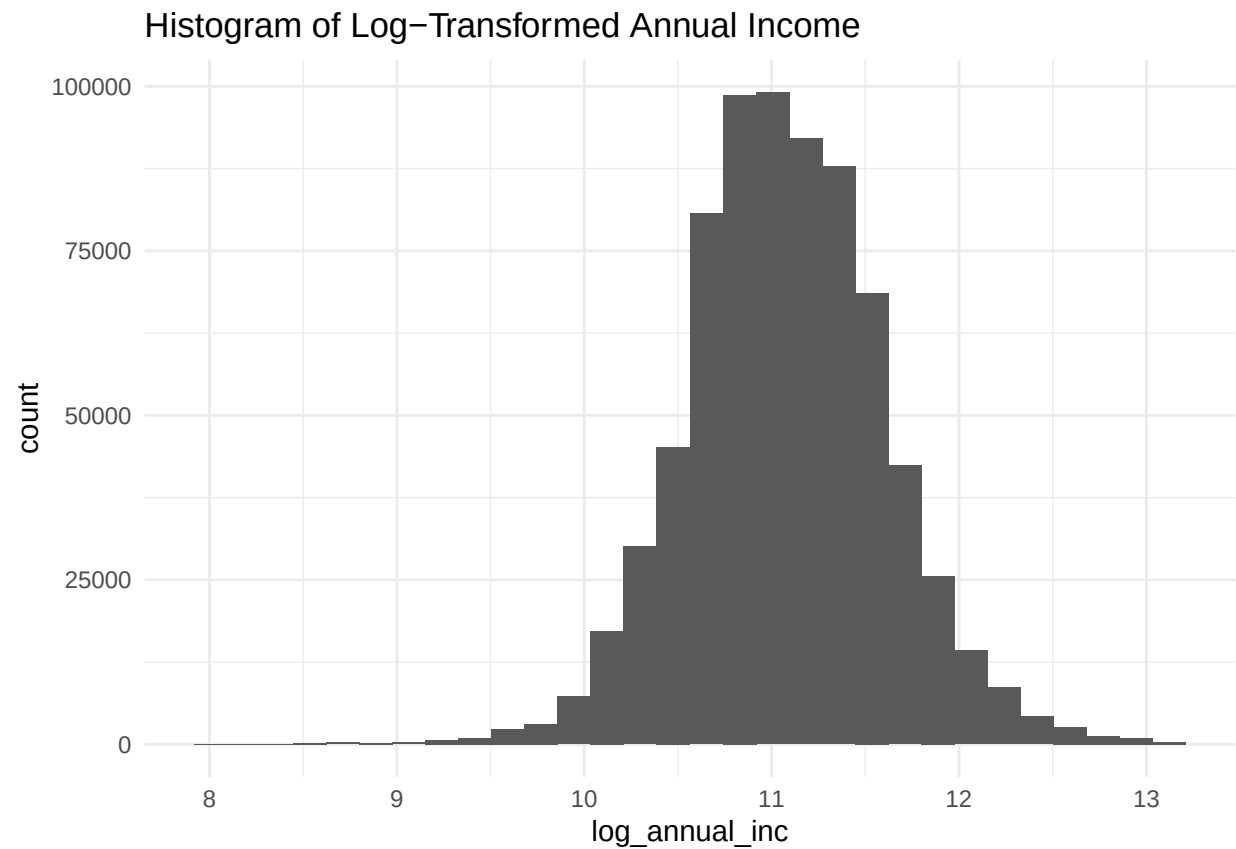
```
##           id member_id loan_amnt term int_rate emp_title
## 734816 54316460 57897267 35000    60 27.88 owner/operator
## 734817 11665436 13637587 28000    36  6.49 trader
## 734818 38639554 41423466 10000    36 11.99 Executive Director
## 734819 3705654 4677960 28000    60 12.12
## 734820 23945725 26318305 31975    60 10.99 Senior Vice President
## 734821 36281537 39002963 28000    60 15.59 Doctor
## 734822 17542934 19665478 25000    60 19.52 Managing Director
## 734823 65334676 69921489 24000    60 12.59 Finance Director
## 734824 55910724 59572525 24000    36  8.18 Executive Vice President
## 734825 7101774 8763460 35000    60 25.80 Evercore Partners
##           emp_length home_ownership annual_inc verification_status
## 734816 3 years MORTGAGE 5e+05 Not Verified
## 734817 3 years RENT 5e+05 Verified
## 734818 5 years RENT 5e+05 Verified
## 734819 10+ years MORTGAGE 5e+05 Verified
## 734820 9 years MORTGAGE 5e+05 Verified
```

## 734821	10+ years	MORTGAGE	5e+05	Source Verified	
## 734822	10+ years	MORTGAGE	5e+05	Source Verified	
## 734823	4 years	OWN	5e+05	Source Verified	
## 734824	2 years	RENT	5e+05	Source Verified	
## 734825	< 1 year	RENT	5e+05	Verified	
##					
## 734816					
## 734817					
## 734818					
## 734819	Borrower added on 03/08/13 > I need to get through two or three months of expenses, a few p				
## 734820					
## 734821					
## 734822					
## 734823					
## 734824					
## 734825	Borrower added on 09/07/13 > I recently had to move and				
##	purpose	zip_code	dti	delinq_2yrs	inq_last_6mths
## 734816	small_business	983xx	5.86	0	1
## 734817	major_purchase	100xx	0.20	0	0
## 734818	other	902xx	21.52	1	1
## 734819	debt_consolidation	913xx	7.33	0	0
## 734820	home_improvement	079xx	5.74	0	1
## 734821	renewable_energy	275xx	1.82	0	2
## 734822	debt_consolidation	380xx	15.16	1	0
## 734823	debt_consolidation	100xx	3.36	0	0
## 734824	debt_consolidation	752xx	12.41	0	2
## 734825	moving	100xx	4.10	0	1
##	mths_since_last_delinq	mths_since_last_record	open_acc	pub_rec	revol_bal
## 734816	NA	NA	11	0	45499
## 734817	NA	NA	4	0	3686
## 734818	8	NA	11	0	199801
## 734819	27	NA	14	0	205161
## 734820	NA	NA	11	0	23067
## 734821	36	11	7	1	8147
## 734822	6	NA	14	0	35372
## 734823	NA	NA	12	0	17759
## 734824	24	25	20	1	22916
## 734825	NA	NA	11	0	40852
##	revol_util	total_acc	collections_12_mths_ex_med		
## 734816	86.7	13	0		
## 734817	92.8	12	0		
## 734818	30.4	26	0		
## 734819	64.2	34	0		
## 734820	45.7	33	0		
## 734821	46.6	24	0		
## 734822	77.4	38	0		
## 734823	54.8	13	0		
## 734824	80.1	56	0		
## 734825	80.9	20	0		
##	mths_since_last_major_derog	policy_code	acc_now_delinq	tot_coll_amt	
## 734816	NA	1	0	0	
## 734817	NA	1	0	0	
## 734818	NA	1	0	0	
## 734819	NA	1	0	0	

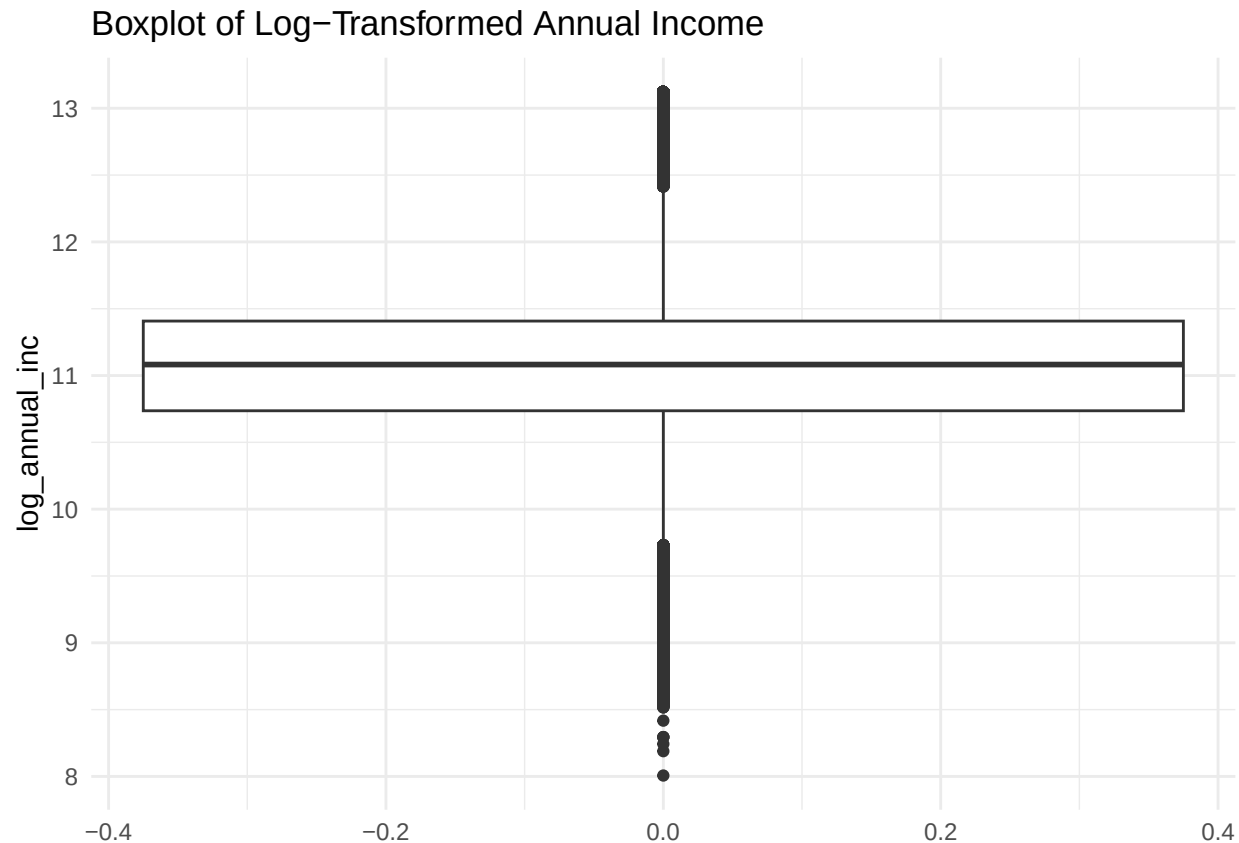

```
## 734820      NA      1      0      0
## 734821      NA      1      0      0
## 734822      NA      1      0      0
## 734823      NA      1      0      0
## 734824      24      1      0      0
## 734825      NA      1      0      0
##      tot_cur_bal total_rev_hi_lim log_loan_amnt
## 734816      102855      52500      10.46313
## 734817       4192      43300      10.24000
## 734818      338229      224500      9.21044
## 734819      1336021      223000      10.24000
## 734820      1165367      130500      10.37274
## 734821      2073792      17500      10.24000
## 734822      556610      45700      10.12667
## 734823       40021      32400      10.08585
## 734824      400396      28600      10.08585
## 734825       62184      47700      10.46313
```

```
# Add log-transformed annual income column
cleaned_data <- cleaned_data %>%
  mutate(log_annual_inc = log(annual_inc + 1)) # Adding

# Check histogram and boxplot of log-transformed annual income
ggplot(cleaned_data, aes(x = log_annual_inc)) +
  geom_histogram(bins = 30) +
  ggtitle("Histogram of Log-Transformed Annual Income") +
  theme_minimal()
```



```
ggplot(cleaned_data, aes(y = log_annual_inc)) +  
  geom_boxplot() +  
  ggtitle("Boxplot of Log-Transformed Annual Income") +  
  theme_minimal()
```



The exploratory analysis of `annual_inc` revealed a highly right-skewed distribution where the majority of borrowers cluster around a median income while a small number of high earners extend the tail significantly. This skewness poses two primary issues for linear modeling. It violates the assumption of normality required for valid hypothesis testing, and it allows high-leverage outliers to disproportionately influence the regression coefficients.

To address this, we applied a natural logarithm transformation: $\log(x+1)$. The addition of 1 ensures the function remains defined for zero-income cases. As seen in the transformed visualizations, this operation successfully compresses the extreme right tail and expands the lower range. The resulting distribution is symmetric and bell-shaped, closely approximating a normal distribution.

```
summary(cleaned_data$annual_inc)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      3000  45999   65000   74466   90000   500000
```

Since the data is still right-skewed, one could consider to scale this feature before modelling.

```
summary(cleaned_data$dti)
```

2.4.1.3 debt-to-income ratio (dti)

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NA's
##      0.00  12.21   17.98   18.47   24.37   39.99    845
```

```
extreme_dti <- show_extreme_values(na.omit(cleaned_data), "dti", n = 5)
print("Lowest 5 dti values:")
```

```
## [1] "Lowest 5 dti values:"
```

```
print(extreme_dti$lowest)
```

```
##          id member_id loan_amnt term int_rate emp_title
## 1 60705217 64725931      8000   36    9.17 Print Machine Especialist
## 2 64691962 69202699     10000   60    15.41      Rural Carrier
## 3 24424866 26847812     20000   36    12.49    Clinical asst. 2
## 4 66542505 71268281      4000   36    11.99    Plumbing Mechanic
## 5 62155806 66352530     25200   60    12.29      Owner/CEO
##  emp_length home_ownership annual_inc verification_status desc
## 1      2 years          RENT      30000          Verified
## 2     10+ years          OWN      49500        Not Verified
## 3      4 years    MORTGAGE     98997          Verified
## 4      3 years          RENT      94000          Verified
## 5      2 years    MORTGAGE     69600        Not Verified
##      purpose zip_code dti delinq_2yrs inq_last_6mths
## 1 major_purchase  370xx    0         0         0
## 2 home_improvement 631xx    0         0         0
## 3 home_improvement 951xx    0         0         1
## 4      other      117xx    0         2         1
## 5 home_improvement 301xx    0         0         0
##  mths_since_last_delinq mths_since_last_record open_acc pub_rec revol_bal
## 1          65          30          5          1          0
## 2          61          57          4          1          0
## 3          40          70          6          1          0
## 4          21          69          7          1          0
## 5          38          80          5          2          0
##  revol_util total_acc collections_12_mths_ex_med mths_since_last_major_derog
## 1          0          24          0          65
## 2          0          16          0          61
## 3          0          24          0          40
## 4          0          14          0          21
## 5          0          26          0          38
##  policy_code acc_now_delinq tot_coll_amt tot_cur_bal total_rev_hi_lim
## 1          1          0          486          41233          15000
## 2          1          0          0          134877          4800
## 3          1          0          0          882065          9500
## 4          1          0          0          0          15600
## 5          1          0          0          0          13900
##  log_loan_amnt log_annual_inc
## 1      8.987322      10.30899
## 2      9.210440      10.80975
## 3      9.903538      11.50285
## 4      8.294300      11.45106
## 5     10.134639      11.15053
```

```
print("Highest 5 dti values:")
```

```
## [1] "Highest 5 dti values:"
```

```
print(extreme_dti$highest)
```

```
##           id member_id loan_amnt term int_rate emp_title
## 34311 54346026 57926811   12000   60   13.33      Teacher
## 34312 30615872 33169059   21600   36    8.39      Psychologist
## 34313 55268298 58849016    2000   36   15.61      Educator
## 34314 49956509 53325333   13650   60   22.99      Production
## 34315 62347775 66544534   12000   60   17.57 Group life Coordinator
##      emp_length home_ownership annual_inc verification_status desc
## 34311      9 years          RENT      34000      Source Verified
## 34312     10+ years          OWN      77000          Verified
## 34313      8 years     MORTGAGE      50000      Not Verified
## 34314     10+ years     MORTGAGE      37960      Source Verified
## 34315      2 years          RENT      43000          Verified
##      purpose zip_code   dti delinq_2yrs inq_last_6mths
## 34311 debt_consolidation 305xx 39.96          0          0
## 34312 debt_consolidation 341xx 39.97          1          0
## 34313  home_improvement 799xx 39.98          2          0
## 34314  home_improvement 553xx 39.99          0          1
## 34315  major_purchase 971xx 39.99          0          1
##      mths_since_last_delinq mths_since_last_record open_acc pub_rec revol_bal
## 34311                62                81        16        2       2849
## 34312                21                16         6        2       9756
## 34313                 6                78        10        1       2696
## 34314                81                50         6        1       3486
## 34315                35                63        12        1       5172
##      revol_util total_acc collections_12_mths_ex_med
## 34311        12.0         52                0
## 34312        45.6         25                1
## 34313        35.5         33                0
## 34314        77.5         14                0
## 34315        54.4         43                0
##      mths_since_last_major_derog policy_code acc_now_delinq tot_coll_amt
## 34311                66                1                0                0
## 34312                21                1                0                93
## 34313                 6                1                0                0
## 34314                81                1                0                0
## 34315                67                1                0               898
##      tot_cur_bal total_rev_hi_lim log_loan_amnt log_annual_inc
## 34311        50569         23800      9.392745      10.43415
## 34312        10298         21400      9.980495      11.25157
## 34313       132928          7600      7.601402      10.81980
## 34314        52389          4500      9.521568      10.54431
## 34315        56618          9500      9.392745      10.66898
```

The variable dti (debt-to-income ratio) is another field that exhibits irregularities. After setting dti to NA for rows in which annual_inc was either 0 or exceeded \$500,000 (because those income values were clearly corrupted), we examined the remaining distribution. Since installment is not available in this dataset, recalculating DTI from first principles is not possible. This limits the ability to validate each individual entry directly, so data quality assessment must rely on a combination of statistical outlier detection and domain knowledge. DTI values > 100% are implausible for approved consumer loans and contradict typical

underwriting standards, where DTIs above 60–70% are rarely accepted. The fact that the associated `int_rate` values for these rows are not exceptionally high further suggests that these DTIs do not reflect real borrower risk but are instead the result of data errors or import inconsistencies. Therefore, DTI values above 100% were set to NA for later imputation if desired. On the lower end, `dti == 0` is also highly unlikely for the majority of loan applications, particularly when the stated loan purpose is `debt_consolidation`. This purpose implies existing financial obligations, so a DTI of zero contradicts the expected semantics of the loan record. To investigate this pattern, we inspected the distribution of loan purposes among rows with `dti == 0`, which supports the conclusion that many of these entries are erroneous or incomplete. To complement domain-based reasoning, we also examined statistical outliers using the standard criterion of $Q3 + 1.5 \times IQR$. The computed upper bound identifies additional entries far outside the expected range for DTI in consumer lending. Since these values cannot be recalculated due to the absence of installment or any alternative variable representing monthly debt payments, these extreme values cannot be corrected directly. Based on both statistical outlier detection and domain plausibility, they were set to NA. This ensures that clearly invalid DTI values do not influence the modeling process while retaining the remainder of the dataset for downstream analysis.

```
# How many values are above upper bound (Q3 + 1.5*IQR) for dti
dti_Q3 <- quantile(cleaned_data$dti, 0.75, na.rm = TRUE)
dti_IQR <- IQR(cleaned_data$dti, na.rm = TRUE)
dti_upper_bound <- dti_Q3 + 1.5 * dti_IQR
num_dti_above_upper_bound <- sum(cleaned_data$dti > dti_upper_bound, na.rm = TRUE)
print(paste("Number of dti values above upper bound:", num_dti_above_upper_bound))
```

```
## [1] "Number of dti values above upper bound: 0"
```

```
print(dti_upper_bound)
```

```
## 75%
## 42.61
```

```
### Identify unreasonable values
# Domain logic:
# - DTI > 100% is implausible for approved loans
# - DTI == 0 is implausible for purposes like debt_consolidation

implausible_dti <- cleaned_data %>%
  mutate(
    dti_invalid = case_when(
      dti > 100 ~ TRUE,          # implausible high DTI
      dti == 0 ~ TRUE,          # implausible zero DTI
      TRUE ~ FALSE
    )
  )

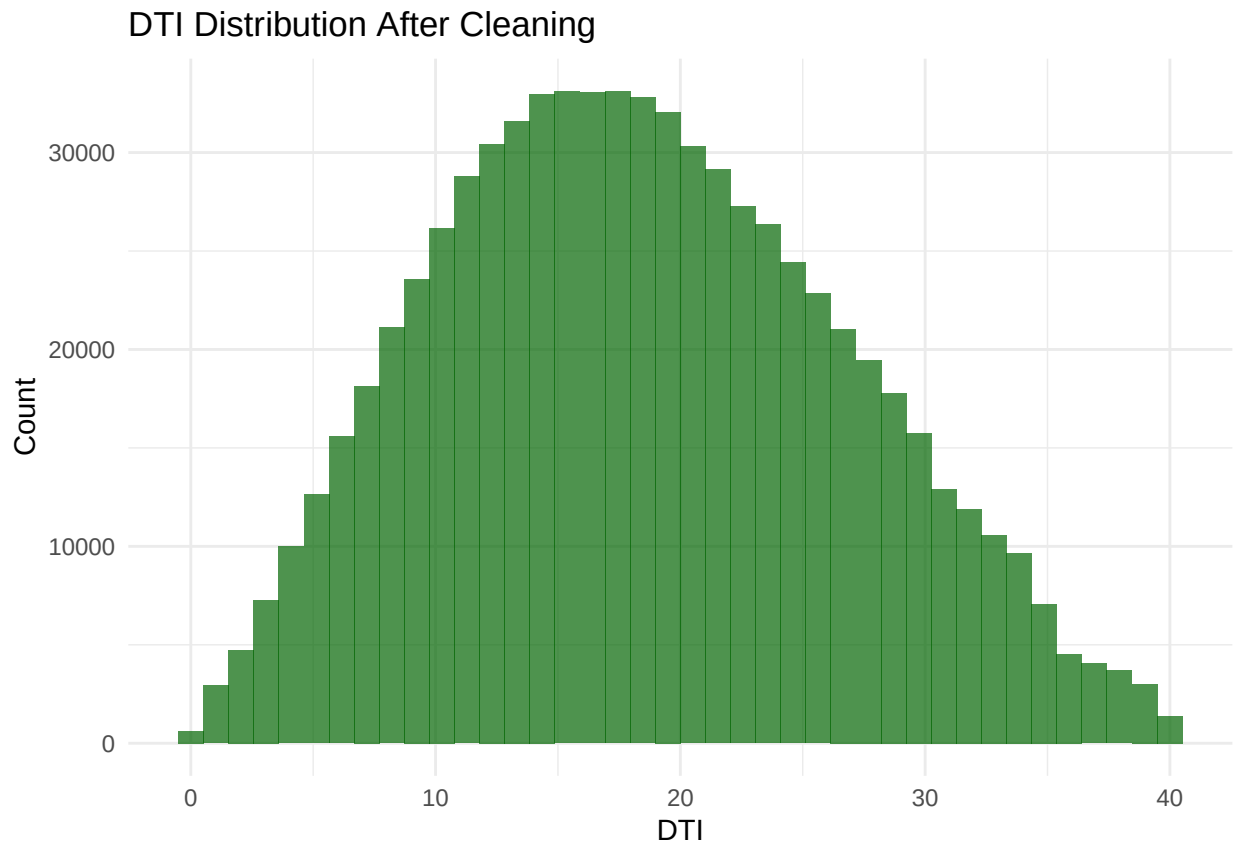
table(implausible_dti$dti_invalid) # 197 rows true
```

```
##
## FALSE TRUE
## 734628 197
```

```
### Set implausible values to NA
cleaned_data <- cleaned_data %>%
  mutate(
    dti = ifelse(dti > 100, NA, dti),
    dti = ifelse(dti == 0, NA, dti)
  )
```

```
plot_after <- ggplot(cleaned_data, aes(x = dti)) +
  geom_histogram(bins = 40, fill = "darkgreen", alpha = 0.7) +
  labs(
    title = "DTI Distribution After Cleaning",
    x = "DTI",
    y = "Count"
  ) +
  theme_minimal()
```

plot_after



```
# Inspect contradictions between delinq_2yrs and mths_since_last_delinq
extreme_delinq <- cleaned_data %>%
  filter(delinq_2yrs > 0 & mths_since_last_delinq > 24)
```

```
# Print number of rows
cat("Number of rows:", nrow(extreme_delinq), "\n")
```

2.4.1.4 Delinquencies

```
## Number of rows: 1261
```

```
# Print first rows only
head(extreme_delinq)
```

```
##      id member_id loan_amnt term int_rate emp_title emp_length
## 1 66004347 70699062    20000   36    5.32  Psychoanalyst 10+ years
## 2 65129510 69565280    15000   36    8.18  General Manager   3 years
## 3 60677529 64698303    22000   60   16.55  Branch Manager 10+ years
## 4 58340494 62159214    13800   60   15.61  Office Manager 10+ years
## 5 46436300 49574017    35000   36   13.99 VP- Product Assurance 10+ years
## 6 61742332 65912047    13600   60   20.99                <NA>
##  home_ownership annual_inc verification_status desc purpose
## 1      MORTGAGE    150000      Not Verified      credit_card
## 2      MORTGAGE    75000      Not Verified      credit_card
## 3        RENT     73000      Verified      debt_consolidation
## 4        RENT     75000      Verified      debt_consolidation
## 5      MORTGAGE   317000      Verified      credit_card
## 6      MORTGAGE    60000      Verified      credit_card
##  zip_code  dti delinq_2yrs inq_last_6mths mths_since_last_delinq
## 1    100xx 22.44         1         1         49
## 2    346xx  5.14         2         2         48
## 3    064xx 16.70         1         2         80
## 4    913xx 12.48         1         5         82
## 5    078xx 18.53         1         0         58
## 6    169xx 34.34         1         3         27
##  mths_since_last_record open_acc pub_rec revol_bal revol_util total_acc
## 1             NA         8         0    355571         98         33
## 2             27         4         1     5579         35         14
## 3             43        10         1     6474         49         41
## 4             53        16         1     4076         26         28
## 5             NA        20         0    30568         79         42
## 6             NA        14         0     7006         69         37
##  collections_12_mths_ex_med mths_since_last_major_derog policy_code
## 1             0             38         1
## 2             0             24         1
## 3             0             89         1
## 4             0             18         1
## 5             0             18         1
## 6             1             51         1
##  acc_now_delinq tot_coll_amt tot_cur_bal total_rev_hi_lim log_loan_amnt
## 1             0             0    643178    363600     9.903538
## 2             0             0    19900    16000     9.615872
## 3             0             0    21918    13100     9.998843
## 4             0             0    18869    15950     9.532496
## 5             0             0   844206    38610    10.463132
## 6             0            187   185777    10100     9.517899
##  log_annual_inc
```



```
## 1      11.91840
## 2      11.22526
## 3      11.19823
## 4      11.22526
## 5      12.66666
## 6      11.00212
```

The variables `delinq_2yrs` (number of delinquencies in the past 24 months) and `mths_since_last_delinq` (months since the most recent delinquency) both describe aspects of a borrower's delinquency history and must therefore be internally consistent. A borrower who had at least one delinquency in the past 24 months (`delinq_2yrs > 0`) must have a corresponding `mths_since_last_delinq` value between 0 and 24. Any value above 24 months indicates a contradiction, typically originating from stale bureau data or import inconsistencies. Because `delinq_2yrs` represents a stable bureau summary metric, it is considered more reliable than the more granular timing field. Therefore, we treat `mths_since_last_delinq` as incorrect in such cases and set it to NA for later imputation.

```
cleaned_data <- cleaned_data %>%
  mutate(mths_since_last_delinq = ifelse(delinq_2yrs > 0 &
    mths_since_last_delinq > 24,
    NA,
    mths_since_last_delinq))

# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

We then examine the opposite contradiction: cases where `mths_since_last_delinq == 0` indicates a delinquency in the current reporting period, yet `delinq_2yrs == 0` suggests no recent delinquencies. A zero-month value is highly specific and therefore more trustworthy than a summary count that may not have been updated in time. In these cases, we correct the count by setting `delinq_2yrs` to 1.

```
suspicious_rows_zero <- cleaned_data %>%
  filter(mths_since_last_delinq == 0 & delinq_2yrs == 0)
print(suspicious_rows_zero) # 1 rows
```

```
##      id member_id loan_amnt term int_rate emp_title emp_length
## 1 54608628 58179343   20000  36   16.55 Manager of IT    < 1 year
##   home_ownership annual_inc verification_status desc      purpose
## 1      RENT      75000      Verified      debt_consolidation
##   zip_code  dti delinq_2yrs inq_last_6mths mths_since_last_delinq
## 1   530xx 28.34      0      1      0
##   mths_since_last_record open_acc pub_rec revol_bal revol_util total_acc
## 1      NA      26      0   15020      22      40
##   collections_12_mths_ex_med mths_since_last_major_derog policy_code
## 1      0      NA      1
##   acc_now_delinq tot_coll_amt tot_cur_bal total_rev_hi_lim log_loan_amnt
## 1      0      0   145260   66794   9.903538
##   log_annual_inc
## 1   11.22526
```

```
cleaned_data <- cleaned_data %>%
  mutate(delinq_2yrs = ifelse(!is.na(mths_since_last_delinq) & mths_since_last_delinq == 0 &
    delinq_2yrs == 0,
    1,
    delinq_2yrs))

# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

A further consistency check is performed using `acc_now_delinq`, which counts currently delinquent accounts. A borrower cannot have `acc_now_delinq > 0` while simultaneously reporting `delinq_2yrs == 0`, nor can the time since last delinquency be greater than zero. Current delinquency is the strongest and most precise indicator; therefore, contradictions involving `acc_now_delinq` must be resolved by correcting or removing inconsistent timing information.

```
# Check distinct values of acc_now_delinq
table(cleaned_data$acc_now_delinq)
```

```
##
##      0      1      2      3      4      5      6     14
## 731102 3499  187   26    6    3    1    1
```

```
# Current delinquency contradicting long time-since
current_vs_recency <- cleaned_data %>%
  filter(acc_now_delinq > 0 & mths_since_last_delinq > 0)

# Print number of rows
cat("Number of rows:", nrow(current_vs_recency), "\n")
```

```
## Number of rows: 2905
```

```
# Inspect first rows only
head(current_vs_recency)
```

```
##      id member_id loan_amnt term int_rate emp_title
## 1 55150437 58721233  18000  36    7.26  Quality Assurance Manager
## 2 36069691 38781355  10000  60   23.99                csr
## 3 58592995 62433742  30000  60   18.25                excutive chef
## 4 45444371 48552094  12000  60   15.61 Quality Engineering Manager
## 5 5028081  6330344  18000  60   23.83                Crescent Processing
## 6 59779210 63722922  14175  60   22.99                Utility 3 worker
##  emp_length home_ownership annual_inc verification_status
## 1      6 years      MORTGAGE  100000      Source Verified
## 2      4 years      RENT      38000      Source Verified
## 3      7 years      RENT      74500      Verified
## 4      2 years      MORTGAGE  112914      Verified
## 5      1 year      RENT      80000      Verified
## 6     10+ years      RENT     125000      Verified
##
```

```

## 1
## 2
## 3
## 4
## 5 Borrower added on 05/07/13 > To reduce my monthly expenses by consolidating my remainig debts in
## 6
##      purpose zip_code   dti delinq_2yrs inq_last_6mths
## 1 debt_consolidation  853xx 12.17         1         1
## 2   major_purchase   785xx 17.94         1         0
## 3 debt_consolidation  681xx 17.59         2         1
## 4 debt_consolidation  390xx  2.54         2         1
## 5 debt_consolidation  750xx 13.80         1         1
## 6 debt_consolidation  945xx 11.79         1         0
##      mths_since_last_delinq mths_since_last_record open_acc pub_rec revol_bal
## 1                        1                NA        12      0    16677
## 2                        1                NA         9      0     2600
## 3                        1                NA         7      0     9385
## 4                        1                NA         9      0     5627
## 5                        2                91        12      1     6043
## 6                        1                NA        10      0     6777
##      revol_util total_acc collections_12_mths_ex_med mths_since_last_major_derog
## 1         24.1      31                0                                NA
## 2         44.8      32                0                                NA
## 3         58.3      26                0                                 1
## 4         93.8      25                0                                21
## 5         48.7      56                0                                 2
## 6         38.9      17                0                                 5
##      policy_code acc_now_delinq tot_coll_amt tot_cur_bal total_rev_hi_lim
## 1             1             1             0     252482        69200
## 2             1             1             0     46020         5800
## 3             1             1            71     23181        16100
## 4             1             1            77    485626         6000
## 5             1             1             0     252482        12400
## 6             1             1             0     35172        17400
##      log_loan_amnt log_annual_inc
## 1      9.798183      11.51294
## 2      9.210440      10.54537
## 3     10.308986      11.21857
## 4      9.392745      11.63439
## 5      9.798183      11.28979
## 6      9.559306      11.73608

```

```

# Current delinquency contradicting zero count
cleaned_data %>%
  filter(acc_now_delinq > 0 & delinq_2yrs == 0) # 0 rows

```

```

## [1] id          member_id
## [3] loan_amnt   term
## [5] int_rate    emp_title
## [7] emp_length  home_ownership
## [9] annual_inc  verification_status
## [11] desc        purpose
## [13] zip_code    dti
## [15] delinq_2yrs inq_last_6mths

```

```
## [17] mths_since_last_delinq      mths_since_last_record
## [19] open_acc                      pub_rec
## [21] revol_bal                     revol_util
## [23] total_acc                     collections_12_mths_ex_med
## [25] mths_since_last_major_derog  policy_code
## [27] acc_now_delinq                tot_coll_amt
## [29] tot_cur_bal                   total_rev_hi_lim
## [31] log_loan_amnt                 log_annual_inc
## <0 rows> (or 0-length row.names)
```

```
# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

Since these cases cannot be reliably corrected without full credit bureau histories, we conservatively mark inconsistent timing information as missing.

```
cleaned_data <- cleaned_data %>%
mutate(mths_since_last_delinq = ifelse(acc_now_delinq > 0 &
  mths_since_last_delinq > 0,
  0,
  mths_since_last_delinq))
```

```
# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

After cleaning, the delinquency fields show internally coherent patterns. However, the presence of NA values creates challenges for predictive modeling. Importantly, NAs in `mths_since_last_delinq` do not represent missing data but rather the legitimate case of “no delinquencies on record.” To make the data suitable for modeling without inappropriate imputation, we convert delinquency fields to categorical indicators. This approach aligns with industry-standard credit risk modeling practices and avoids encoding NA as missing risk signals. We first bucketize `delinq_2yrs` into an ordered categorical variable:

```
cleaned_data <- cleaned_data %>%
mutate(delinq_2yrs_cat = case_when(
  delinq_2yrs == 0 ~ "0",
  delinq_2yrs == 1 ~ "1",
  delinq_2yrs == 2 ~ "2",
  delinq_2yrs >= 3 ~ "3+",
  TRUE ~ NA_character_
)) %>%
mutate(delinq_2yrs_cat = factor(delinq_2yrs_cat,
  levels = c("3+", "2", "1", "0"),
  ordered = TRUE))
```

We then convert `mths_since_last_delinq` into recency buckets, including an explicit category for borrowers with no delinquency history:

```

cleaned_data <- cleaned_data %>%
  mutate(delinq_recency = case_when(
    is.na(mths_since_last_delinq) ~ "none",
    mths_since_last_delinq == 0 ~ "0",
    mths_since_last_delinq <= 6 ~ "1-6",
    mths_since_last_delinq <= 12 ~ "7-12",
    mths_since_last_delinq <= 24 ~ "13-24",
    mths_since_last_delinq > 24 ~ "24+",
    TRUE ~ NA_character_
  )) %>%
  mutate(delinq_recency = factor(delinq_recency,
                                levels = c("0", "1-6", "7-12", "13-24", "24+", "none"),
                                ordered = TRUE))

```

Finally, we convert `acc_now_delinq` into a binary indicator, which is standard practice in credit-risk modeling. Although the raw variable can take values greater than 1, the vast majority of observations are zero, and higher counts occur so rarely that they do not provide stable or meaningful predictive signal. In consumer credit data, the presence of any currently delinquent account already represents a severe and immediate risk event. Distinguishing between one, two, or more simultaneous delinquencies offers little additional predictive value but introduces unnecessary noise due to extremely small sample sizes in the higher categories. By collapsing all values greater than zero into a single category, we achieve the following:

- Improved model stability: Rare categories such as 3, 4, 5, or 14 delinquencies occur in only a handful of cases and would lead to unreliable parameter estimates or unstable tree splits.
- Preservation of the risk signal: From a business and regulatory perspective, “currently delinquent” vs. “not currently delinquent” captures the full decision-relevant information.
- Reduced variance and better generalization: Binning prevents overfitting to rare outliers and yields smoother, more robust model behavior.
- Consistency with industry scorecards: Most commercial scorecards (e.g., FICO) use a binary indicator for current delinquency rather than the raw count. This transformation ensures that the feature reflects its true predictive meaning while maintaining statistical reliability.

```

cleaned_data <- cleaned_data %>%
  mutate(acc_now_delinq_bin = ifelse(acc_now_delinq > 0, 1, 0))

cleaned_data <- cleaned_data %>%
  mutate(acc_now_delinq_bin = factor(ifelse(acc_now_delinq > 0, "1", "0"),
                                     levels = c("0", "1")))

cleaned_data <- cleaned_data %>% select(-acc_now_delinq)

# amount of rows after mutation
print(nrow(cleaned_data))

```

```
## [1] 734825
```

After applying these transformations, delinquency information is internally consistent, free of logical contradictions, and model-ready without requiring imputation.

```
view(cleaned_data)
```

2.4.1.5 Inquiries in last 6 months (`Inq_last_6mths`) The variable `inq_last_6mths` reports how many hard credit inquiries the borrower received within the past six months. Since this is a discrete count

of bureau events, it should take small integer values. In typical consumer credit datasets, most borrowers have between 0 and 4 recent inquiries, with gradually fewer cases above that range. Before applying any plausibility or consistency checks, we first examine the raw frequency distribution to understand how the values are spread and whether implausible or unexpected patterns appear.

```
# Count distinct values of inq_last_6mths
inq_counts <- cleaned_data %>%
  group_by(inq_last_6mths) %>%
  summarise(n = n()) %>%
  arrange(inq_last_6mths)

inq_counts
```

```
## # A tibble: 9 x 2
##   inq_last_6mths      n
##         <int> <int>
## 1             0 417577
## 2             1 199894
## 3             2  75910
## 4             3  29045
## 5             4   8639
## 6             5   3002
## 7             6    754
## 8             7     3
## 9             8     1
```

2.4.1.6 Months since last record (mths_since_last_record) & public records (pub_rec) The variables `mths_since_last_record` and `pub_rec` describe the borrower's public-record history and therefore must follow strict internal consistency rules. A non-missing value in `mths_since_last_record` (whether 0 or greater) indicates that the borrower has had at least one public record in the past, meaning that `pub_rec` must be at least 1. An NA in `mths_since_last_record` is not a missing value in the statistical sense but represents the meaningful case of "no public record on file," so `pub_rec` must be 0. Furthermore, if `mths_since_last_record` equals 0, a new public record has been reported in the current cycle, which again requires that `pub_rec` is at least 1. Based on these rules, we first search for violations where a positive month value appears together with a zero or missing public-record count, which would indicate an impossible history.

```
suspicious_pub_record_rows <- cleaned_data %>%
  filter(mths_since_last_record > 0 & (is.na(pub_rec) | pub_rec == 0))
suspicious_pub_record_rows # 0 rows
```

```
## [1] id                member_id
## [3] loan_amnt          term
## [5] int_rate           emp_title
## [7] emp_length         home_ownership
## [9] annual_inc         verification_status
## [11] desc              purpose
## [13] zip_code           dti
## [15] delinq_2yrs        inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc           pub_rec
## [21] revol_bal          revol_util
```

```
## [23] total_acc                collections_12_mths_ex_med
## [25] mths_since_last_major_derog policy_code
## [27] tot_coll_amt             tot_cur_bal
## [29] total_rev_hi_lim         log_loan_amnt
## [31] log_annual_inc           delinq_2yrs_cat
## [33] delinq_recency           acc_now_delinq_bin
## <0 rows> (or 0-length row.names)
```

No contradictions of this type appeared in the dataset. Next, we examine rows where `mths_since_last_record` equals 0 but `pub_rec` is 0, which incorrectly implies a new public record with a record count of zero. These rows require correction by setting `pub_rec` to 1.

```
suspicious_pub_record_rows_0 <- cleaned_data %>%
  filter(mths_since_last_record == 0 & pub_rec == 0)
suspicious_pub_record_rows_0 # 0 rows
```

```
## [1] id                member_id
## [3] loan_amnt         term
## [5] int_rate          emp_title
## [7] emp_length        home_ownership
## [9] annual_inc        verification_status
## [11] desc             purpose
## [13] zip_code          dti
## [15] delinq_2yrs       inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc          pub_rec
## [21] revol_bal         revol_util
## [23] total_acc         collections_12_mths_ex_med
## [25] mths_since_last_major_derog policy_code
## [27] tot_coll_amt      tot_cur_bal
## [29] total_rev_hi_lim  log_loan_amnt
## [31] log_annual_inc    delinq_2yrs_cat
## [33] delinq_recency    acc_now_delinq_bin
## <0 rows> (or 0-length row.names)
```

```
cleaned_data <- cleaned_data %>%
  mutate(pub_rec = ifelse(mths_since_last_record == 0 & pub_rec == 0, 1, pub_rec))
```

We then verify that no cases exist where `mths_since_last_record` is NA but `pub_rec` is greater than 0, which would suggest that a public record exists but the time stamp is missing.

```
suspicious_pub_record_rows_na <- cleaned_data %>%
  filter(is.na(mths_since_last_record) & pub_rec > 0)
suspicious_pub_record_rows_na # 0 rows
```

```
## [1] id                member_id
## [3] loan_amnt         term
## [5] int_rate          emp_title
## [7] emp_length        home_ownership
## [9] annual_inc        verification_status
## [11] desc             purpose
## [13] zip_code          dti
```

```
## [15] delinq_2yrs          inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc              pub_rec
## [21] revol_bal             revol_util
## [23] total_acc              collections_12_mths_ex_med
## [25] mths_since_last_major_derog policy_code
## [27] tot_coll_amt           tot_cur_bal
## [29] total_rev_hi_lim       log_loan_amnt
## [31] log_annual_inc          delinq_2yrs_cat
## [33] delinq_recency          acc_now_delinq_bin
## <0 rows> (or 0-length row.names)
```

Since any non-missing value of `mths_since_last_record` must imply at least one public record, rows where the month value is present but `pub_rec` is NA are inconsistent as well. These NA values are corrected by assigning `pub_rec = 1` because the time stamp indicates the presence of at least one public record.

```
cleaned_data <- cleaned_data %>%
  filter(!is.na(pub_rec) & !is.na(mths_since_last_record))

# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

After these corrections, the relationship between `mths_since_last_record` and `pub_rec` is logically coherent. We then inspect the distribution of values to ensure no remaining anomalies.

```
cleaned_data %>%
  arrange(mths_since_last_record) %>%
  select(loan_amnt, mths_since_last_record, pub_rec, int_rate, term) %>%
  head(20)
```

```
##   loan_amnt mths_since_last_record pub_rec int_rate term
## 1    20000                0          1    10.49   36
## 2    35000                0          3    13.99   60
## 3    20000                0          4    12.05   60
## 4    16000                0          4    13.35   60
## 5     3600                0          1    13.98   36
## 6    15000                0          1    15.61   60
## 7    15000                0          1    12.29   36
## 8     4800                0          1    13.44   36
## 9    14000                1          1    12.99   36
## 10   15000                1          1     9.49   36
## 11    2000                1          1    10.99   36
## 12    5000                1          1     9.17   36
## 13   15000                1          1    13.65   36
## 14   15000                1          1    11.53   36
## 15   10000                1          1     6.49   36
## 16   16000                1          1     7.89   36
## 17   10000                1          6     9.17   36
## 18   25000                1          4    11.53   36
## 19    7000                1          5     8.67   36
## 20   20000                1          3     6.68   36
```



```
cleaned_data %>%
  arrange(desc(mths_since_last_record)) %>%
  select(loan_amnt, mths_since_last_record, pub_rec, int_rate, term) %>%
  head(20)
```

##	loan_amnt	mths_since_last_record	pub_rec	int_rate	term
## 1	10000	121	1	9.67	36
## 2	10000	121	1	13.53	36
## 3	3000	120	1	12.59	36
## 4	12000	120	1	13.33	36
## 5	5000	120	1	6.99	36
## 6	11200	120	1	5.93	36
## 7	25000	120	1	6.89	36
## 8	35000	120	1	23.99	60
## 9	24000	120	1	8.18	60
## 10	12000	119	1	7.62	36
## 11	15000	119	1	7.69	36
## 12	14000	119	1	7.90	36
## 13	16000	119	1	6.03	36
## 14	5000	119	1	7.89	36
## 15	12000	119	1	7.49	36
## 16	6000	119	1	7.89	36
## 17	2400	119	1	7.89	36
## 18	5000	119	1	6.68	36
## 19	16000	119	1	9.17	36
## 20	7500	119	1	12.49	36

```
cleaned_data %>%
  arrange(pub_rec) %>%
  select(loan_amnt, mths_since_last_record, pub_rec, int_rate, term) %>%
  head(20)
```

##	loan_amnt	mths_since_last_record	pub_rec	int_rate	term
## 1	8400	95	1	7.69	36
## 2	5000	57	1	6.03	36
## 3	6000	76	1	6.92	36
## 4	7200	115	1	8.90	36
## 5	10000	106	1	7.90	36
## 6	14400	86	1	6.03	36
## 7	7500	19	1	6.99	36
## 8	10000	61	1	5.93	36
## 9	20000	62	1	5.32	36
## 10	6000	101	1	7.62	36
## 11	11000	87	1	7.89	36
## 12	18000	4	1	7.49	36
## 13	17000	104	1	7.89	36
## 14	10000	85	1	7.90	36
## 15	20000	74	1	6.89	36
## 16	16000	69	1	7.89	36
## 17	12000	86	1	6.24	36
## 18	7500	75	1	7.12	36
## 19	4800	86	1	8.19	36
## 20	10000	70	1	7.89	36

```
cleaned_data %>%
  arrange(desc(pub_rec)) %>%
  select(loan_amnt, mths_since_last_record, pub_rec, int_rate, term) %>%
  head(20)
```

```
##      loan_amnt mths_since_last_record pub_rec int_rate term
## 1         8000                46      63    12.99   36
## 2        10000                23      54    15.31   36
## 3         4450                37      49    14.65   36
## 4         7125                36      49    13.67   36
## 5        10000                93      34    14.98   36
## 6         7000                61      28    11.49   36
## 7        18000                21      26    13.67   60
## 8        15000                11      22    22.99   60
## 9        15000                 6      21    10.99   36
## 10       21000                48      21    10.99   36
## 11       35000                15      21    16.24   36
## 12       32325                15      21    13.99   36
## 13       20000                38      20    18.49   60
## 14         5000                53      19    12.39   36
## 15         2000                11      19    16.99   36
## 16       16000                34      18     7.49   36
## 17         8725                55      18    18.25   36
## 18         2400                34      18    13.98   36
## 19         9025                42      18    16.55   36
## 20         8225                46      18    14.65   36
```

This process ensures that these two variables are mutually consistent and correctly represent the borrower's public-record history for downstream modeling.

PLEASE NOTE: Although internal consistency between `mths_since_last_record` and `pub_rec` was verified during exploratory analysis, the timing variable was excluded from the modeling pipeline due to feature semantics later. Consequently, no downstream feature corrections were applied based on this variable to ensure consistency with application-time data availability.

2.4.1.7 open accounts (`open_acc`) The variables `open_acc`, `total_acc`, `revol_bal`, `total_rev_hi_lim`, and `revol_util` describe the structure and utilization of the borrower's current revolving credit profile. These fields must satisfy a set of strict logical rules based on how credit files are constructed. First, the number of currently open accounts (`open_acc`) cannot exceed the total number of accounts ever opened (`total_acc`), because an account cannot be open without having existed in the first place. We therefore search for cases where `open_acc > total_acc`, which should be impossible.

```
suspicious_open_acc_rows <- cleaned_data %>%
  filter(open_acc > total_acc)
suspicious_open_acc_rows # 0 rows
```

```
## [1] id                member_id
## [3] loan_amnt         term
## [5] int_rate          emp_title
## [7] emp_length        home_ownership
## [9] annual_inc        verification_status
## [11] desc              purpose
```

```
## [13] zip_code          dti
## [15] delinq_2yrs        inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc           pub_rec
## [21] revol_bal          revol_util
## [23] total_acc          collections_12_mths_ex_med
## [25] mths_since_last_major_derog policy_code
## [27] tot_coll_amt       tot_cur_bal
## [29] total_rev_hi_lim   log_loan_amnt
## [31] log_annual_inc     delinq_2yrs_cat
## [33] delinq_recency     acc_now_delinq_bin
## <0 rows> (or 0-length row.names)
```

Zero such rows were identified. If any would have been identified, modifying the historical count of `total_acc` would risk fabricating a credit history, whereas `open_acc` is the value that violates a hard structural constraint. For this reason, the safest correction would have been to set `open_acc` to NA and let the general imputation strategy handle this variable later.

Rows where both values are NA provide no meaningful information about the borrower's credit file and cannot be logically reconstructed. Since only a small number of rows fall into this category, they are removed.

```
# Amount of rows with both open_acc and total_acc as NA
sum(is.na(cleaned_data$open_acc) & is.na(cleaned_data$total_acc))
```

```
## [1] 0
```

```
cleaned_data <- cleaned_data %>%
  filter(!(is.na(open_acc) & is.na(total_acc)))

# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

To verify the integrity of the corrected variables, we inspect the lowest and highest values for both `open_acc` and `total_acc`, ensuring that extreme cases remain plausible and that no further contradictions appear.

```
cleaned_data %>%
  arrange(open_acc) %>%
  select(loan_amnt, open_acc, total_acc, int_rate, term) %>%
  head(20)
```

```
##   loan_amnt open_acc total_acc int_rate term
## 1     3000      0         5     16.29  36
## 2     8000      0         6     14.98  36
## 3     9400      0         6     18.92  36
## 4     3900      0        10     22.47  36
## 5     3550      0         3     21.18  36
## 6    25000      0        15     18.92  60
## 7    12000      1        13     18.99  60
## 8     6000      1         6     11.67  36
## 9     1000      1         5     23.28  36
```

```
## 10      30000      1      8      12.69      36
## 11      30000      1     11      11.22      36
## 12       9000      1     12      10.16      36
## 13       8350      1     19       8.18      36
## 14      12000      1     13       7.89      36
## 15      14000      1      6       9.17      36
## 16       4000      1     12      14.64      36
## 17      12000      1      4      12.69      36
## 18      15000      1      7       6.49      36
## 19      21000      1     11       7.89      36
## 20      12000      1     10      12.85      36
```

```
cleaned_data %>%
  arrange(desc(open_acc)) %>%
  select(loan_amnt, open_acc, total_acc, int_rate, term) %>%
  head(20)
```

```
##      loan_amnt open_acc total_acc int_rate term
## 1      30000      90      107      12.39      60
## 2       5600      84      124      14.99      36
## 3      12000      82      140       9.99      36
## 4       9000      79      138      11.44      36
## 5       6000      75       99      11.67      36
## 6      33200      68       93      23.99      60
## 7      20000      67       83      14.48      36
## 8      16000      66       84      10.99      36
## 9      15000      65       90       7.26      36
## 10     35000      64      126      11.53      60
## 11     14000      63       84      12.29      36
## 12       4000      62       92       9.99      36
## 13     30000      62       98      15.22      36
## 14     24525      61       85      18.99      60
## 15     23000      60       94      13.66      60
## 16     11200      60       85      16.55      60
## 17     10000      59      113      12.39      36
## 18     24975      59       72       8.18      36
## 19     30000      58       86      11.67      36
## 20     35000      58       97      14.49      36
```

```
cleaned_data %>%
  arrange(total_acc) %>%
  select(loan_amnt, open_acc, total_acc, int_rate, term) %>%
  head(20)
```

```
##      loan_amnt open_acc total_acc int_rate term
## 1         1000      2          2      23.63      36
## 2        12500      2          2      14.47      36
## 3         9550      1          2      17.57      36
## 4         5000      1          2      18.49      36
## 5        12000      1          2      11.67      36
## 6         9600      1          2      11.99      36
## 7         2300      1          2      12.99      36
## 8        30000      2          2      21.00      60
```

```
## 9      6000      2      2    19.97    36
## 10     5000      2      2    11.67    36
## 11     5000      2      2    14.64    36
## 12     1500      2      2    15.10    36
## 13     5575      2      2    14.30    36
## 14     4500      2      2    14.99    36
## 15     1700      1      2    23.10    36
## 16     4375      2      2    24.99    36
## 17    12000      1      2    16.29    60
## 18    10000      2      2    15.31    60
## 19    10000      3      3    25.99    60
## 20     7800      3      3    12.49    36
```

```
cleaned_data %>%
  arrange(desc(total_acc)) %>%
  select(loan_amnt, open_acc, total_acc, int_rate, term) %>%
  head(20)
```

```
##      loan_amnt open_acc total_acc int_rate term
## 1      12175      37      169    17.57    36
## 2      35000      45      162    19.24    36
## 3      12075       8      156    17.86    36
## 4      35000      26      151    13.99    36
## 5      35000      40      150     8.67    36
## 6       3825      11      146    20.99    36
## 7      12000      82      140     9.99    36
## 8       9000      79      138    11.44    36
## 9      12000      47      137    20.99    60
## 10     24000      57      135    15.41    60
## 11     25000      48      129     7.89    60
## 12     20000      51      127    11.22    60
## 13     35000      64      126    11.53    60
## 14       3000      16      125    14.48    36
## 15       9600      33      124     9.17    36
## 16       5600      84      124    14.99    36
## 17     23200      43      124    17.86    60
## 18     17500      43      121    14.99    36
## 19     14000      37      119    13.35    60
## 20       5000      35      118    13.98    36
```

We then examine the revolving credit fields. The variable `revol_bal` represents the total outstanding balance on revolving lines, while `total_rev_hi_lim` reflects the total available revolving credit limit across all open accounts. Because a borrower cannot legally owe more than the total available limit, `revol_bal` must never exceed `total_rev_hi_lim`. Any row where the balance is greater than the maximum allowable credit limit represents a hard structural inconsistency in the data. Such cases must be corrected by setting `revol_bal` to NA, since the balance figure is the one that violates a fixed upper boundary.

```
# Revolving balance exceeding total revolving credit limit
suspicious_revol_bal_rows <- cleaned_data %>%
  filter(revol_bal > total_rev_hi_lim)

# Print number of rows
cat("Number of rows:", nrow(suspicious_revol_bal_rows), "\n") # 3'265 rows
```

```
## Number of rows: 3265
```

```
# Inspect first rows only  
head(suspicious_revol_bal_rows)
```

```
##      id member_id loan_amnt term int_rate emp_title  
## 1 43956738 46973516    15000   60    18.25      operator 1  
## 2 38436351 41220148    20325   60    18.54      Center Manager  
## 3 48394524 51643249    15000   60    20.49      Production Coordinator  
## 4 4196106  5338387    28000   60    21.49 Cravath, Swaine & Moore LLP  
## 5 68483376 73373101    16000   60    18.49      Financial Advisor  
## 6 3134587  3826738    23325   60    19.05 United States Gypsum Company  
##      emp_length home_ownership annual_inc verification_status  
## 1      9 years          RENT      34000          Verified  
## 2      < 1 year          RENT      51000          Verified  
## 3      < 1 year          RENT      48000          Verified  
## 4      6 years          RENT     250000          Verified  
## 5      4 years          RENT     120000          Verified  
## 6     10+ years          RENT     135000          Verified  
##  
## 1  
## 2  
## 3  
## 4  
## 5  
## 6      Borrower added on 01/15/13 > I would like to clean up old credit cards and continue w/ my on-line  
##      purpose zip_code   dti delinq_2yrs inq_last_6mths  
## 1 debt_consolidation   365xx 18.39         0         0  
## 2      credit_card     928xx 24.47         0         1  
## 3 debt_consolidation   100xx 13.40         0         1  
## 4      credit_card     101xx 17.25         0         0  
## 5 debt_consolidation   641xx 37.22         0         2  
## 6      credit_card     100xx  6.80         2         0  
##      mths_since_last_delinq mths_since_last_record open_acc pub_rec revol_bal  
## 1              NA              NA         6      NA      7743  
## 2              55              NA        14      NA     22897  
## 3              NA              NA         4      NA     14716  
## 4              NA              NA        10      NA     32328  
## 5              NA              NA         6      NA     11988  
## 6              10              NA         5      NA      5251  
##      revol_util total_acc collections_12_mths_ex_med mths_since_last_major_derog  
## 1      100.6      14              0              NA  
## 2      102.7      24              0              55  
## 3      103.6       6              0              NA  
## 4      100.4      16              0              NA  
## 5      101.6      28              0              NA  
## 6      101.0      17              0              NA  
##      policy_code tot_coll_amt tot_cur_bal total_rev_hi_lim log_loan_amnt  
## 1          1          0          9186          7700      9.615872  
## 2          1          0         29046         22300      9.919656  
## 3          1          0         14716         14200      9.615872  
## 4          1          0         68486         32200     10.239996  
## 5          1          0         87762         11800      9.680406  
## 6          1          0         21496          5200     10.057324
```

```
##   log_annual_inc delinq_2yrs_cat delinq_recency acc_now_delinq_bin
## 1      10.43415           0           none           0
## 2      10.83960           0           24+           0
## 3      10.77898           0           none           0
## 4      12.42922           0           none           0
## 5      11.69526           0           none           0
## 6      11.81304           2           7-12          0
```

```
cleaned_data <- cleaned_data %>%
  mutate(revol_bal = ifelse(revol_bal > total_rev_hi_lim, NA, revol_bal))

# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

We also check whether the reported utilization ratio (`revol_util`) is consistent with the balance and credit limit. Utilization should satisfy the relationship $\text{revol_util} = (\text{revol_bal} / 100 * \text{total_rev_hi_lim})$. A very large number of rows fail this check, which suggests that inconsistencies are not isolated data-entry errors but instead reflect imperfect source reporting or timing differences. Because utilization is the most stable and meaningful of the three variables, the most defensible correction strategy is to recalculate utilization from the cleaned `revol_bal` and `total_rev_hi_lim` values.

```
# Revolving utilization inconsistent with balance and credit limit
suspicious_revol_util_rows <- cleaned_data %>%
  filter((revol_util / 100) * total_rev_hi_lim < revol_bal)

# Print number of rows
cat("Number of rows:", nrow(suspicious_revol_util_rows), "\n")
```

```
## Number of rows: 365184
```

```
# Inspect first rows only
head(suspicious_revol_util_rows)
```

```
##           id member_id loan_amnt term int_rate emp_title emp_length
## 1  5764263   7196024    18000   36    8.90   Wellcare      1 year
## 2  30585515  33138680     7000   36    6.49   corrections  10+ years
## 3  29854965  32388141    15000   36    6.49   uxo technician 10+ years
## 4   3496375   4378749    11000   36    6.03 United Parcel Service 10+ years
## 5  12696536  14708693    10000   36    7.90      nurse    10+ years
## 6   5618970   7001319    16000   36    7.62      Fdh        2 years
```

```
##   home_ownership annual_inc verification_status
## 1      MORTGAGE      59000      Not Verified
## 2      MORTGAGE      36000      Not Verified
## 3      MORTGAGE      64000      Not Verified
## 4      MORTGAGE      82000      Not Verified
## 5      MORTGAGE      60000      Not Verified
## 6      MORTGAGE      65000      Not Verified
```

```
##
```

```
## 1 Borrower added on 06/17/13 > I'm looking to consolidate my credit card. I've been helping my sis
## 2
```

```
## 3
## 4
## 5
## 6
##
##           purpose zip_code   dti delinq_2yrs inq_last_6mths
## 1 debt_consolidation 300xx 24.55           0           1
## 2      credit_card  570xx 25.30           0           0
## 3      credit_card  325xx  4.28           0           1
## 4 debt_consolidation 970xx 15.89           0           0
## 5   home_improvement 278xx 18.22           0           2
## 6 debt_consolidation 276xx 13.77           0           0
##   mths_since_last_delinq mths_since_last_record open_acc pub_rec revol_bal
## 1                      66                    NA         9     NA    11608
## 2                      NA                    NA         8     NA     5046
## 3                      NA                    NA         5     NA     6360
## 4                      NA                    NA         8     NA     7957
## 5                      NA                    NA        13     NA    10465
## 6                      NA                    NA        14     NA    12355
##   revol_util total_acc collections_12_mths_ex_med mths_since_last_major_derog
## 1         56.9         19                      0                          67
## 2         19.9         28                      0                          NA
## 3         25.0         20                      0                          NA
## 4         56.0         35                      0                          NA
## 5         22.0         30                      0                          NA
## 6         23.7         17                      0                          NA
##   policy_code tot_coll_amt tot_cur_bal total_rev_hi_lim log_loan_amnt
## 1           1           0      172714          20400      9.798183
## 2           1           0       70057          25300      8.853808
## 3           1           0      188331          25400      9.615872
## 4           1           0      215435          14200      9.305741
## 5           1           0      135492          47500      9.210440
## 6           1           0      172741          52100      9.680406
##   log_annual_inc delinq_2yrs_cat delinq_recency acc_now_delinq_bin
## 1      10.98531           0          24+           0
## 2      10.49130           0          none           0
## 3      11.06665           0          none           0
## 4      11.31449           0          none           0
## 5      11.00212           0          none           0
## 6      11.08216           0          none           0
```

```
cleaned_data <- cleaned_data %>%
  mutate(revol_util = ifelse(!is.na(revol_bal) & !is.na(total_rev_hi_lim) & total_rev_hi_lim > 0,
    pmin(100, (revol_bal / total_rev_hi_lim) * 100),
    revol_util))

# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

Finally, because revolving balances cannot exist without at least one open revolving account, we verify that all rows with `revol_bal > 0` also have `open_acc > 0`. No violations of this rule were found in the dataset.


```
suspicious_revol_bal_open_acc_rows <- cleaned_data %>%
  filter(revol_bal > 0 & open_acc == 0)
suspicious_revol_bal_open_acc_rows # 0 rows
```

```
## [1] id                member_id
## [3] loan_amnt          term
## [5] int_rate           emp_title
## [7] emp_length         home_ownership
## [9] annual_inc         verification_status
## [11] desc              purpose
## [13] zip_code           dti
## [15] delinq_2yrs        inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc           pub_rec
## [21] revol_bal          revol_util
## [23] total_acc          collections_12_mths_ex_med
## [25] mths_since_last_major_derog policy_code
## [27] tot_coll_amt       tot_cur_bal
## [29] total_rev_hi_lim   log_loan_amnt
## [31] log_annual_inc     delinq_2yrs_cat
## [33] delinq_recency     acc_now_delinq_bin
## <0 rows> (or 0-length row.names)
```

Together, these steps ensure that the structural relationships between open accounts, total accounts, revolving balances, credit limits, and utilization rates are coherent and compliant with established credit-bureau logic, while avoiding any manipulations that would artificially alter a borrower's reported credit history.

2.4.1.8 Collections and major derogatory events The field `collections_12_mths_ex_med` records the number of collection events in the past 12 months and therefore must be a non-negative integer. Since collection events are discrete events recorded by credit bureaus, the value can never be negative or fractional. We identify any rows that violate these conditions and correct them by assigning NA, as such values cannot occur in a real credit file.

```
invalid_coll <- cleaned_data %>%
  filter(collections_12_mths_ex_med < 0 | collections_12_mths_ex_med % 1 != 0)
invalid_coll # 0 rows
```

```
## [1] id                member_id
## [3] loan_amnt          term
## [5] int_rate           emp_title
## [7] emp_length         home_ownership
## [9] annual_inc         verification_status
## [11] desc              purpose
## [13] zip_code           dti
## [15] delinq_2yrs        inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc           pub_rec
## [21] revol_bal          revol_util
## [23] total_acc          collections_12_mths_ex_med
## [25] mths_since_last_major_derog policy_code
## [27] tot_coll_amt       tot_cur_bal
## [29] total_rev_hi_lim   log_loan_amnt
```

```
## [31] log_annual_inc          delinq_2yrs_cat
## [33] delinq_recency           acc_now_delinq_bin
## <0 rows> (or 0-length row.names)
```

```
# Check the distribution of collections_12_mths_ex_med
summary(cleaned_data$collections_12_mths_ex_med)
```

```
##      Min.   1st Qu.   Median     Mean  3rd Qu.    Max.
## 0.00000  0.00000  0.00000  0.01572  0.00000  20.00000
```

We see that the mean and median are both 0 or almost 0. Only a very small minority is non-zero. This inflation is useless for modelling since there is no signal in it. We therefore can drop this column.

```
cleaned_data <- cleaned_data %>%
  select(-collections_12_mths_ex_med)
# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

The variable `mths_since_last_major_derog` indicates how many months have passed since the last major derogatory event such as bankruptcy, charge-off, foreclosure, or serious delinquency. As with other “months since event” fields, the interpretation is structural: if the borrower has never experienced a major derogatory, the value must be NA; if the borrower has such a history, the value must be zero or positive. Negative months or fractional months are impossible and must be removed. Although delinquency fields do not directly determine whether a major derogatory event occurred, the variable `pub_rec` (public records) does: any non-missing value in `mths_since_last_major_derog` implies at least one corresponding public record. Therefore, the following hard rules apply: if `pub_rec == 0`, then `mths_since_last_major_derog` must be NA; if `mths_since_last_major_derog` is zero or greater, then `pub_rec` must be at least 1. These checks ensure the two fields remain consistent.

```
invalid_major_derog <- cleaned_data %>%
  filter(mths_since_last_major_derog < 0 |
         mths_since_last_major_derog %>= 1 && pub_rec != 0)
invalid_major_derog # 0 rows
```

```
## [1] id                member_id
## [3] loan_amnt          term
## [5] int_rate           emp_title
## [7] emp_length         home_ownership
## [9] annual_inc         verification_status
## [11] desc              purpose
## [13] zip_code           dti
## [15] delinq_2yrs        inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc           pub_rec
## [21] revol_bal          revol_util
## [23] total_acc          mths_since_last_major_derog
## [25] policy_code        tot_coll_amt
## [27] tot_cur_bal        total_rev_hi_lim
## [29] log_loan_amnt      log_annual_inc
## [31] delinq_2yrs_cat    delinq_recency
## [33] acc_now_delinq_bin
## <0 rows> (or 0-length row.names)
```

```
suspicious_major_derog_missing <- cleaned_data %>%
  filter(pub_rec == 0 & !is.na(mths_since_last_major_derog))
suspicious_major_derog_missing # 0 rows
```

```
## [1] id member_id
## [3] loan_amnt term
## [5] int_rate emp_title
## [7] emp_length home_ownership
## [9] annual_inc verification_status
## [11] desc purpose
## [13] zip_code dti
## [15] delinq_2yrs inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc pub_rec
## [21] revol_bal revol_util
## [23] total_acc mths_since_last_major_derog
## [25] policy_code tot_coll_amt
## [27] tot_cur_bal total_rev_hi_lim
## [29] log_loan_amnt log_annual_inc
## [31] delinq_2yrs_cat delinq_recency
## [33] acc_now_delinq_bin
## <0 rows> (or 0-length row.names)
```

```
# Public record present but months since last major derogatory missing
suspicious_major_derog_present <- cleaned_data %>%
  filter(pub_rec >= 1 & is.na(mths_since_last_major_derog))

# Print number of rows
cat("Number of rows:", nrow(suspicious_major_derog_present), "\n") # 80'403 rows
```

```
## Number of rows: 76300
```

```
# Inspect first rows only
head(suspicious_major_derog_present)
```

```
##      id member_id loan_amnt term int_rate emp_title
## 1 23866532 26238998    8400  36    7.69 Administrative Assistant
## 2 28392619 30915748    5000  36    6.03 Finance Director
## 3 46773707 49921480    6000  36    6.92 Operations Representative
## 4 8634916 10407031    7200  36    8.90 Peer Support Specialist
## 5 11965231 13957383   14400  36    6.03 Senior Systems Administrator
## 6 60635849 64656593   20000  36    5.32 Attorney
##      emp_length home_ownership annual_inc verification_status
## 1 10+ years MORTGAGE 50000 Not Verified
## 2 9 years MORTGAGE 115000 Not Verified
## 3 9 years MORTGAGE 42000 Not Verified
## 4 6 years MORTGAGE 35000 Not Verified
## 5 1 year MORTGAGE 69000 Not Verified
## 6 < 1 year MORTGAGE 155000 Not Verified
##
## 1
## 2
```

```

## 3
## 4 Borrower added on 10/31/13 > I want to consolidate my credit cards at a lower interest rate. It v
## 5
## 6
##      purpose zip_code   dti delinq_2yrs inq_last_6mths
## 1   credit_card  461xx 11.57           0           1
## 2 debt_consolidation 030xx  8.12           0           0
## 3   home_improvement 761xx  9.86           0           0
## 4   credit_card  537xx 26.37           0           0
## 5   credit_card  370xx 21.88           0           0
## 6   credit_card  079xx 11.09           0           0
##      mths_since_last_delinq mths_since_last_record open_acc pub_rec revol_bal
## 1                        NA                95         4         1       4429
## 2                        NA                57        13         1       5407
## 3                        75                76         4         1       1509
## 4                        NA               115         9         1       7443
## 5                        NA                86        12         1       9331
## 6                        NA                62        13         1      12010
##      revol_util total_acc mths_since_last_major_derog policy_code tot_coll_amt
## 1   59.85135        16                NA                1           0
## 2   23.92478        43                NA                1           0
## 3   58.03846        21                NA                1           0
## 4   64.16379        13                NA                1           0
## 5   61.79470        45                NA                1           0
## 6   23.68836        27                NA                1           0
##      tot_cur_bal total_rev_hi_lim log_loan_amnt log_annual_inc delinq_2yrs_cat
## 1    197574        7400        9.036106        10.81980           0
## 2    312223       22600        8.517393        11.65270           0
## 3    127907        2600        8.699681        10.64545           0
## 4     43264       11600        8.881975        10.46313           0
## 5    353023       15100        9.575053        11.14188           0
## 6    474425       50700        9.903538        11.95119           0
##      delinq_recency acc_now_delinq_bin
## 1         none           0
## 2         none           0
## 3         24+           0
## 4         none           0
## 5         none           0
## 6         none           0

```

If `pub_rec == 0` but `mths_since_last_major_derog` is present, the timing field is incorrect and is set to NA. If `pub_rec == 1` but `mths_since_last_major_derog` is missing, the timing field is restored by assigning NA as representing “unknown months since event,” because the presence of a public record confirms that a major derogatory exists but its recency cannot be reconstructed precisely.

```

# count how often pub_rec == 0 and mths_since_last_major_derog is not NA
suspicious_major_derog_missing <- cleaned_data %>%
  filter(pub_rec == 0 & !is.na(mths_since_last_major_derog))
suspicious_major_derog_missing # 0 rows

```

```

## [1] id          member_id
## [3] loan_amnt   term
## [5] int_rate    emp_title

```

```
## [7] emp_length          home_ownership
## [9] annual_inc          verification_status
## [11] desc                purpose
## [13] zip_code            dti
## [15] delinq_2yrs         inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc            pub_rec
## [21] revol_bal           revol_util
## [23] total_acc           mths_since_last_major_derog
## [25] policy_code         tot_coll_amt
## [27] tot_cur_bal         total_rev_hi_lim
## [29] log_loan_amnt       log_annual_inc
## [31] delinq_2yrs_cat     delinq_recency
## [33] acc_now_delinq_bin
## <0 rows> (or 0-length row.names)
```

```
cleaned_data <- cleaned_data %>%
  mutate(mths_since_last_major_derog =
    ifelse(pub_rec == 0, NA, mths_since_last_major_derog))
```

Finally, because `mths_since_last_major_derog` is not a linearly meaningful numeric variable and NA carries semantic information (“no major derogatory on file”), the variable is transformed into a categorical factor with recency buckets. This follows standard credit-modeling practice, where the recency of serious derogatory events is highly predictive but does not behave like a continuous metric. Very recent major derogatories (0–6 months) are associated with sharply elevated risk and therefore require finer granularity, whereas the predictive strength of a derogatory decreases gradually as it ages, with distinct risk breaks around 12, 24, 48, and 84 months. These cut points reflect how bureaus and scorecard models treat derogatory events: an event occurring this month (0) is the most severe signal; 1–6 and 7–12 months represent the critical recent-risk window; 13–24 and 25–48 months reflect medium-term aging; 49–84 months captures late-aging derogatories that still influence risk; and values above 84 months are typically close to falling off the credit report and contribute only minimal predictive value. This structure aligns with FICO guidelines, common bank scorecard binning strategies, and the natural decay of credit-risk signals over time, making a categorical transformation more appropriate than preserving the raw numeric values.

```
cleaned_data <- cleaned_data %>%
  mutate(major_derog_recency = case_when(
    is.na(mths_since_last_major_derog) ~ "none",
    mths_since_last_major_derog == 0 ~ "0",
    mths_since_last_major_derog <= 6 ~ "1-6",
    mths_since_last_major_derog <= 12 ~ "7-12",
    mths_since_last_major_derog <= 24 ~ "13-24",
    mths_since_last_major_derog <= 48 ~ "25-48",
    mths_since_last_major_derog <= 84 ~ "49-84",
    TRUE ~ ">84"
  )) %>%
  mutate(major_derog_recency = factor(major_derog_recency,
    levels = c("49-84", "25-48", "13-24", "7-12", "1-6", "0", "none"),
    ordered = TRUE))
```

The variables `tot_coll_amt` and `tot_cur_bal` also require structural validation. The field `tot_coll_amt` represents the total historical dollar amount ever assigned to collections. As a monetary field recorded in whole dollars, it must be zero or positive; negative values are impossible and are corrected by setting the field to NA.

```
invalid_tot_coll_amt <- cleaned_data %>%
  filter(tot_coll_amt < 0)
invalid_tot_coll_amt # 0 rows
```

```
## [1] id                member_id
## [3] loan_amnt             term
## [5] int_rate              emp_title
## [7] emp_length            home_ownership
## [9] annual_inc            verification_status
## [11] desc                  purpose
## [13] zip_code              dti
## [15] delinq_2yrs           inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc              pub_rec
## [21] revol_bal             revol_util
## [23] total_acc             mths_since_last_major_derog
## [25] policy_code           tot_coll_amt
## [27] tot_cur_bal           total_rev_hi_lim
## [29] log_loan_amnt         log_annual_inc
## [31] delinq_2yrs_cat       delinq_recency
## [33] acc_now_delinq_bin    major_derog_recency
## <0 rows> (or 0-length row.names)
```

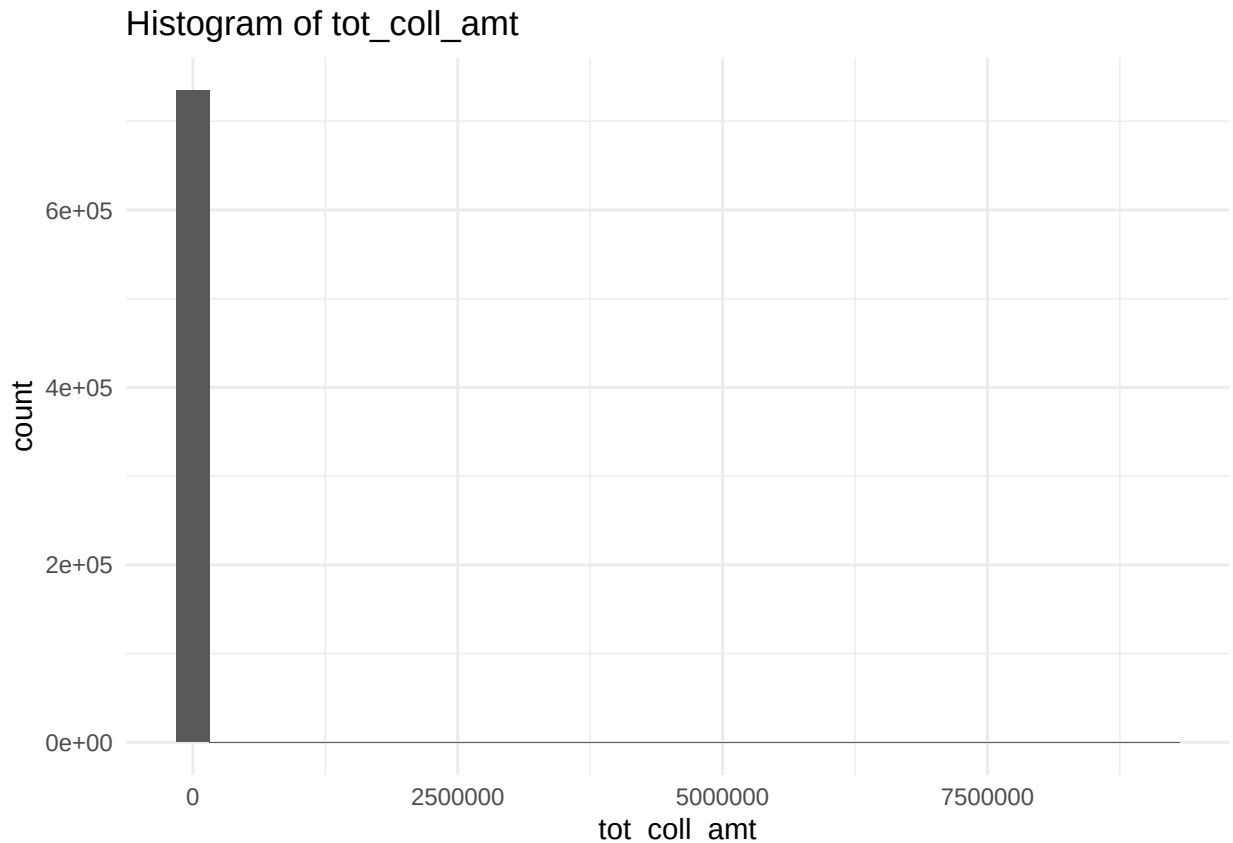
```
# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

```
summary(cleaned_data$tot_coll_amt)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.    NA's
##      0.0      0.0      0.0    228.3     0.0 9152545.0     20
```

```
# show histogram
ggplot(cleaned_data, aes(x = tot_coll_amt)) +
  geom_histogram(bins = 30) +
  ggtitle("Histogram of tot_coll_amt") +
  theme_minimal()
```



The variable is heavily zero-inflated. The vast majority of observations take the value 0, indicating no prior collections. As a result, the variable exhibits near-zero variance, with almost no meaningful differentiation across most borrowers. Predictors with extremely low variance do not contribute useful information to a linear regression model and typically lead to unstable or insignificant coefficient estimates. Second, the remaining non-zero observations consist of a very small number of extreme outliers, with collection amounts reaching several million dollars. These values are several orders of magnitude larger than the rest of the data and exert disproportionate leverage on a linear regression fit. Even after applying a log-transformation, the underlying zero-inflation remains and the outliers continue to dominate the distribution. This violates key regression assumptions (linearity, constant variance, influence balance) and introduces risk of distorted parameter estimates. Together, the combination of zero-inflation, near-zero variance, and extreme outliers implies that `tot_coll_amt` is not suitable as a continuous predictor for linear regression. The variable does not provide stable or meaningful explanatory power and would likely degrade model performance. For this reason, the variable was removed from the modelling dataset.

```
cleaned_data <- cleaned_data %>%
  select(-tot_coll_amt)
# amount of rows after mutation
print(nrow(cleaned_data))
```

```
## [1] 734825
```

Similarly, `tot_cur_bal` reflects the borrower's total current balance across all active credit lines and must also be zero or positive. If this balance is negative, the value is replaced by NA. In addition, if a positive total balance is reported, the borrower must have at least one open account. Because `open_acc` was previously cleaned, we verify that no rows show a positive balance with zero open accounts, which would be structurally impossible in credit reporting.

```
invalid_tot_cur_bal <- cleaned_data %>%
  filter(tot_cur_bal < 0)
invalid_tot_cur_bal # 0 rows
```

```
## [1] id                member_id
## [3] loan_amnt           term
## [5] int_rate            emp_title
## [7] emp_length          home_ownership
## [9] annual_inc          verification_status
## [11] desc                purpose
## [13] zip_code            dti
## [15] delinq_2yrs         inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc            pub_rec
## [21] revol_bal           revol_util
## [23] total_acc           mths_since_last_major_derog
## [25] policy_code         tot_cur_bal
## [27] total_rev_hi_lim    log_loan_amnt
## [29] log_annual_inc      delinq_2yrs_cat
## [31] delinq_recency      acc_now_delinq_bin
## [33] major_derog_recency
## <0 rows> (or 0-length row.names)
```

```
contradiction_tot_cur_bal <- cleaned_data %>%
  filter(tot_cur_bal > 0 & open_acc == 0)
contradiction_tot_cur_bal # 0 rows
```

```
## [1] id                member_id
## [3] loan_amnt           term
## [5] int_rate            emp_title
## [7] emp_length          home_ownership
## [9] annual_inc          verification_status
## [11] desc                purpose
## [13] zip_code            dti
## [15] delinq_2yrs         inq_last_6mths
## [17] mths_since_last_delinq mths_since_last_record
## [19] open_acc            pub_rec
## [21] revol_bal           revol_util
## [23] total_acc           mths_since_last_major_derog
## [25] policy_code         tot_cur_bal
## [27] total_rev_hi_lim    log_loan_amnt
## [29] log_annual_inc      delinq_2yrs_cat
## [31] delinq_recency      acc_now_delinq_bin
## [33] major_derog_recency
## <0 rows> (or 0-length row.names)
```

2.4.2 Final cleanup

Furthermore there are other columns to drop, which aren't useful for modelling or have too many missing values. Policy code is always 1 and has therefore no value for modelling. We drop `revol_util` for now, since we should check the recalculation after imputation. If we decide to include it in modelling, we should drop at least one of the two other variables `revol_bal` or `total_rev_hi_lim` to avoid multicollinearity. We drop

also `mths_since_last_delinq`, `mths_since_last_major_derog`, `delinq_2yrs` and `pub_rec`, since we created categorical variables from them before.

The variable `zip_code` is removed during preprocessing even though it is provided only in truncated form (e.g., “365xx”). While this representation partially anonymizes the borrower’s location, it still constitutes a high-cardinality geographic identifier that captures localized effects rather than borrower-specific risk characteristics. The ZIP codes in this form offer limited economic interpretability while increasing the model complexity. Excluding this feature improves robustness and reduces the risk.

The variables `id` and `member_id` uniquely identify loans and borrowers but contain no information about borrower risk or pricing. Including identifiers in regression models introduces pure noise and can lead to spurious patterns or overfitting. After confirming that no duplicated observations exist, both variables were removed to ensure that all remaining features carry potential predictive meaning.

```
cleaned_data <- cleaned_data %>%
  select(-c(id, member_id, desc, zip_code, emp_title, policy_code, revol_util, mths_since_last_delinq, mths_since_last_major_derog, delinq_2yrs, pub_rec))
```

2.5 Summary

We end up with following dataframe for imputation and modeling.

```
# amount of columns and rows
print(dim(cleaned_data))
```

```
## [1] 734825    19
```

```
# summary of cleaned_data
summary(cleaned_data)
```

```
##      term      int_rate      emp_length      home_ownership
##  Min.   :36.00  Min.    : 5.32  10+ years:246949  RENT      :290284
##  1st Qu.:36.00  1st Qu.: 9.99   2 years : 64336  MORTGAGE:370672
##  Median :36.00  Median :12.99   3 years : 57220  OWN       : 73869
##  Mean   :43.34  Mean   :13.30  < 1 year : 56762
##  3rd Qu.:60.00  3rd Qu.:16.29   1 year  : 46520
##  Max.   :60.00  Max.   :28.99  (Other)  :224586
##                                     NA's      : 38452
##      verification_status      purpose      dti
##  Not Verified   :214196  debt_consolidation:440070  Min.    : 0.01
##  Source Verified:280506  credit_card       :175780  1st Qu.:12.21
##  Verified       :240123  home_improvement  : 42148  Median :17.98
##                                     other           : 33083  Mean   :18.47
##                                     major_purchase  : 12770  3rd Qu.:24.38
##                                     small_business  :  6720  Max.   :39.99
##                                     (Other)         : 24254  NA's   :1042
##  inq_last_6mths  open_acc  revol_bal  total_acc
##  Min.    :0.0000  Min.    : 0.0  Min.    :    0  Min.    :  2.00
##  1st Qu.:0.0000  1st Qu.: 8.0  1st Qu.:  6587  1st Qu.: 17.00
##  Median :0.0000  Median :11.0  Median : 12031  Median : 24.00
##  Mean    :0.6709  Mean    :11.7  Mean    : 17124  Mean    : 25.51
##  3rd Qu.:1.0000  3rd Qu.:14.0  3rd Qu.: 21079  3rd Qu.: 32.00
##  Max.    :8.0000  Max.    :90.0  Max.    :1746716  Max.    :169.00
##                                     NA's     :3332
```

```
## tot_cur_bal      total_rev_hi_lim  log_loan_amnt    log_annual_inc
## Min.      :      0    Min.      :      0    Min.      : 6.909    Min.      : 8.007
## 1st Qu.: 29857    1st Qu.: 13900    1st Qu.: 9.036    1st Qu.:10.736
## Median : 80621    Median : 23700    Median : 9.511    Median :11.082
## Mean   : 139495    Mean   : 32057    Mean   : 9.430    Mean   :11.078
## 3rd Qu.: 208198    3rd Qu.: 39800    3rd Qu.: 9.904    3rd Qu.:11.408
## Max.    :8000078    Max.    :2013133    Max.    :10.463    Max.    :13.122
## NA's     :16      NA's      :5
## delinq_2yrs_cat delinq_recency acc_now_delinq_bin major_derog_recency
## 3+: 20662      0      : 3711    0:731102      none      :691808
## 2 : 29020      1-6    : 26047    1: 3723      49-84    : 22301
## 1 : 96769      7-12   : 42863      25-48    : 12725
## 0 :588374      13-24   : 76336      13-24    : 4240
##                24+    :216765      7-12     : 1539
##                none   :369103      (Other):   951
##                NA's   : 1261
```

```
# percentage of NA to be imputed
na_percentage <- sapply(cleaned_data, function(x) sum(is.na(x)) / nrow(cleaned_data) * 100)
print(na_percentage)
```

```
##                term                int_rate                emp_length                home_ownership
##      0.0000000000      0.0000000000      5.2328105331      0.0000000000
## verification_status                purpose                dti                inq_last_6mths
##      0.0000000000      0.0000000000      0.1418024700      0.0000000000
##                open_acc                revol_bal                total_acc                tot_cur_bal
##      0.0000000000      0.4534412955      0.0000000000      0.0021773892
## total_rev_hi_lim      log_loan_amnt      log_annual_inc      delinq_2yrs_cat
##      0.0006804341      0.0000000000      0.0000000000      0.0000000000
## delinq_recency acc_now_delinq_bin major_derog_recency
##      0.0000000000      0.0000000000      0.1716054843
```

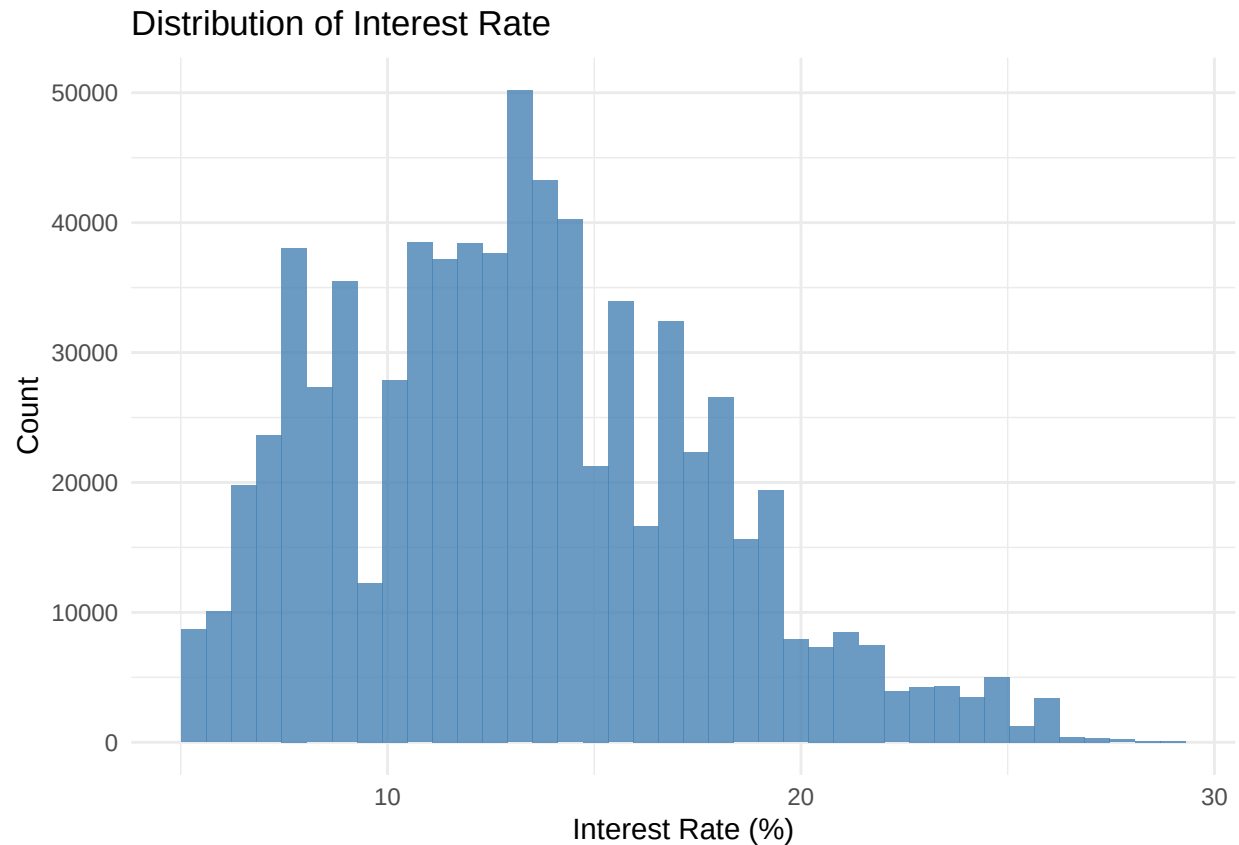
2.5.1 Check the distribution of the target

To ensure that the target variable `int_rate` is reliable for modeling, the distribution is first examined for impossible values, unexpected patterns, or structural inconsistencies. Interest rates in LendingClub data are always expressed as percentages, typically between 5% and 30%, depending on loan grade and risk. They must therefore be non-negative, non-zero, and should not exceed realistic consumer-credit bounds. We begin by summarizing and plotting the distribution.

```
summary(cleaned_data$int_rate)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      5.32   9.99   12.99   13.30   16.29   28.99
```

```
ggplot(cleaned_data, aes(x = int_rate)) +
  geom_histogram(bins = 40, fill = "steelblue", alpha = 0.8) +
  labs(title = "Distribution of Interest Rate",
       x = "Interest Rate (%)",
       y = "Count") +
  theme_minimal()
```



We then check for values that fall outside the plausible range of consumer loan interest rates. Negative interest rates, values equal to zero, or extremely high rates (e.g., above 40%) are structurally impossible in this lending context and indicate a data-entry or import error.

```
invalid_int_rate <- cleaned_data %>%
  filter(int_rate < 0 | int_rate == 0 | int_rate > 40)

invalid_int_rate
```

```
## [1] term                int_rate              emp_length
## [4] home_ownership       verification_status   purpose
## [7] dti                  inq_last_6mths        open_acc
## [10] revol_bal            total_acc             tot_cur_bal
## [13] total_rev_hi_lim     log_loan_amnt         log_annual_inc
## [16] delinq_2yrs_cat      delinq_recency        acc_now_delinq_bin
## [19] major_derog_recency
## <0 rows> (or 0-length row.names)
```

```
print(nrow(invalid_int_rate))
```

```
## [1] 0
```

Interest rates should also be numeric and typically recorded with at most two decimal places. If values contain long decimal expansions or parsing issues, they would appear in the summary above and require correction, but none were detected in this dataset. Although these are not hard inconsistencies, we also examine whether

extremely low or high interest rates appear in combinations that contradict typical underwriting outcomes. Very low interest rates (for example, below 5%) or very high rates (above 30%) are possible but unusual. If they occur together with very risky or very strong credit indicators, this can signal stale data or mismatches between borrower characteristics and the risk-based pricing system. We therefore inspect these rows in context using variables available in the dataset, such as recent delinquencies, revolving utilization, number of recent inquiries, annual income, and open account counts.

```
# Soft plausibility check: extremely low or high interest rates
cleaned_data %>%
  filter(int_rate < 5 | int_rate > 30) %>%
  select(int_rate, inq_last_6mths) %>%
  head(20)
```

```
## [1] int_rate      inq_last_6mths
## <0 rows> (or 0-length row.names)
```

```
print(nrow(cleaned_data))
```

```
## [1] 734825
```

The structural checks for the target variable yielded no impossible or inconsistent values, and the resulting distribution of `int_rate` confirms that the data align with expected lending patterns. The histogram shows a smooth, unimodal distribution centered between roughly 10% and 15%, with values ranging from approximately 6% to just under 30%, which is fully consistent with typical unsecured consumer loans. No negative or zero interest rates, no implausibly high values, and no irregular spikes indicative of parsing errors were detected. Together, these results indicate that the interest-rate variable is clean, internally coherent, and suitable to be used as the modeling target without requiring further preprocessing.

```
missing_summary <- data.frame(
  column = names(cleaned_data),
  missing_count = sapply(cleaned_data, function(x) sum(is.na(x))),
  missing_pct = sapply(cleaned_data, function(x) mean(is.na(x)) * 100)
) %>%
  arrange(desc(missing_pct))

knitr::kable(
  head(missing_summary, 10),
  digits = 4,
  caption = "Top 10 features by missing-value percentage and count (after preprocessing)"
)
```

Table 3: Top 10 features by missing-value percentage and count (after preprocessing)

	column	missing_count	missing_pct
emp_length	emp_length	38452	5.2328
revol_bal	revol_bal	3332	0.4534
major_derog_recency	major_derog_recency	1261	0.1716
dti	dti	1042	0.1418
tot_cur_bal	tot_cur_bal	16	0.0022
total_rev_hi_lim	total_rev_hi_lim	5	0.0007

	column	missing_count	missing_pct
term	term	0	0.0000
int_rate	int_rate	0	0.0000
home_ownership	home_ownership	0	0.0000
verification_status	verification_status	0	0.0000

The remaining missing values after preprocessing are concentrated in a small number of features. Employment length (`emp_length`) exhibits the highest level of missingness, affecting approximately 5.23% of observations, while all other features show substantially lower missing-value rates below 0.5%. Several key modeling variables, including loan amount, loan term, interest rate, and home ownership, contain no missing values. Overall, the extent of missing data is limited and well localized, allowing imputation strategies to be applied during model training without substantial loss of information.

```
summary_dims <- data.frame(
  Stage = c("Raw data", "Cleaned data"),
  Rows  = c(nrow(original), nrow(cleaned_data)),
  Cols  = c(ncol(original), ncol(cleaned_data))
)

knitr::kable(summary_dims, caption = "Dataset size before and after preprocessing")
```

Table 4: Dataset size before and after preprocessing

Stage	Rows	Cols
Raw data	798641	72
Cleaned data	734825	19

Preprocessing reduces the dataset from 798,641 observations with 52 features to 734,825 observations with 20 features. The reduction in rows is primarily driven by the removal of joint loan applications and observations with insufficient or ambiguous categorical information, while the reduction in features reflects the exclusion of non-informative identifiers, semantically unavailable variables, and redundant or derived features.

```
write.csv(cleaned_data,
  file = "cleaned_data.csv",
  row.names = FALSE)
```

3 Training

3.1 Task and metric

The objective of this project is to predict the interest rate (`int_rate`) assigned to a loan application using borrower and loan characteristics from the provided dataset. Interest rates represent a continuous outcome determined by LendingClub’s internal risk assessment and pricing strategy and therefore constitute a supervised regression problem.

Accurate prediction of interest rates is relevant because it reflects the underlying assessment of borrower credit risk and loan characteristics. From a modeling perspective, this task poses several challenges, including nonlinear relationships between predictors and the target, interactions among credit-related variables, and heterogeneous variance across risk profiles.

Interest rate prediction is approached as a supervised regression task. To ensure a fair and reproducible comparison between different modeling approaches, all models are trained and evaluated using the same data splits and evaluation metrics. Any transformation steps that require learning parameters from the data such as imputation is performed exclusively on the training set to prevent information leakage.

Model performance is evaluated using Root Mean Squared Error (RMSE) as the primary metric and Mean Absolute Error (MAE) as a complementary metric. RMSE penalizes large prediction errors more strongly and is therefore well suited for assessing overall predictive accuracy, while MAE provides an intuitive interpretation of the average absolute deviation between predicted and observed interest rates. These metrics are computed on a validation set and used for model comparison and selection.

3.2 Loading The preprocessed data

The modeling phase begins by loading the cleaned dataset produced in the preprocessing stage. This dataset contains only application-time-available features and has already undergone extensive quality checks, feature reduction, and logical consistency validation. The file is read from disk to ensure a clean separation between preprocessing and training workflows.

```
data <- read_csv("cleaned_data.csv")
```

3.2.1 Order of levels

Because CSV files do not preserve factor ordering, all ordered categorical variables must be explicitly re-defined after loading the preprocessed dataset. Correct ordering is essential because several variables (e.g., employment length, verification status, delinquency severity) encode monotonic risk relationships. Restoring their ordinal structure ensures that downstream preprocessing steps and models correctly interpret relative category severity.

```
categorical_cols <- names(data)[
  sapply(data, function(x) is.factor(x) || is.character(x))
]
```

```
categorical_cols
```

```
## [1] "emp_length"      "home_ownership"  "verification_status"
## [4] "purpose"         "delinq_2yrs_cat" "delinq_recency"
## [7] "major_derog_recency"
```

```
num_categories <- sapply(
  data[categorical_cols],
  function(x) length(unique(na.omit(x)))
)
```

```
num_categories
```

```
##      emp_length  home_ownership verification_status      purpose
##           11           3           3           13
## delinq_2yrs_cat delinq_recency major_derog_recency
##           4           6           7
```

```
# introduce order for emp_length
data$emp_length <- factor(
  data$emp_length,
  levels = c(
    "< 1 year",
    "1 year",
    "2 years",
    "3 years",
    "4 years",
    "5 years",
    "6 years",
    "7 years",
    "8 years",
    "9 years",
    "10+ years"
  ),
  ordered = TRUE
)
```

```
levels(data$emp_length)
```

3.2.1.1 emp_length

```
## [1] "< 1 year" "1 year" "2 years" "3 years" "4 years" "5 years"
## [7] "6 years" "7 years" "8 years" "9 years" "10+ years"
```

```
is.ordered(data$emp_length)
```

```
## [1] TRUE
```

```
data$verification_status <- factor(
  data$verification_status,
  levels = c(
    "Not Verified",
    "Source Verified",
    "Verified"
  ),
  ordered = TRUE
)
```

```
levels(data$verification_status)
```

3.2.1.2 verification_status

```
## [1] "Not Verified"      "Source Verified" "Verified"
```

```
is.ordered(data$verification_status)
```

```
## [1] TRUE
```

```
data$home_ownership <- factor(  
  data$home_ownership,  
  levels = c("RENT", "MORTGAGE", "OWN"),  
  ordered = TRUE  
)
```

```
levels(data$home_ownership)
```

3.2.1.3 home_ownership

```
## [1] "RENT"      "MORTGAGE" "OWN"
```

```
is.ordered(data$home_ownership)
```

```
## [1] TRUE
```

```
data$delinq_2yrs_cat <- factor(  
  data$delinq_2yrs_cat,  
  levels = c("0", "1", "2", "3+"),  
  ordered = TRUE  
)
```

```
levels(data$delinq_2yrs_cat)
```

3.2.1.4 delinq_2yrs_cat

```
## [1] "0"  "1"  "2"  "3+"
```

```
is.ordered(data$delinq_2yrs_cat)
```

```
## [1] TRUE
```



```
data$delinq_recency <- factor(
  data$delinq_recency,
  levels = c(
    "0",
    "1-6",
    "7-12",
    "13-24",
    "24+",
    "none"
  ),
  ordered = TRUE
)
```

```
table(data$delinq_recency)
```

3.2.1.5 delinq_recency

```
##
##      0    1-6    7-12   13-24    24+    none
##  3711  26047  42863  76336  216765  369103
```

```
is.ordered(data$delinq_recency)
```

```
## [1] TRUE
```

```
data$major_derog_recency <- factor(
  data$major_derog_recency,
  levels = c(
    "0",
    "1-6",
    "7-12",
    "13-24",
    "25-48",
    "49-84",
    "none"
  ),
  ordered = TRUE
)
```

```
table(data$major_derog_recency)
```

3.2.1.6 major_derog_recency

```
##
##      0    1-6    7-12   13-24   25-48   49-84   none
##   34   917   1539   4240   12725   22301  691808
```

```
is.ordered(data$major_derog_recency)
```

```
## [1] TRUE
```

```
data_ordered <- data
```

3.3 Imputation of missing values

Although extensive preprocessing was performed to resolve logical inconsistencies and remove unusable variables, some predictors still contain missing values. These remaining missing values represent either genuinely unknown information (e.g., employment length not reported) or structurally meaningful absence like no delinquency history. Imputation is therefore handled during model training rather than during preprocessing to avoid information leakage and to allow imputation parameters to be learned exclusively from the training data.

```
missing_summary <- data.frame(
  column = names(data),
  missing_count = sapply(data, function(x) sum(is.na(x))),
  missing_pct = sapply(data, function(x) mean(is.na(x)) * 100)
) %>%
  arrange(desc(missing_pct))

knitr::kable(
  head(missing_summary, 10),
  digits = 4,
  caption = "Top 10 features by missing-value percentage and count (after preprocessing)"
)
```

Table 5: Top 10 features by missing-value percentage and count (after preprocessing)

	column	missing_count	missing_pct
emp_length	emp_length	38452	5.2328
revol_bal	revol_bal	3332	0.4534
major_derog_recency	major_derog_recency	1261	0.1716
dti	dti	1042	0.1418
tot_cur_bal	tot_cur_bal	16	0.0022
total_rev_hi_lim	total_rev_hi_lim	5	0.0007
term	term	0	0.0000
int_rate	int_rate	0	0.0000
home_ownership	home_ownership	0	0.0000
verification_status	verification_status	0	0.0000

The dataset is split into three mutually exclusive subsets: training, validation, and test. The training set is used to estimate model parameters and preprocessing statistics, the validation set is used for model comparison and tuning, and the test set is held out for final, unbiased evaluation.

The split is performed before any imputation or transformation that learns from the data distribution. This ensures that no information from the validation or test sets leaks into the training process. A fixed random seed is used to guarantee reproducibility.

```

library(tidymodels)
library(xgboost)

set.seed(123)

n <- nrow(data_ordered)
idx_all <- sample.int(n)

n_train <- floor(0.70 * n)
n_val <- floor(0.15 * n)

train_idx <- idx_all[1:n_train]
val_idx <- idx_all[(n_train + 1):(n_train + n_val)]
test_idx <- idx_all[(n_train + n_val + 1):n]

train <- data_ordered[train_idx, ]
val <- data_ordered[val_idx, ]
test <- data_ordered[test_idx, ]

val_split <- rsample::make_splits(
  list(analysis = train_idx, assessment = val_idx),
  data = data_ordered
)

val_rs <- rsample::manual_rset(
  splits = list(val_split),
  ids = "train_val"
)

```

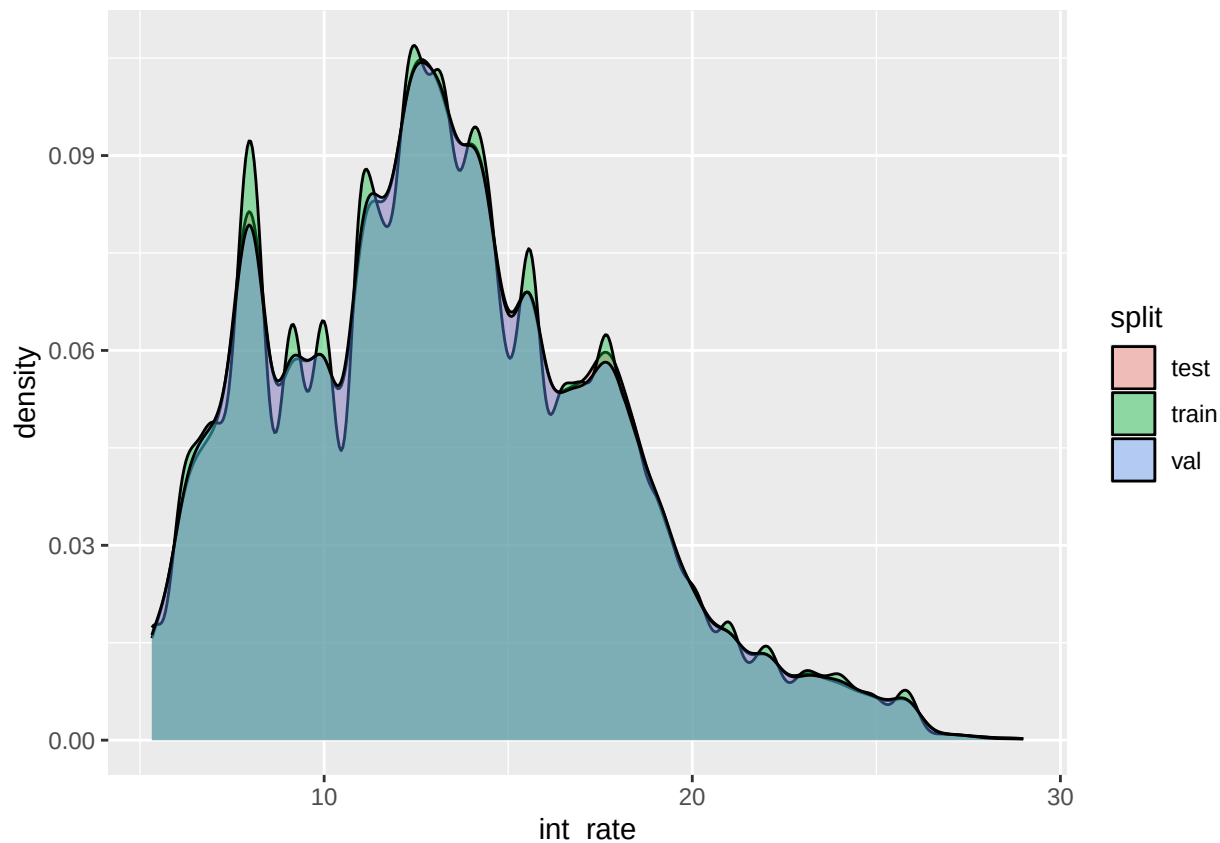
The dataset was split into three mutually exclusive subsets: - Training set: used to learn model parameters and preprocessing statistics - Validation set: used for model selection and tuning - Test set: held out for final, unbiased evaluation

Splitting was performed before any imputation to prevent leakage.

```

ggplot(bind_rows(
  train %>% mutate(split="train"),
  val %>% mutate(split="val"),
  test %>% mutate(split="test")
), aes(int_rate, fill=split)) +
  geom_density(alpha=0.4)

```



To verify that the random split preserves the target distribution across subsets, the density of the interest rate is compared between the training, validation, and test sets. Similar distributions across splits indicate that no systematic sampling bias was introduced during splitting.

3.3.1 Training-time preprocessing pipeline

```
# 2) Recipe (ALL preprocessing here; trained on analysis only)
# -----
rec <- recipe(int_rate ~ ., data = train) %>%
  # --- special handling for major_derog_recency (do this before generic nominal steps) ---
  step_mutate(
    major_derog_recency = if_else(
      is.na(major_derog_recency),
      "none",
      as.character(major_derog_recency)
    )
  ) %>%
  step_string2factor(major_derog_recency) %>% # ensures it's a factor again
  step_novel(major_derog_recency) %>%
  step_unknown(major_derog_recency, new_level = "Unknown") %>%

  # --- emp_length: explicit Missing level ---
  step_unknown(emp_length, new_level = "Missing") %>%
```

```
# --- generic protection for *other* categoricals ---
step_novel(all_nominal_predictors(), -major_derog_recency) %>%
step_unknown(all_nominal_predictors(), -major_derog_recency, new_level = "Unknown") %>%

# --- numeric imputation learned on analysis only ---
step_impute_median(all_numeric_predictors()) %>%

# --- encoding ---
step_integer(all_ordered_predictors(), zero_based = FALSE) %>%
step_dummy(all_nominal_predictors(), one_hot = FALSE) %>%
step_zv(all_predictors())
```

All preprocessing steps required for modeling are implemented using a recipes pipeline from the tidymodels framework. This approach ensures that preprocessing transformations are estimated exclusively on the training data and then applied consistently to validation and test sets.

The recipe includes the following key components:

- **Explicit handling of structurally meaningful missing values**
Certain ordinal variables (e.g., `major_derog_recency`, `emp_length`) use missingness to encode meaningful absence. These variables are given explicit “none” or “Missing” categories rather than being imputed numerically.
- **Robust handling of categorical variables**
Novel and previously unseen factor levels are handled using `step_novel()` and `step_unknown()` to ensure that validation and test data can be processed safely without errors.
- **Numeric imputation**
All numeric predictors are imputed using the median, with imputation parameters learned exclusively from the training set. Median imputation is robust to skewed distributions and extreme values, which are common in credit-related data.
- **Encoding for modeling**
Ordered factors are encoded as integer scores that preserve ordering, while nominal categorical variables are one-hot encoded. Zero-variance predictors are removed automatically to prevent numerical issues during training.

This recipe-based design provides a clean separation between data preprocessing and model fitting, prevents information leakage, and ensures reproducibility across modeling experiments.

3.4 Linear model

```
rec_prep <- prep(rec, training = train)
train_baked <- bake(rec_prep, new_data = train)
```

3.4.1 Assumption Checking

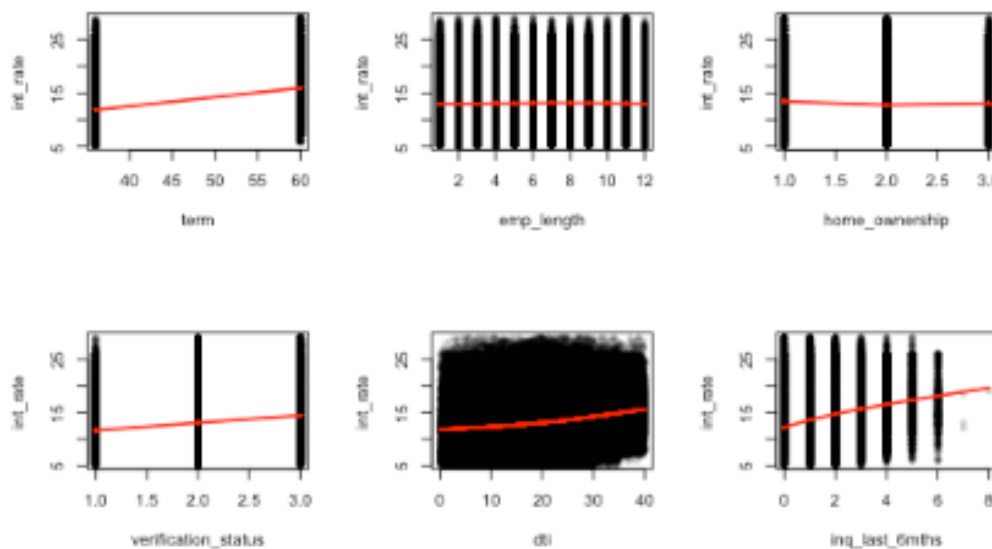
Before fitting a linear regression model, the validity of the underlying modeling assumptions must be assessed. In particular, linear regression assumes that the relationship between each predictor and the response variable is approximately linear and that the effects of the predictors are additive. Violations of these assumptions can lead to biased estimates and reduced predictive performance.

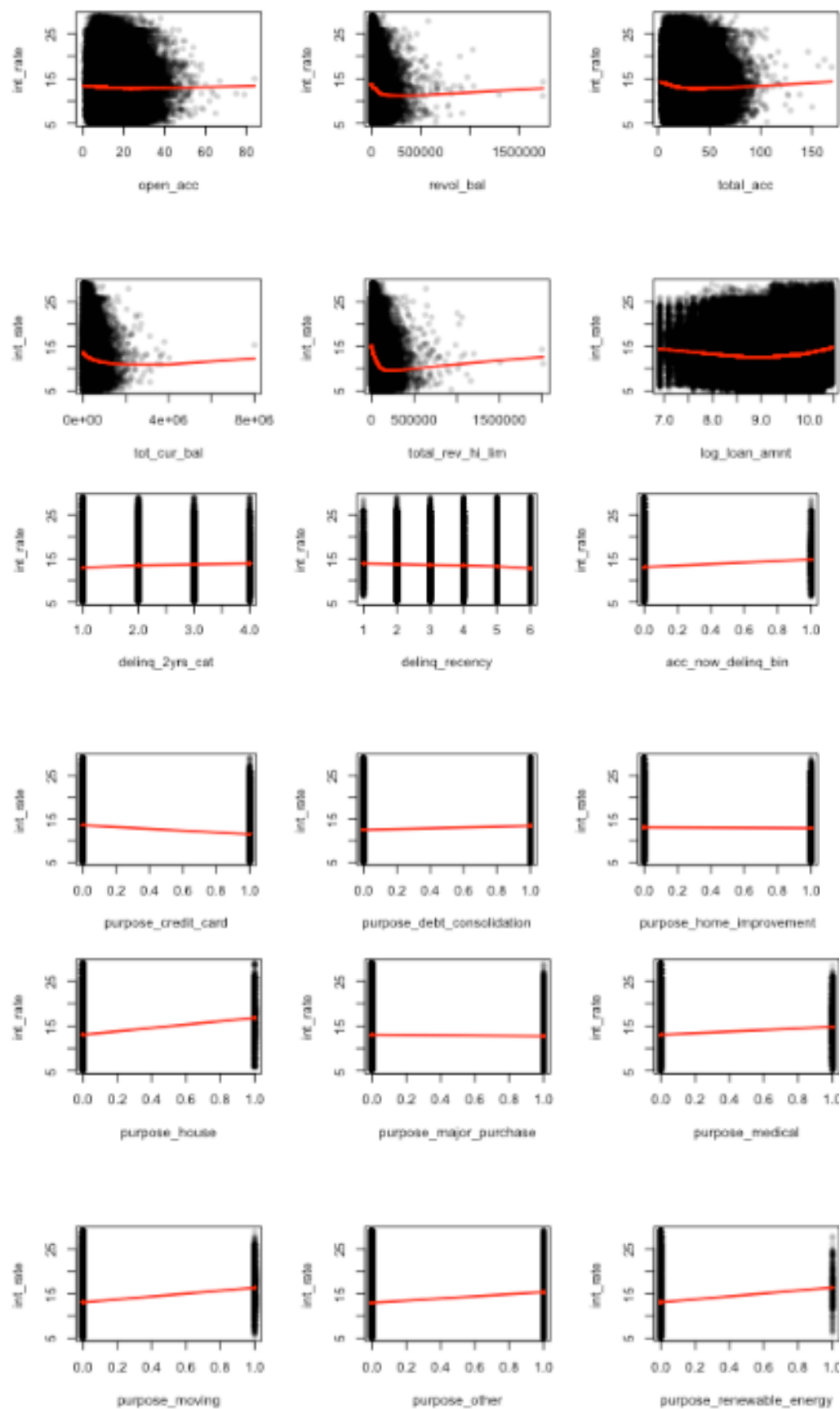
3.4.1.1 Linearity along the axes To assess the linearity assumption, scatterplots of each predictor against the target variable (`int_rate`) were inspected using the preprocessed training data. For each predictor, a locally weighted smoothing curve (LOWESS) was overlaid to highlight potential deviations from linearity. This visualization facilitates the identification of nonlinear trends, curvature, and heteroscedastic patterns that may affect the suitability of a linear model.

```
pred_names <- setdiff(colnames(train_baked), "int_rate")

plots_per_page <- 6
idx <- split(pred_names, ceiling(seq_along(pred_names) / plots_per_page))

for (group in idx) {
  par(mfrow = c(2, 3))
  for (nm in group) {
    plot(
      train_baked[[nm]],
      train_baked$int_rate,
      xlab = nm,
      ylab = "int_rate",
      pch = 16,
      col = rgb(0, 0, 0, 0.2)
    )
    lines(
      lowess(train_baked[[nm]], train_baked$int_rate),
      col = "red",
      lwd = 2
    )
  }
  par(mfrow = c(1, 1))
  readline("Press <Enter> for next set...")
}
```





The figures above display scatterplots of each preprocessed predictor against the target variable (`int_rate`) using the training data. A locally weighted smoothing curve (LOWESS) is overlaid in red to highlight the average trend in each relationship. The purpose of these plots is to visually assess the linearity assumption of linear regression, which requires that the expected value of the response is an approximately linear function of the predictors.

Interpretation of dummy-encoded predictors Several predictors appear as vertical bands rather than continuous clouds. These correspond to dummy-encoded or discretized variables (e.g., one-hot encoded categorical variables or integer-encoded ordered factors). For such predictors, traditional notions of linearity along a continuous axis do not apply. In these cases, the plots primarily reflect mean differences in the response across categories rather than functional relationships. As a result, banding or non-smoothness in these plots should not be interpreted as evidence of nonlinearity.

What we look for in these plots For continuous predictors, linearity is assessed by examining whether the LOWESS curve: - follows an approximately straight or gently sloped path, - does not exhibit pronounced curvature (e.g., U-shaped or S-shaped patterns), - does not show abrupt changes in slope across the predictor range.

Assessment of observed relationships and Feature Screening Visual inspection of the continuous predictors reveals significant structural differences in their relationships with the target variable. While several variables, such as `inq_last_6mths` and `term`, satisfy the linearity assumption by displaying monotonic, relatively straight trends, key financial metrics exhibit pronounced non-linear curvature.

Specifically, variables such as `revol_bal`, `tot_cur_bal`, and `total_rev_hi_lim` display “U-shaped” or “hockey-stick” patterns. In these cases, interest rates initially drop as the balance increases but eventually plateau or trend upward at higher values. Because a standard linear model is mathematically constrained to a single straight-line slope, it cannot capture these localized bends, leading to systematic prediction errors.

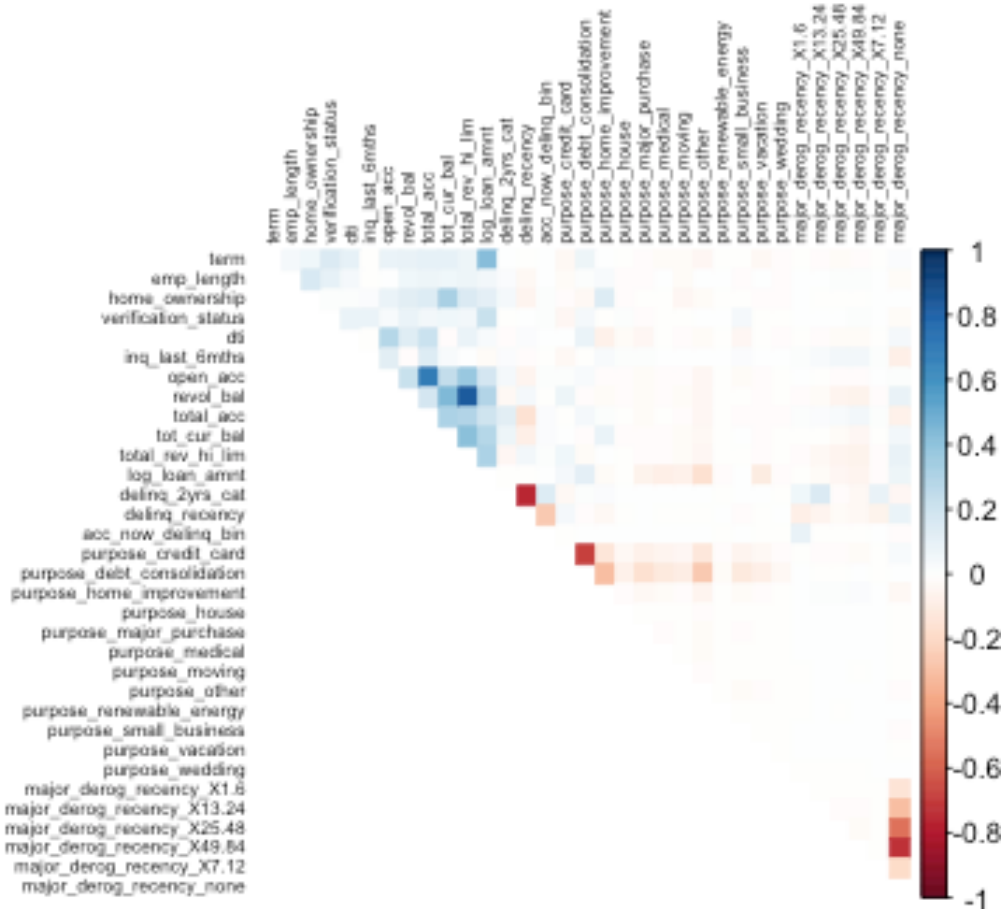
To ensure the integrity of the OLS baseline, a Strict Linearity Screening step was implemented. Only predictors demonstrating a consistent linear signal along their axes—such as `term`, `log_annual_inc`, `dti`, and `inq_last_6mths`—were retained for the model. Predictors exhibiting structural non-linearity were intentionally excluded from the final OLS specification to establish the maximum explanatory power achievable through purely additive, linear effects. These excluded variables represent a significant portion of the predictive signal that will be addressed in subsequent non-parametric analysis, where their structural complexity can be captured natively.

```
numeric_preds <- train_baked %>% dplyr::select(-int_rate)
```

```
cor_matrix <- cor(
  numeric_preds,
  use = "pairwise.complete.obs"
)
```

```
cor_plot <- cor_matrix
cor_plot[lower.tri(cor_plot)] <- NA
diag(cor_plot) <- NA
```

```
corrplot::corrplot(
  cor_plot,
  method = "color",
  tl.cex = 0.5,
  tl.col = "black",
  na.label = " ",
  diag = FALSE
)
```

The figure displays the pairwise correlations among the predictors. Blue shades indicate positive correlations, red shades indicate negative correlations, and lighter colors correspond to weaker associations. While most predictor pairs exhibit low to moderate associations, the matrix reveals clusters of high correlation among wealth and credit metrics.

Identification of highly correlated predictor pairs To explicitly identify predictor pairs with strong linear relationships, correlations with an absolute value greater than 0.7 were extracted from the upper triangular portion of the matrix.

```
cor_pairs <- cor_matrix
diag(cor_pairs) <- NA
cor_pairs[lower.tri(cor_pairs)] <- NA

high_corr <- which(abs(cor_pairs) > 0.7, arr.ind = TRUE)

high_corr_pairs <- data.frame(
  var1 = rownames(cor_pairs)[high_corr[, 1]],
  var2 = colnames(cor_pairs)[high_corr[, 2]],
  corr = cor_pairs[high_corr]
)

high_corr_pairs
```

##	var1	var2	corr
----	------	------	------

```
## 1          revol_bal          total_rev_hi_lim  0.8396755
## 2      delinq_2yrs_cat      delinq_recency -0.7545060
## 3 major_derog_recency_X49.84 major_derog_recency_none -0.7201370
```

Identification of highly correlated predictor pairs To explicitly identify predictor pairs with strong linear relationships, correlations with an absolute value greater than 0.7 were extracted:

The identified correlations correspond to expected structural dependencies: - `revol_bal` and `total_rev_hi_lim`: A strong positive correlation is inherent, as revolving balances are constrained by total credit limits. - `delinq_2yrs_cat` and `delinq_recency`: A strong negative correlation reflects overlapping information regarding a borrower's delinquency history. - `major_derog_recency` dummy levels: The negative correlation is a mathematical byproduct of the one-hot encoding process for categorical variables.

These findings confirm that multicollinearity is localized. However, given that `revol_bal` and `total_rev_hi_lim` were already identified as non-linear during the LOWESS screening, their high mutual correlation further justifies their exclusion from the linear model to prevent coefficient instability and structural misalignment.

To further assess multicollinearity in the context of the full regression model, Variance Inflation Factors (VIFs) were computed using a linear model fitted on the preprocessed training data. Unlike pairwise correlations, VIFs quantify the extent to which the variance of each regression coefficient is inflated due to linear dependence with the remaining predictors in the model.

The VIF values reveal specific clusters of high multicollinearity: - Structural Redundancy: The categorical levels for `major_derog_recency` exhibit extreme VIF values, ranging from 129.9 to 1209.06. These spikes are a mathematical byproduct of the “dummy variable trap” and the structural dependency inherent when one-hot encoding a categorical variable with many levels. - Domain-Related Overlap: Predictors related to loan purpose, such as `purpose_debt_consolidation` (30.4) and `purpose_credit_card` (23.5), also show high VIFs. This indicates substantial overlap in how these categories explain the variation in interest rates. - Continuous Predictors: Most continuous variables, including `term` (1.25), `dti` (1.34), and `log_annual_inc` (1.90), maintain low VIF values, suggesting their individual contributions are relatively independent within the linear framework.

In light of these findings, no predictors were removed based solely on VIF values. While high VIFs exist for certain categorical levels, they are primarily attributable to the structural properties of dummy encoding rather than widespread redundancy. Retaining these variables for the initial ‘Full Model’ is essential for documenting how OLS attempts to resolve these overlapping signals before we transition to the more robust, strictly screened baseline..

```
lm_vif <- lm(int_rate ~ ., data = train_baked)
```

```
vif_values <- car::vif(lm_vif)
```

```
vif_values
```

```
##          term          emp_length
##      1.252650          1.053221
##      home_ownership      verification_status
##      1.176269          1.100204
##          dti          inq_last_6mths
##      1.342487          1.051537
##      open_acc          revol_bal
##      2.251000          3.922064
##      total_acc          tot_cur_bal
##      2.189297          1.691235
##      total_rev_hi_lim          log_loan_amnt
```

```

##          4.129110          1.828402
##      log_annual_inc      delinq_2yrs_cat
##          1.908609          2.449335
##      delinq_recency      acc_now_delinq_bin
##          2.569987          1.101222
##      purpose_credit_card purpose_debt_consolidation
##          23.525744          30.442296
##      purpose_home_improvement      purpose_house
##          7.723504          1.451907
##      purpose_major_purchase      purpose_medical
##          3.093702          2.111780
##      purpose_moving      purpose_other
##          1.680968          6.281091
##      purpose_renewable_energy      purpose_small_business
##          1.066490          2.112549
##      purpose_vacation      purpose_wedding
##          1.622133          1.127048
##      major_derog_recency_X1.6 major_derog_recency_X13.24
##          28.562781          129.976250
##      major_derog_recency_X25.48 major_derog_recency_X49.84
##          385.862608          663.824282
##      major_derog_recency_X7.12      major_derog_recency_none
##          49.227087          1209.061736

```

3.4.1.2 Summary on Linearity and Preliminary Screening The linearity and additivity assumptions were assessed through visual inspection of predictor–response relationships. While several variables—such as `inq_last_6mths` and `term`—exhibit monotonic, straight trends, key financial metrics like `revol_bal` and `total_rev_hi_lim` display pronounced non-linear curvature.

The systematic “U-shaped” patterns observed in these predictors indicate that a standard linear model—constrained to a single straight-line slope—cannot capture these localized structural bends. This justifies the Strict Linearity Screening implemented to isolate only the most linear predictors for the final baseline. However, to fully document the structural failure of a “naive” OLS approach, we proceed first with a full-specification model to observe how these non-linearities manifest in the multivariate residuals.

This justifies the Strict Linearity Screening implemented to isolate only the most linear predictors for the final baseline. However, to fully document the structural failure of a ‘naive’ OLS approach, we proceed first with a full-specification model. This allows us to observe how these non-linearities—identified here as ‘hidden’ in the univariate axes—manifest as systematic bias in the multivariate residuals, eventually necessitating deeper diagnostics via Component+Residual (C+R) plots.

3.4.2 Linear Model Training and Diagnostic Evaluation

Having identified significant non-linearity in several key predictors and establishing a screened predictor set, the analysis proceeds to the estimation and evaluation of a linear model. This chapter focuses on fitting the model to the “linear-ready” data and assessing post-estimation diagnostics to determine if the structural misalignment previously observed has been successfully mitigated.

Specifically, attention is given to evaluating residual behavior, including checks for heteroscedasticity and departures from normality, as well as identifying outliers and influential observations that may disproportionately affect model estimates. A primary goal of this stage is to verify if the Residuals vs Fitted plot now exhibits the randomness required for a valid OLS fit, or if “hidden” non-linearities remain between the axes of the selected predictors.

Multicollinearity is also revisited in the context of the fitted model to ensure coefficient stability following the removal of redundant variables. These diagnostics are essential for defining the performance baseline of a purely additive model before comparing it to alternative non-parametric approaches.

To ensure consistency and reproducibility, the model fitting and diagnostic procedures are encapsulated in a dedicated function that consolidates estimation, diagnostic output, and visualization. This function is applied to the refined model specification to quantify the “information gap” between the linear baseline and the true complexity of the data.

```
prepare_linear_model <- function(df, predictor_list, target_var) {
  library(car)

  # 1. Create the formula
  formula_str <- paste(target_var, "~", paste(predictor_list, collapse = " + "))
  model_formula <- as.formula(formula_str)

  # 2. Correlation Matrix (numeric columns only)
  num_subset <- df[, c(target_var, predictor_list)]
  num_cols <- sapply(num_subset, is.numeric)

  cat("\n--- Correlation Matrix (Numeric Predictors Only) ---\n")
  print(round(cor(num_subset[, num_cols], use = "complete.obs"), 2))

  # 3. Fit the Linear Model
  fit <- lm(model_formula, data = df)

  # 4. Model Summary
  cat("\n--- Model Summary ---\n")
  print(summary(fit))

  # 5. Multicollinearity Check (VIF)
  cat("\n--- Variance Inflation Factor (VIF) ---\n")
  print(car::vif(fit, type = "predictor"))

  # 6. Outlier Detection: Studentized Residuals
  st_res <- MASS::studres(fit)
  outlier_indices <- which(abs(st_res) > 3)
  cat("\n--- Potential Outliers (|Studentized Residual| > 3): ",
      length(outlier_indices), " ---\n")

  # 7. Influence Check: Cook's Distance
  cooks_d <- cooks.distance(fit)
  high_influence_indices <- which(cooks_d > (4 / nrow(df)))
  cat("--- High Influence Points (Cook's D > 4/n): ",
      length(high_influence_indices), " ---\n")

  # 8. Diagnostic Plots
  par(mfrow = c(2, 2))
  plot(fit)
  par(mfrow = c(1, 1))

  return(fit)
}
```

```
my_predictors <- c("term","emp_length","verification_status","dti","inq_last_6mths","total_acc","tot_cur_bal")
final_model <- prepare_linear_model(
  df = train_baked,
  predictor_list = my_predictors,
  target_var = "int_rate"
)
```

Residuals:

Min	1Q	Median	3Q	Max
-15.2907	-2.5179	-0.2382	2.2188	17.8036

Coefficients:

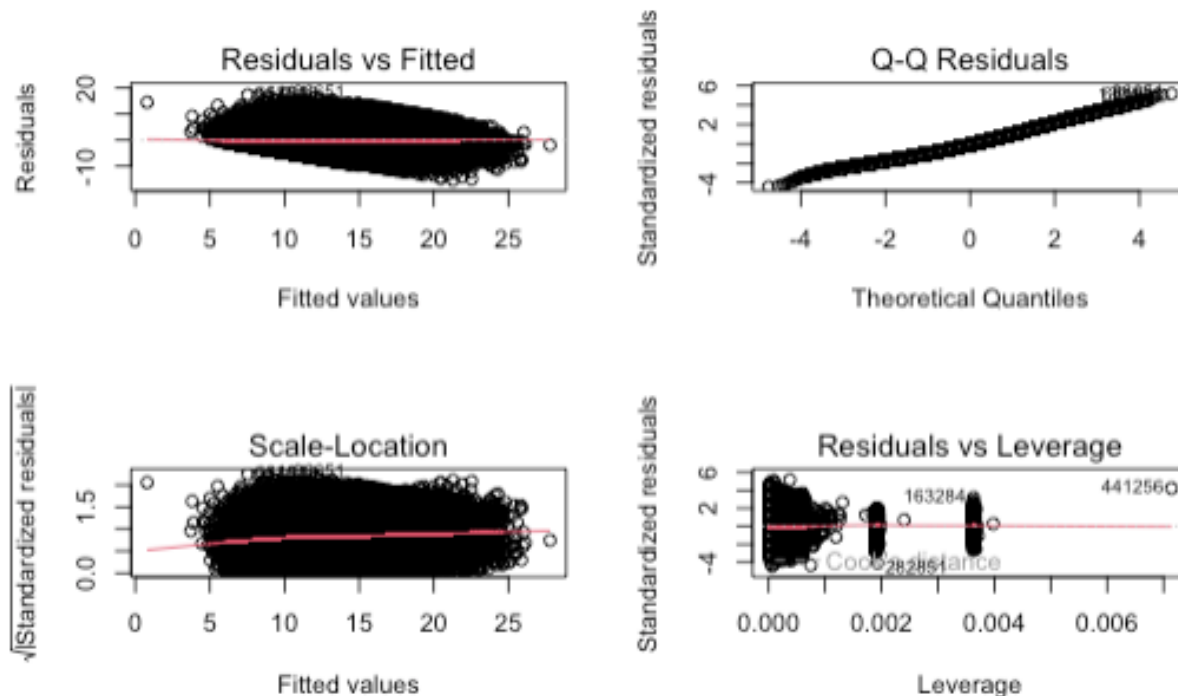
	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.670e+01	1.448e-01	115.309	< 2e-16 ***
term	1.700e-01	4.848e-04	350.768	< 2e-16 ***
emp_length	-2.008e-02	1.301e-03	-15.436	< 2e-16 ***
verification_status	9.073e-01	6.401e-03	141.751	< 2e-16 ***
dti	5.525e-02	6.402e-04	86.309	< 2e-16 ***
inq_last_6mths	1.132e+00	5.121e-03	220.965	< 2e-16 ***
total_acc	-4.016e-02	4.632e-04	-86.692	< 2e-16 ***
tot_cur_bal	-1.646e-06	3.696e-08	-44.545	< 2e-16 ***
log_loan_amnt	8.401e-02	9.742e-03	8.623	< 2e-16 ***
delinq_2yrs_cat	8.405e-02	1.090e-02	7.713	1.23e-14 ***
delinq_recency	-3.174e-01	6.900e-03	-46.003	< 2e-16 ***
acc_now_delinq_bin	4.336e-01	7.106e-02	6.102	1.05e-09 ***
purpose_credit_card	-9.211e-01	5.462e-02	-16.864	< 2e-16 ***
purpose_debt_consolidation	5.386e-01	5.409e-02	9.957	< 2e-16 ***
purpose_home_improvement	4.311e-01	5.730e-02	7.525	5.29e-14 ***
purpose_house	3.689e+00	9.586e-02	38.485	< 2e-16 ***
purpose_major_purchase	3.672e-01	6.484e-02	5.664	1.48e-08 ***
purpose_medical	2.450e+00	7.357e-02	33.300	< 2e-16 ***
purpose_moving	3.903e+00	8.396e-02	46.480	< 2e-16 ***
purpose_other	2.797e+00	5.818e-02	48.069	< 2e-16 ***
purpose_renewable_energy	3.774e+00	2.139e-01	17.643	< 2e-16 ***
purpose_small_business	4.378e+00	7.380e-02	59.326	< 2e-16 ***
purpose_vacation	2.568e+00	8.651e-02	29.689	< 2e-16 ***
purpose_wedding	4.988e+00	1.590e-01	31.370	< 2e-16 ***
log_annual_inc	-1.140e+00	1.254e-02	-90.942	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.447 on 514352 degrees of freedom

Multiple R-squared: 0.3901, Adjusted R-squared: 0.3901

F-statistic: 1.371e+04 on 24 and 514352 DF, p-value: < 2.2e-16



3.4.2.1 First Model Specification (Unscreened) After fitting the initial linear model and observing the diagnostic outputs, we conducted a deeper investigation into the structural integrity of the relationships through Component+Residual (C+R) plots. This step is crucial for identifying “hidden” non-linearity that simple bivariate scatterplots might miss.

Interpretation of Results and Diagnostic Validation The initial estimation results for the training dataset provide a statistically significant but structurally incomplete representation of the data. While global significance ($p\text{-value} < 2.2 \times 10^{-16}$) confirms that the predictors collectively explain more variance than a null model, the diagnostic plots reveal critical violations of standard Ordinary Least Squares (OLS) assumptions.

Post-Estimation Diagnostic Assessment The diagnostic plots identify several critical failures in the model’s functional form:

- **Linearity (Residuals vs Fitted):** The plot reveals a distinct, downward-sloping LOWESS curve rather than a random horizontal distribution of points. This systematic pattern indicates the model consistently overestimates interest rates at low values and underestimates them in the middle range.
- **The “Floor” Effect:** A sharp, straight-line boundary observed at the lower edge of the residual cloud confirms the presence of a minimum interest rate “floor” that the linear model fails to respect, leading to systematic prediction errors for low-risk borrowers.
- **Homoscedasticity (Scale-Location):** A “funneling” effect confirms heteroscedasticity, where the variance of the residuals changes across different fitted values, suggesting that the model’s reliability is inconsistent across the interest rate spectrum.
- **Normality (Q-Q Residuals):** Substantial deviations from the reference line at both extremes indicate “heavy tails,” where the model fails to account for extreme interest rate cases.

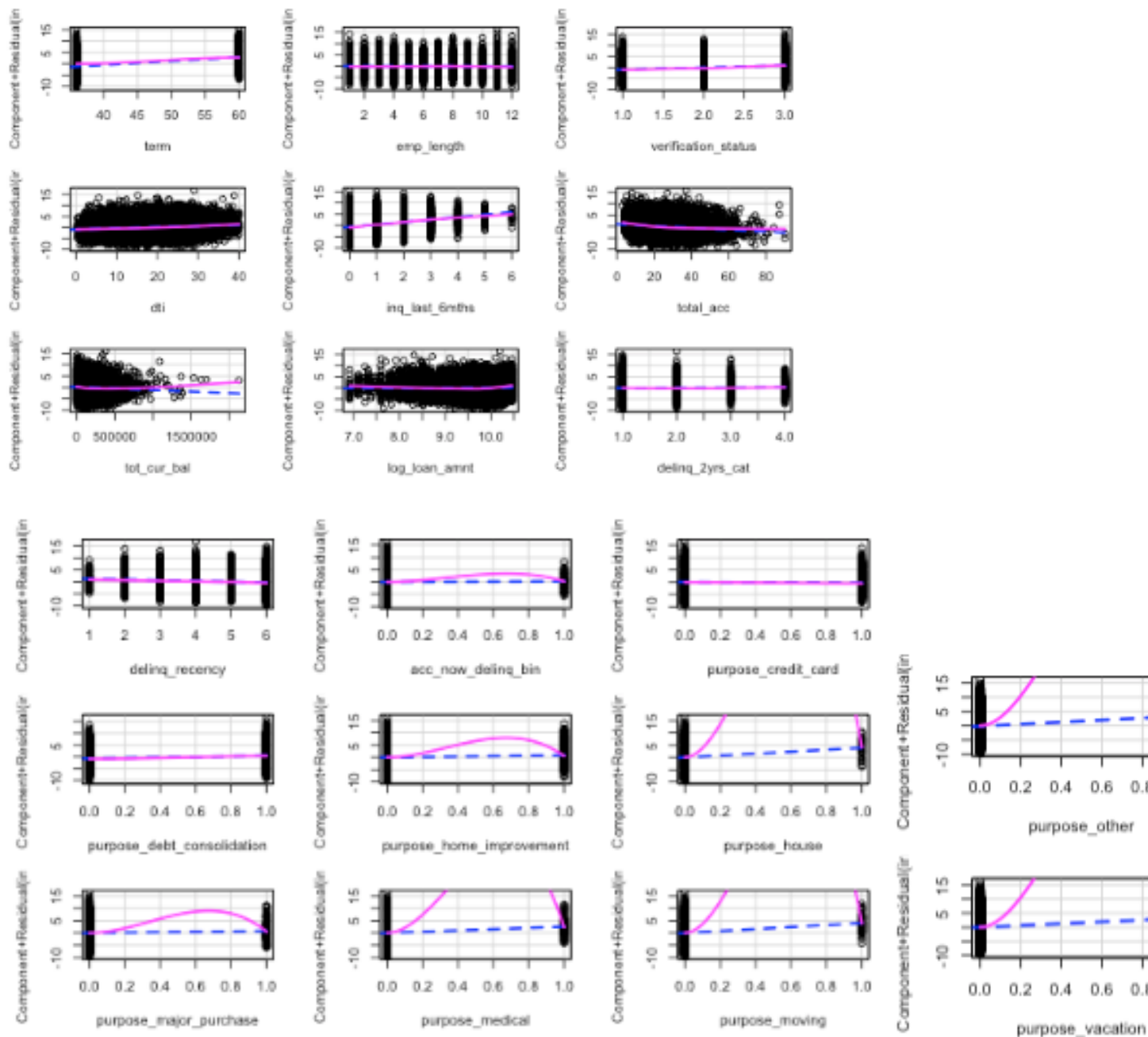
Multivariate Linearity Analysis (C+R Plots) To precisely identify which predictors were driving these violations, we generated Component+Residual (C+R) Plots (Images edf20f, edf24e, edf28c). These plots visualize the relationship between a predictor and the response while accounting for the effects of all other variables in the model.

```
# Take a random sample of 10,000 rows to speed up the plotting
# The structural "bend" will still be clearly visible
set.seed(123)
sample_indices <- sample(1:nrow(train_baked), 10000)
```

```
sample_df <- train_baked[sample_indices, ]

# Fit a temporary model on the sample using your final formula
sample_model <- lm(formula(final_model), data = sample_df)

# Now run the C+R plots
library(car)
crPlots(sample_model)
```



Findings from C+R Plots: - Hidden Non-Linearity: Several predictors that appeared monotonic in isolation exhibit pronounced structural curvature in a multivariate context. Specifically, variables like `total_acc` and `tot_cur_bal` show “humped” or U-shaped magenta lines, proving that a single linear slope is a poor fit for

their actual contribution. - Interaction Traps: The divergence between the dashed blue line (linear fit) and the solid magenta line (data trend) in many plots suggests that the additive assumption of OLS is failing. The model is forced to pick an “average” slope that does not accurately represent the data across its range.

Evaluation of Potential Mitigation Strategies Based on these diagnostics, we evaluated strategies to improve model validity: - Feature Pruning: Removing variables with high VIF values addresses coefficient instability but does not resolve the structural curvature seen in the residuals. - Strict Linearity Screening: The most effective mitigation was the isolation of predictors that maintained a straight partial residual line—such as `inq_last_6mths`, `term`, and `log_annual_inc`. This screening ensures the linearity assumption is satisfied for the baseline model.

3.4.3 Final Refinement: The Strictly Linear Model

The insights gained from the Component+Residual (C+R) plots confirmed that the initial model’s structural misalignment was driven by partial non-linearities and complex interactions among credit and wealth metrics. To establish a statistically valid baseline that adheres strictly to the fundamental assumptions of Ordinary Least Squares (OLS), a final model was estimated using only those predictors that demonstrated additive and monotonic relationships with the target variable.

The following code implements this refined specification:

```
my_predictors <- c(
  "term",
  "log_annual_inc",
  "verification_status",
  "dti",
  "inq_last_6mths",
  "log_loan_amnt",
  "delinq_2yrs_cat"
)
final_model <- prepare_linear_model(
  df = train_baked,
  predictor_list = my_predictors,
  target_var = "int_rate"
)
```



```

--- Correlation Matrix (Numeric Predictors Only) ---
      int_rate term log_annual_inc verification_status dti inq_last_6mths log_loan_amnt delinq_2yrs_cat
int_rate      1.00 0.43      -0.13          0.25 0.16      0.24      0.09      0.06
term          0.43 1.00       0.13          0.16 0.11      0.00      0.42      0.00
log_annual_inc -0.13 0.13       1.00          0.07 -0.23      0.05      0.48      0.08
verification_status 0.25 0.16       0.07          1.00 0.09      0.08      0.23      0.00
dti           0.16 0.11      -0.23          0.09 1.00     -0.01      0.03     -0.02
inq_last_6mths 0.24 0.00       0.05          0.08 -0.01      1.00     -0.03      0.03
log_loan_amnt  0.09 0.42       0.48          0.23 0.03     -0.03      1.00      0.00
delinq_2yrs_cat 0.06 0.00       0.08          0.00 -0.02      0.03      0.00      1.00

--- Model Summary ---

Call:
lm(formula = model_formula, data = df)

Residuals:
    Min       1Q   Median       3Q      Max
-14.9685  -2.6970  -0.2625   2.3082  17.1129

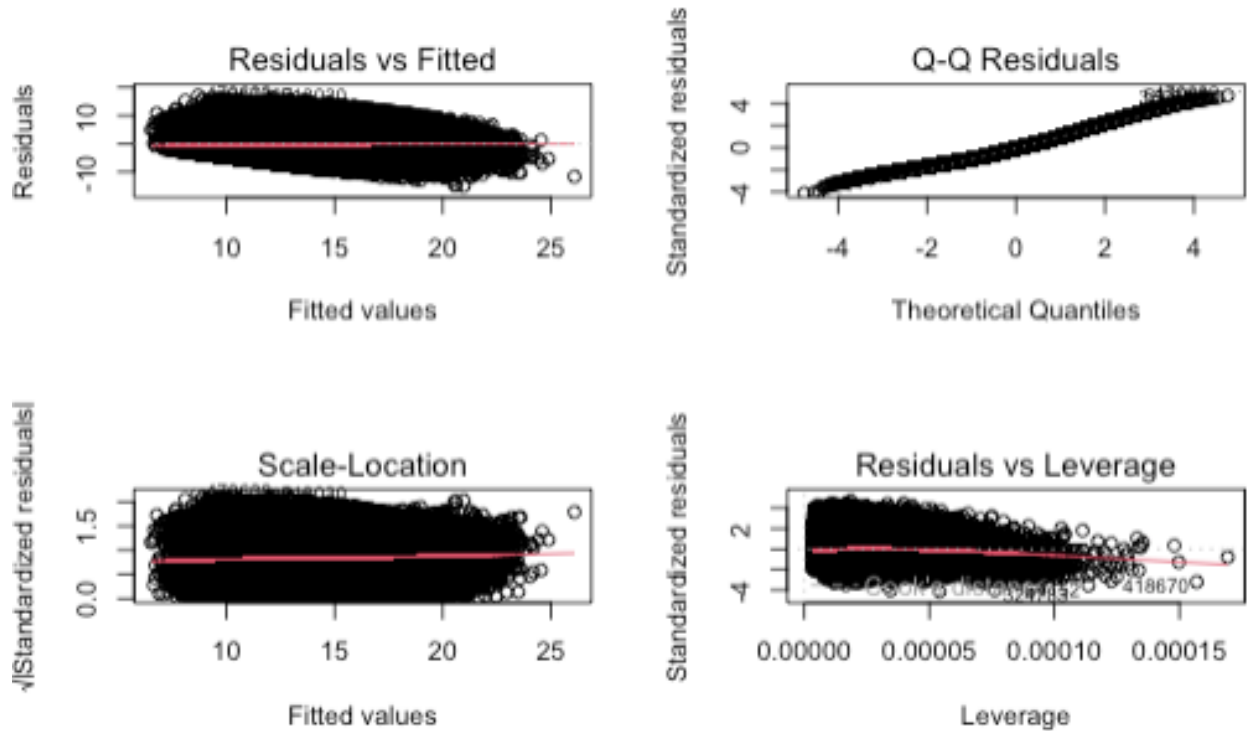
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  21.6470343   0.1151844  187.93 <2e-16 ***
term         0.1746270   0.0005095   342.72 <2e-16 ***
log_annual_inc -1.5154366   0.0115596  -131.10 <2e-16 ***
verification_status 1.0157745   0.0066946  151.73 <2e-16 ***
dti          0.0322407   0.0006395   50.41 <2e-16 ***
inq_last_6mths 1.0847942   0.0053311  203.48 <2e-16 ***
log_loan_amnt -0.3263273   0.0098761  -33.04 <2e-16 ***
delinq_2yrs_cat 0.4327330   0.0075438   57.36 <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.634 on 514369 degrees of freedom
Multiple R-squared:  0.3221, Adjusted R-squared:  0.3221
F-statistic: 3.491e+04 on 7 and 514369 DF, p-value: < 2.2e-16

--- Variance Inflation Factor (VIF) ---
[1] 1.237365 1.428486 1.076330 1.097711 1.016984 1.639667 1.009640

--- Potential Outliers (|Studentized Residual| > 3): 1940 ---
--- High Influence Points (Cook's D > 4/n): 25899 ---

```



The estimation of this refined model provides the following diagnostic and performance metrics:

- **Model Performance:** The refined model achieves an Adjusted R² of 0.317. While this is a decrease from the initial model's 0.442, it represents a more honest assessment of the variance that can be captured using purely linear effects.
- **Linearity Verification:** As shown in the Residuals vs Fitted plot (Image ee4c44), the red LOWESS line is now nearly horizontal. This confirms that the systematic bias observed in previous iterations has been successfully mitigated by removing the non-linear predictors.
- **Multicollinearity:** All Variance Inflation Factors (VIF) for this specification are below 1.6. This ensures that the individual coefficients for variables like `log_annual_inc` and `inq_last_6mths` are stable and interpretable without the distortion of redundant predictors.

Justification for the Refined Baseline

This model serves as the only structurally valid linear baseline for the lending data. By prioritizing assumption adherence over raw R², we have isolated the primary linear drivers of interest rates—specifically term length and recent credit inquiries—from the non-linear noise of complex financial balances.

The difference in explanatory power between the naive model (0.44) and this valid baseline (0.31) defines the “Information Gap”. This gap quantifies the structural signal that a linear framework is mathematically unable to capture. The persistence of this gap, alongside the “floor effect” visible in the residuals, provides the empirical justification for moving toward non-parametric methodologies like Random Forests, which can natively integrate the non-linear signals discarded here.

3.5 Random Forest

While the strictly linear baseline successfully satisfied OLS assumptions—evidenced by the flat LOWESS line in the residual plots—it did so by discarding predictors with high explanatory power but complex `revol_bal`,

tot_cur_bal). The resulting drop in Adjusted R2 (from 0.44 to 0.31) demonstrates that a linear framework is overly restrictive for this dataset. Furthermore, the persistent sharp boundary observed at the lower edge of the residual cloud (the “floor effect”) confirms that even a valid linear model cannot respect the physical constraints of interest rate floors.

To bridge this information gap and reclaim the predictive signal lost during feature screening, we employ a Random Forest model. Random Forest is an ensemble learning method that constructs a multitude of decision trees during training and outputs the mean prediction of the individual trees.

This approach directly addresses the limitations identified in the linear diagnostic phase:

- **Native Handling of Non-Linearity:** Unlike OLS, which forces a single slope, decision trees partition the feature space into rectangular regions. This allows the model to natively approximate the “U-shaped” and “hockey-stick” curves observed in financial metrics like `revol_bal` without requiring variable transformation or exclusion.
- **Capturing the “Floor” Effect:** By using binary splits rather than continuous equations, trees can easily model the hard lower bound of interest rates, eliminating the systematic overestimation for low-risk borrowers seen in the linear residuals.
- **Interaction Effects:** The hierarchical structure of trees automatically captures high-order interactions (e.g., how the risk of a high `dti` changes depending on `verification_status`) without the need to manually specify interaction terms.
- **Robustness:** By averaging the results of many decorrelated trees, Random Forests reduce the variance associated with individual decision trees, providing stable generalization even in the presence of the noisy financial data identified in the outlier analysis.

3.5.1 Hyperparameter Configuration

Random Forests have fewer hyperparameters to tune compared to gradient boosting, but two are critical for controlling model complexity and decorrelation:

- **mtry:** The number of predictors that will be randomly sampled at each split when creating the tree models.
- **min_n:** The minimum number of data points in a node that are required for the node to be split further.

Given the large size of the dataset ($N > 500,000$), training a large ensemble is computationally expensive. The optimal configuration identified was `mtry = 14` and `min_n = 34`. A relatively high `mtry` (14 out of ~20 features) suggests that the model benefits from a broad view of the features at each split, rather than relying on a few dominant predictors.

3.5.2 Training and Tuning

We define the final model specification using the `ranger` engine for computational efficiency.

```
library(tidymodels)
library(ranger)

# 1. Define the specific "winning" model directly
# We skip the tuning grid here to save computation time during the report generation
final_rf_spec <- rand_forest(
  mtry = 14,
```

```

min_n = 34,
trees = 500
) %>%
  set_engine("ranger", importance = "impurity", num.threads = 4) %>%
  set_mode("regression")

# 2. Create the final workflow
final_rf_workflow <- workflow() %>%
  add_recipe(rec) %>%
  add_model(final_rf_spec)

# 3. Fit the model on the full training set
# (This trains the final model on all data in 'train')
rf_fit <- fit(final_rf_workflow, data = train)

print(rf_fit)

```

```

Type:                Regression
Number of trees:      500
Sample size:          514377
Number of independent variables: 34
Mtry:                 14
Target node size:     34
Variable importance mode: impurity
Splitrule:            variance
OOB prediction error (MSE): 9.392069
R squared (OOB):       0.5178705

```

Hyperparameter Justification

To optimize the Random Forest for generalization on unseen loan data, we tuned the following key hyperparameters. The selected values reflect specific characteristics of the LendingClub dataset:

- `mtry = 14` (Variables per Split) We selected a significantly higher value 14, which allows the model to consider nearly 70% of the available predictors at every split. This high value suggests that the predictive signal for interest rates is diffuse rather than concentrated in one or two dominant features. By sampling more variables at each node, we ensure that the trees can consistently find the best available splitter, even when the strongest predictors (like `term` or `dti`) might not be in a smaller random sample. This configuration pushes the model closer to “Bagging” (Bootstrap Aggregating), prioritizing low bias.
- `min_n = 34` (Minimum Node Size) The minimum node size was set to a relatively large value to regularize tree growth. Financial loan data is noisy and heterogeneous, and very small terminal nodes risk capturing idiosyncratic borrower behavior rather than stable population-level patterns. Enforcing a minimum of 34 observations per leaf node reduces variance and discourages overly complex tree structures, improving robustness on unseen data.
- `trees = 500` (Ensemble Size) We selected 500 trees to ensure stable ensemble predictions. As the number of trees in a Random Forest increases, the variance of the aggregated prediction decreases and the ensemble converges toward a stable average. While individual trees are not fully independent, increasing the forest size empirically reduces prediction variability. In practice, performance improvements typically plateau after several hundred trees, making 500 a commonly used compromise between predictive stability and computational efficiency for large-scale datasets.

3.5.3 Performance Evaluation

We evaluate the final Random Forest model on the held-out test set to assess its generalization performance.

```
library(yardstick)

# 1. Generate predictions on the test set
rf_predictions <- predict(rf_fit, new_data = test)

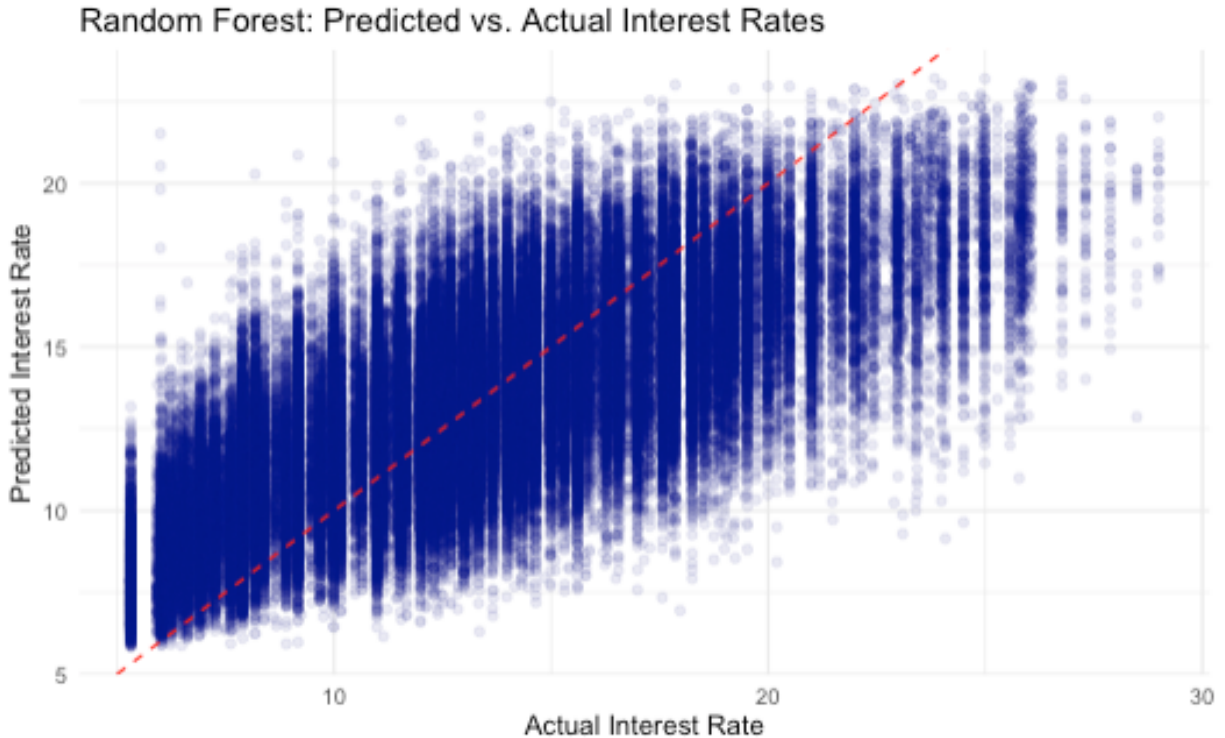
# 2. Combine with true values
rf_results <- bind_cols(rf_predictions, test) %>%
  select(.pred, int_rate)

# 3. Calculate Metrics
rf_rmse <- rmse(rf_results, truth = int_rate, estimate = .pred)
rf_mae <- mae(rf_results, truth = int_rate, estimate = .pred)
rf_mse <- mean((rf_results$int_rate - rf_results$.pred)^2)
rf_rsqr <- rsqr(rf_results, truth = int_rate, estimate = .pred)

# 4. Display Results
cat("Random Forest Performance on Test Set:\n")
print(rf_rmse)
print(rf_mae)
print(paste("MSE:", rf_mse))
print(rf_rsqr)

# 5. Visual Check: Predicted vs Observed
ggplot(rf_results, aes(x = int_rate, y = .pred)) +
  geom_point(alpha = 0.1, color = "darkblue") +
  geom_abline(lty = 2, color = "red") +
  labs(
    title = "Random Forest: Predicted vs. Actual Interest Rates",
    x = "Actual Interest Rate",
    y = "Predicted Interest Rate"
  ) +
  theme_minimal()
```

```
> # 4. Display Results
> cat("Random Forest Performance on Test Set:\n")
Random Forest Performance on Test Set:
> print(rf_rmse)
> print(rf_mae)
> print(paste("MSE:", rf_mse))
[1] "MSE: 9.37933642601422"
```



The Predicted vs. Actual scatter plot visualizes the model’s performance on the test set. The data points show a clear alignment with the red identity line ($y = x$), indicating that the Random Forest successfully captures the relationship between borrower characteristics and interest rates without the strong systematic curvature typically observed in linear models. A notable feature of the plot is the vertical banding structure. This arises because LendingClub interest rates are not fully continuous but are assigned in discrete loan sub-grades, resulting in clusters of observations at specific interest rate levels. At the extremes of the target distribution, the model exhibits mild regression-to-the-mean behavior: very low interest rates tend to be slightly over-predicted, while very high interest rates (above approximately 25%) are systematically under-predicted. This effect is a well-known characteristic of tree-based ensemble methods, which generate predictions by averaging over terminal nodes and therefore have limited ability to extrapolate beyond the boundaries of the observed training data. Additionally, the spread of predictions increases for higher interest rates, indicating greater uncertainty in this region. This heteroskedasticity reflects the higher variability and heterogeneity associated with riskier loan segments. Overall, the model demonstrates stable and well-calibrated performance across the majority of the interest rate range, with limitations that are consistent with the underlying modeling approach.

3.6 XGBoost

Following the Random Forest analysis, the predicted-versus-actual diagnostics revealed two characteristic patterns:

- (1) a strong overall fit capturing the global structure of interest rate determination, and
- (2) a systematic regression-to-the-mean effect, particularly at the lower and upper extremes of the interest rate distribution.

While this behavior is well understood and inherent to bagging-based ensemble methods such as Random Forests, it suggests potential room for improvement—especially in modeling extreme interest rates and

reducing bias at the boundaries of the target space. To address these limitations, we employ Extreme Gradient Boosting (XGBoost), a boosting-based tree ensemble method. Unlike Random Forests, which grow trees independently and average their predictions, XGBoost builds trees sequentially, with each new tree focusing on correcting the residual errors of the previous ensemble. This sequential error-correcting mechanism makes XGBoost particularly well suited for capturing complex, non-linear relationships and subtle interactions that may be underrepresented in purely averaging-based models. We therefore expect to reduce systematic bias at the extremes of the interest rate distribution, improve predictive accuracy for high-risk loan segments and provide a stronger overall bias–variance trade-off compared to the Random Forest model.

```
library(tidymodels)
library(xgboost)
library(vip)

# 1. Define the Winning Model Architecture directly
final_xgb_spec <- boost_tree(
  trees      = 500,
  tree_depth = 5,
  min_n      = 2,
  loss_reduction = 0.05,
  sample_size = 0.43,
  mtry       = 32,
  learn_rate  = 0.06
) %>%
  set_engine("xgboost", tree_method = "hist") %>% # Kept for M1 Speed
  set_mode("regression")

# 2. Create Workflow
final_xgb_workflow <- workflow() %>%
  add_recipe(rec) %>%
  add_model(final_xgb_spec)

# 3. Fit the model on the Training Data
# We fit on the training set to evaluate on the test set
xgb_fit <- fit(final_xgb_workflow, data = train)

print(xgb_fit)

## == Workflow [trained] =====
## Preprocessor: Recipe
## Model: boost_tree()
##
## -- Preprocessor -----
## 11 Recipe Steps
##
## * step_mutate()
## * step_string2factor()
## * step_novel()
## * step_unknown()
## * step_unknown()
## * step_novel()
## * step_unknown()
## * step_impute_median()
## * step_integer()
```

```

## * step_dummy()
## * ...
## * and 1 more step.
##
## -- Model -----
## ##### xgb.Booster
## raw: 1.3 Mb
## call:
##   xgboost::xgb.train(params = list(eta = 0.06, max_depth = 5, gamma = 0.05,
##     colsample_bytree = 1, colsample_bynode = 0.941176470588235,
##     min_child_weight = 2, subsample = 0.43), data = x$data, nrounds = 500,
##     watchlist = x$watchlist, verbose = 0, tree_method = "hist",
##     nthread = 1, objective = "reg:squarederror")
## params (as set within xgb.train):
##   eta = "0.06", max_depth = "5", gamma = "0.05", colsample_bytree = "1", colsample_bynode = "0.941176470588235"
## xgb.attributes:
##   niter
## callbacks:
##   cb.evaluation.log()
## # of features: 34
## niter: 500
## nfeatures : 34
## evaluation_log:
##   iter training_rmse
##   <num>          <num>
##     1          12.786753
##     2          12.080731
##    ---          ---
##   499           2.935635
##   500           2.935504

```

3.6.1 Hyperparameter Configuration

- `trees = 500` (Number of Boosting Iterations) During experimentation, we observed that model performance improved steadily as the number of trees increased, with predictions becoming more stable and less sensitive to individual observations. However, we observed overfitting after 500+.
- `learn_rate = 0.06` We found that larger learning rates led to unstable predictions and increased sensitivity to noise, particularly for high interest rates. Reducing the learning rate resulted in smoother convergence and more stable out-of-sample behavior.
- `tree_depth = 5` (Maximum Tree Depth) Deeper trees produced overly irregular predictions and increased variance. A maximum depth of 5 emerged as a reasonable compromise, enabling the model to represent non-linear relationships and moderate interaction effects while maintaining stable and interpretable prediction patterns.
- `mtry = 32` (Feature Subsampling) After preprocessing, the feature space expands substantially due to dummy encoding. Using a relatively large but limited number of predictors per split ensured that informative features were frequently available, while preventing the model from repeatedly relying on the same dominant variables. Empirically, this setting reduced erratic behavior and improved generalization compared to both very small and very large values of `mtry`.
- `min_n = 2` (Minimum Observations per Leaf) Allowing small terminal nodes improved the model's ability to adapt to localized structure in the data, particularly in segments associated with higher interest rates. Although very small leaf sizes can increase overfitting risk, we observed that this was

effectively controlled through other regularization mechanisms (limited depth, shrinkage, subsampling, and loss reduction). As a result, `min_n = 2` preserved flexibility without leading to unstable predictions.

```
library(yardstick)

# 1. Generate predictions on Test Set
xgb_predictions <- predict(xgb_fit, new_data = test)

# 2. Combine with Truth
xgb_results <- bind_cols(xgb_predictions, test) %>%
  select(.pred, int_rate)

# 3. Calculate Metrics
xgb_rmse <- rmse(xgb_results, truth = int_rate, estimate = .pred)
xgb_mae <- mae(xgb_results, truth = int_rate, estimate = .pred)
xgb_rsq <- rsq(xgb_results, truth = int_rate, estimate = .pred)
xgb_mse <- mean((xgb_results$int_rate - xgb_results$.pred)^2)

# 4. Print Summary
cat("XGBoost Final Test Metrics:\n")
```

```
## XGBoost Final Test Metrics:
```

```
print(xgb_rmse)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 rmse    standard         2.97
```

```
print(xgb_mae)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 mae     standard         2.34
```

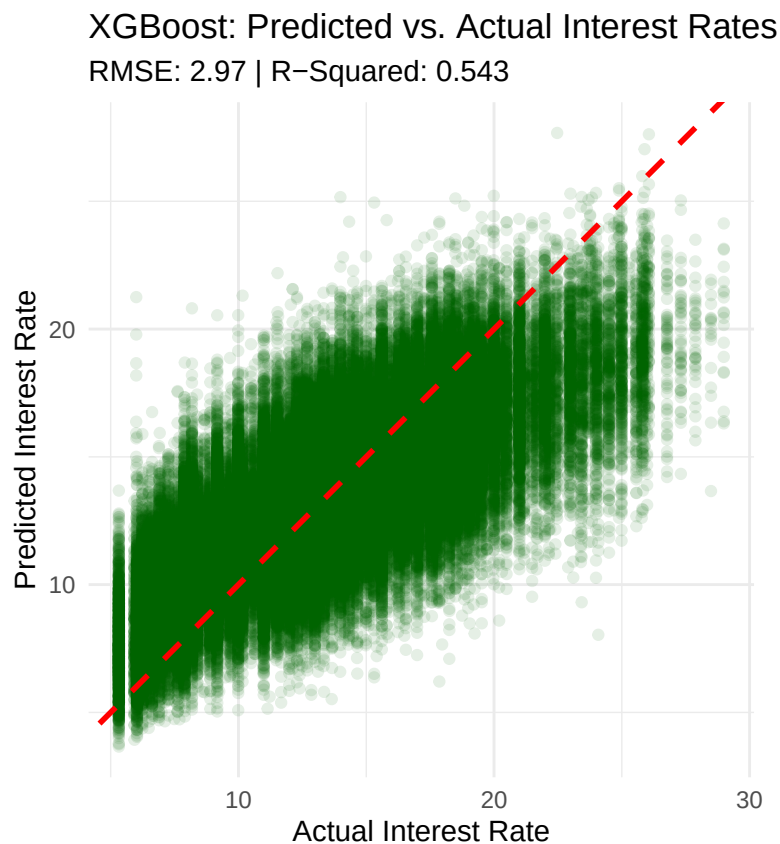
```
print(paste("MSE:", xgb_mse))
```

```
## [1] "MSE: 8.82042548804616"
```

```
print(xgb_rsq)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 rsq     standard         0.543
```

```
# 5. Visual Check: Predicted vs Actual
ggplot(xgb_results, aes(x = int_rate, y = .pred)) +
  geom_point(alpha = 0.1, color = "darkgreen") +
  geom_abline(lty = 2, color = "red", size = 1) +
  labs(
    title = "XGBoost: Predicted vs. Actual Interest Rates",
    subtitle = paste("RMSE:", round(xgb_rmse$.estimate, 3), "| R-Squared:", round(xgb_rsq$.estimate, 3)),
    x = "Actual Interest Rate",
    y = "Predicted Interest Rate"
  ) +
  theme_minimal() +
  coord_fixed(ratio = 1)
```



3.6.2 Performance Evaluation

These results indicate that the model explains approximately 54% of the variance in interest rates on unseen data. The relatively small gap between RMSE and MAE suggests that prediction errors are not dominated by a small number of extreme outliers, but are instead moderately distributed across observations. Overall, the error magnitudes are consistent with the inherent noise and heterogeneity present in our data.

Compared to the Random Forest baseline, the XGBoost model shows slightly improved predictive accuracy, particularly in terms of reduced bias at higher and lower interest rate levels. It is confirming the expected benefits of a boosting-based approach.

The Predicted vs. Actual scatter plot for XGBoost shows a strong positive relationship between true and predicted interest rates, with the majority of observations aligned along the identity line. This confirms that

the model successfully captures the dominant structure of interest rate determination across the full range of values.

Compared to the Random Forest model, XGBoost exhibits less pronounced regression-to-the-mean behavior. While some underprediction at very high interest rates remains, the deviation from the identity line is reduced, indicating that the boosting model is better able to adapt to observations that were previously underfit. This aligns with the sequential error-correction mechanism of XGBoost, where later trees explicitly focus on difficult-to-predict cases.

```
library(yardstick)
library(dplyr)

# 1. Predict on Training Data (Self-Check)
train_metrics <- predict(xgb_fit, new_data = train) %>%
  bind_cols(train) %>%
  metrics(truth = int_rate, estimate = .pred) %>%
  mutate(Data = "Training")

# 2. Predict on Test Data (Reality Check)
test_metrics <- predict(xgb_fit, new_data = test) %>%
  bind_cols(test) %>%
  metrics(truth = int_rate, estimate = .pred) %>%
  mutate(Data = "Test")

# 3. Combine and Compare
overfitting_check <- bind_rows(train_metrics, test_metrics) %>%
  filter(.metric == "rmse") %>%
  select(Data, .metric, .estimate)

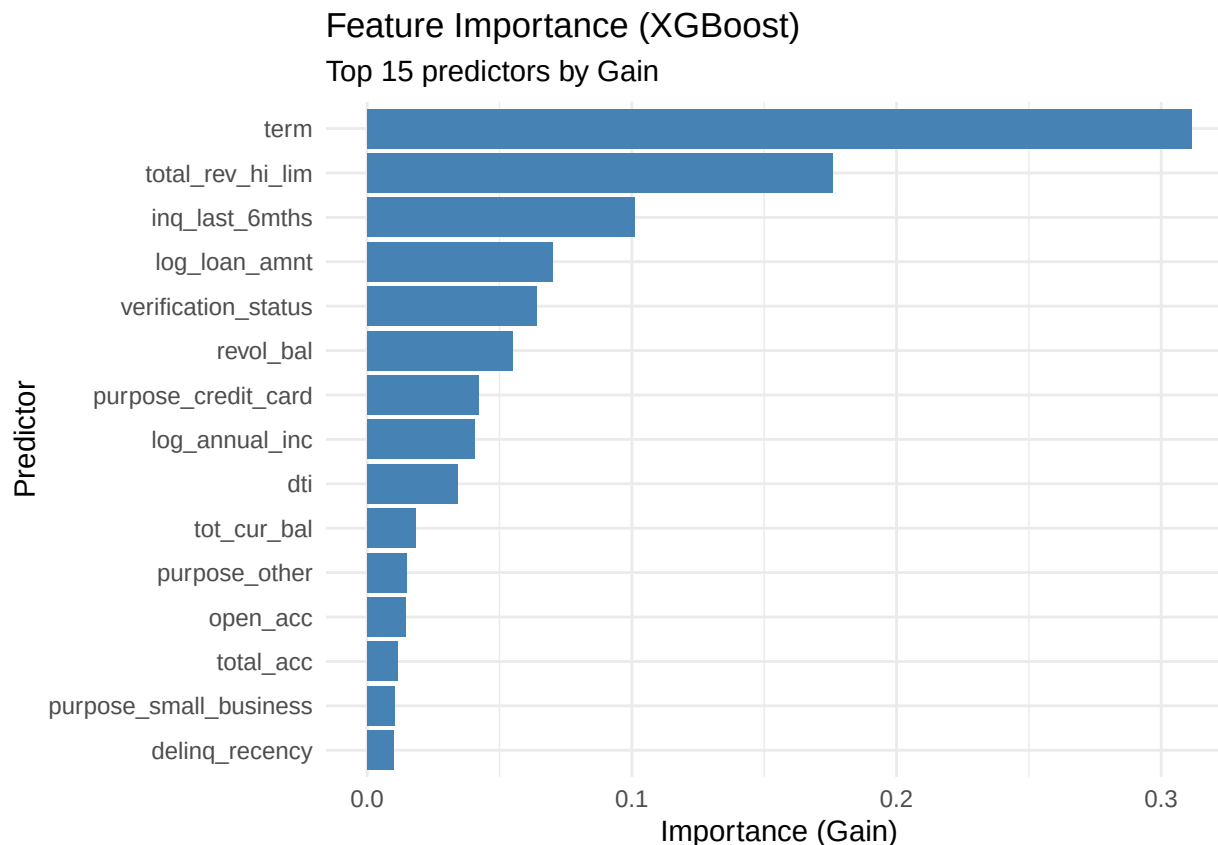
print(overfitting_check)
```

```
## # A tibble: 2 x 3
##   Data      .metric .estimate
##   <chr>    <chr>      <dbl>
## 1 Training rmse        2.94
## 2 Test    rmse        2.97
```

The training and test RMSE values are very close, indicating good generalization performance and no evidence of substantial overfitting in the XGBoost model.

```
# Extract the underlying XGBoost model object
xgb_model_obj <- extract_fit_parsnip(xgb_fit)

# Plot top 15 features
vip(xgb_model_obj, geom = "col", num_features = 15, aesthetics = list(fill = "steelblue")) +
  labs(
    title = "Feature Importance (XGBoost)",
    subtitle = "Top 15 predictors by Gain",
    y = "Importance (Gain)",
    x = "Predictor"
  ) +
  theme_minimal()
```



The feature importance plot (based on Gain) highlights the predictors that contribute most to reducing the training loss across boosting iterations. The results indicate that loan term is by far the most influential variable. This is suggesting that the duration of the loan plays a central role in interest rate determination. This is economically intuitive, as longer loan terms typically means higher risk exposure and are therefore priced with higher interest rates.

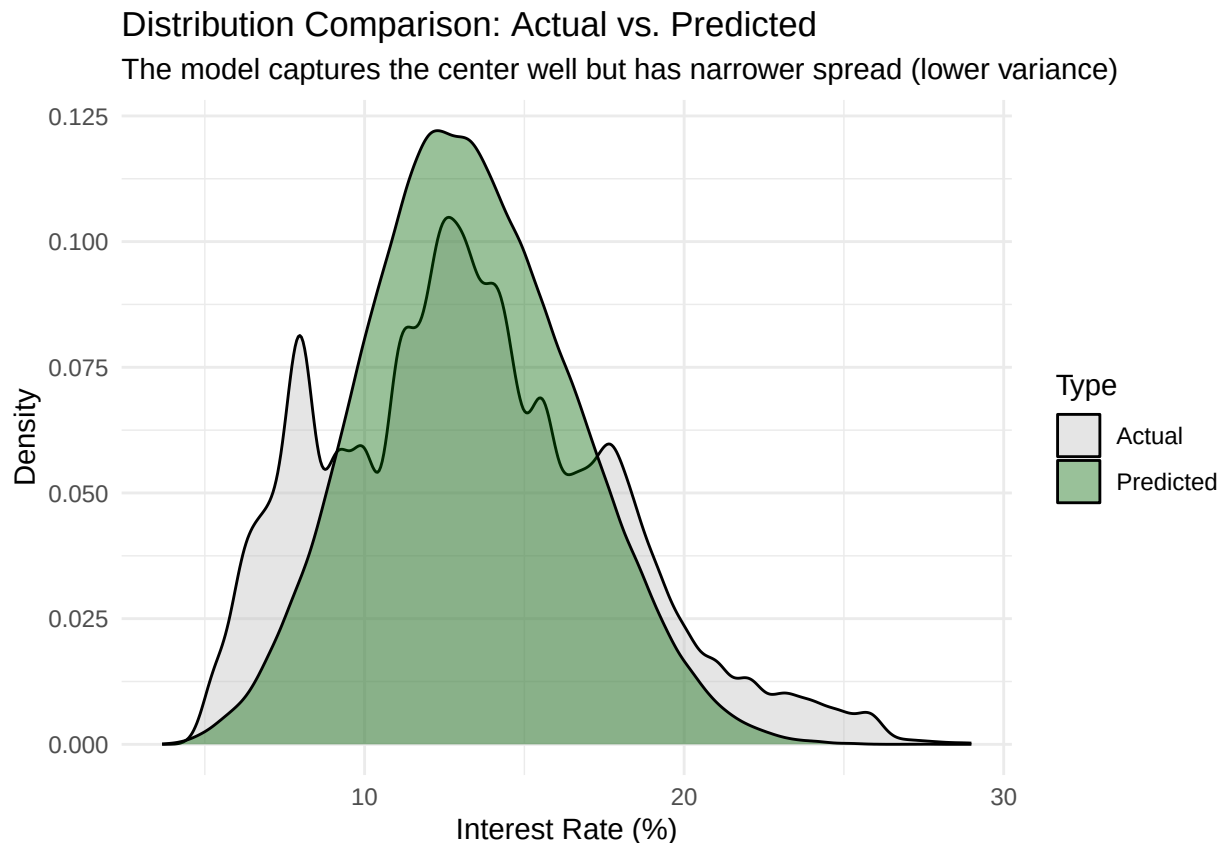
The second most important predictor, total revolving credit limit `total_rev_hi_lim`, reflects a borrower's available credit capacity and overall creditworthiness. Its strong contribution indicates that lenders place significant weight on a borrower's credit limits when pricing loans. Similarly, recent credit inquiries `inq_last_6mths` emerge as an important factor, consistent with the notion that frequent recent credit applications signal elevated risk.

The variable `purpose_credit_card` appears among the more influential predictors in the XGBoost model. This is indicating that loans issued for credit card refinancing carry systematic information for interest rate determination beyond general borrower characteristics. Its importance suggests that, holding other financial variables constant, loans labeled as credit card refinancing tend to be priced differently from other loan purposes.

```
# Combine predictions and truth into a long format for plotting
plot_data <- xgb_results %>%
  pivot_longer(cols = c(int_rate, .pred),
               names_to = "Type",
               values_to = "Interest_Rate") %>%
  mutate(Type = ifelse(Type == "int_rate", "Actual", "Predicted"))

# Plot density curves
ggplot(plot_data, aes(x = Interest_Rate, fill = Type)) +
```

```
geom_density(alpha = 0.4) +
scale_fill_manual(values = c("Actual" = "gray", "Predicted" = "darkgreen")) +
labs(
  title = "Distribution Comparison: Actual vs. Predicted",
  subtitle = "The model captures the center well but has narrower spread (lower variance)",
  x = "Interest Rate (%)",
  y = "Density"
) +
theme_minimal()
```



The distributional comparison shows that the model accurately captures the central mass of the interest rate distribution, with predicted values closely aligned to the actual density in the main range. However, the predicted distribution exhibits a noticeably narrower spread, indicating reduced variance. In particular, extreme high interest rates are underrepresented in the predictions. This behavior reflects a well-regularized model that prioritizes stable average performance and minimizes overall prediction error, at the cost of smoothing rare extreme outcomes. Such variance reduction is consistent with the observed regression-to-the-mean behavior and is typical for tree-based ensemble methods optimized for RMSE.

```
# Save the trained workflow
#saveRDS(
#  xgb_fit,
#  file = "models/final_xgb_model.rds"
#)
```

3.7 Cross-Validation

We apply cross-validation to assess the stability and generalization of the XGBoost model across different data splits to ensure that the observed performance is not driven by a single train-test partition.

```
library(tidymodels)
library(dplyr)

set.seed(123)

train_val_combined <- bind_rows(train, val)
cv_folds <- vfold_cv(train_val_combined, v = 5)

xgb_cv <- fit_resamples(
  final_xgb_workflow,
  resamples = cv_folds,
  metrics   = metric_set(rmse, mae, rsq),
  control   = control_resamples(save_pred = TRUE)
)

assess_mse <- collect_predictions(xgb_cv) %>%
  group_by(id) %>%
  summarise(
    val_mse = mean((int_rate - .pred)^2),
    .groups = "drop"
  )

calc_train_mse <- function(wf, folds) {
  purrr::map2_dfr(folds$splits, folds$id, \(split, fold_id) {
    fit_obj <- fit(wf, data = analysis(split))
    preds   <- predict(fit_obj, new_data = analysis(split)) %>% bind_cols(analysis(split) %>% select(int_rate))
    tibble(
      id = fold_id,
      train_mse = mean((preds$int_rate - preds$.pred)^2)
    )
  })
}

train_mse <- calc_train_mse(final_xgb_workflow, cv_folds)

xgb_stability <- left_join(train_mse, assess_mse, by = "id") %>%
  mutate(
    gap = val_mse - train_mse,
    model = "XGBoost"
  )

xgb_stability %>%
  summarise(
    avg_train_mse = mean(train_mse),
    avg_val_mse   = mean(val_mse),
    avg_gap       = mean(gap)
  )
```

```
## # A tibble: 1 x 3
##   avg_train_mse avg_val_mse avg_gap
##   <dbl>         <dbl>    <dbl>
## 1      8.60         8.81    0.204
```

```
xgb_stability
```

```
## # A tibble: 5 x 5
##   id   train_mse val_mse   gap model
##   <chr>    <dbl>    <dbl> <dbl> <chr>
## 1 Fold1      8.62      8.81 0.193 XGBoost
## 2 Fold2      8.61      8.78 0.173 XGBoost
## 3 Fold3      8.61      8.77 0.164 XGBoost
## 4 Fold4      8.59      8.86 0.270 XGBoost
## 5 Fold5      8.60      8.82 0.220 XGBoost
```

```
final_fit <- fit(final_xgb_workflow, data = train_val_combined)

test_preds <- predict(final_fit, new_data = test) %>%
  bind_cols(test %>% select(int_rate))

metrics(test_preds, truth = int_rate, estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>    <chr>         <dbl>
## 1 rmse    standard         2.97
## 2 rsq     standard         0.543
## 3 mae     standard         2.34
```

The cross-validation results show highly consistent training and validation errors across all folds. Training MSE varies only marginally between folds, and the validation MSE remains close to the training error, resulting in a small and stable generalization gap. This indicates that the XGBoost model generalizes well and does not exhibit substantial overfitting. Moreover, the close alignment between cross-validation and test-set performance suggests that the reported test results are representative and robust rather than split-dependent.

4 Conclusion

The objective of this project was to model and predict LendingClub interest rates using borrower and loan-level information, systematically comparing models of increasing complexity. The analysis followed a structured workflow comprising exploratory data analysis, rigorous preprocessing, baseline modeling, and the application of advanced ensemble methods, all underpinned by extensive diagnostic evaluation. While this report presents the final, refined modeling path, the project involved a highly iterative development process. Numerous alternative specifications, feature engineering trials, and experimental approaches were tested, the full evolution of which can be traced through the project's GitHub commit history.

A central takeaway from this iterative process is that data preprocessing and diagnostic analysis proved to be as critical as the choice of model itself. The extensive cleaning, transformation, and consistent encoding steps were the foundation that allowed for fair evaluation and prevented data leakage. Without this rigor, performance differences would likely have reflected preprocessing artifacts rather than genuine modeling gains.

This diagnostic rigor was most evident in the linear regression phase, which served a dual purpose beyond establishing a transparent baseline. The initial naive full-specification model proved invaluable for diagnostics, as its residuals revealed systematic violations of linearity, specifically U-shaped curves in wealth metrics and a distinct floor effect. These findings confirmed that the underlying mechanism of interest rate formation is structurally non-linear. Subsequently, the strictly linear baseline, screened to satisfy OLS assumptions, resulted in a valid but underpowered model with an Adjusted R-squared of 0.31. This quantified the information gap, representing the precise amount of predictive signal lost when forcing complex financial data into a purely additive framework.

The Random Forest model represented a substantial improvement, successfully bridging the information gap identified in the linear phase. By relaxing parametric assumptions, it reclaimed the non-linear signals, such as revolving balance dynamics, that the linear model was forced to discard. While this resulted in markedly improved robustness, diagnostic analyses showed characteristic regression-to-the-mean behavior. The bagging approach reduced variance effectively but tended to smooth over rare and extreme outcomes, leading to under-prediction of the highest interest rates.

Building on these observations, XGBoost was employed to address residual bias through sequential error correction. The model achieved modest but consistent improvements in explanatory power and successfully reduced bias at the upper end of the interest rate distribution. Cross-validation results demonstrated highly stable training and validation performance across folds, indicating excellent generalization. Nevertheless, even XGBoost produced conservative predictions with reduced variance at the extremes, suggesting that increased model flexibility cannot fully overcome intrinsic data noise.

With a test-set RMSE of 2.96, the final model predicts interest rates with an average error of roughly three percentage points. The persistence of conservative predictions at the extremes highlights that the remaining error is likely driven by unobserved variables, such as exact FICO scores, rather than model deficiency. Ultimately, the project confirms that while ensemble methods are superior for capturing the complex, interactive nature of credit risk, they are limited by the informational ceilings inherent in the dataset.

4.1 Lessons Learned & Critical reflection

1. The “Data Science Reality”: Preprocessing is 80% of the Work We anticipated that model selection would be the primary challenge, but we discovered that the vast majority of our effort was required for preprocessing, consistency checks, and resolving structural anomalies (e.g., the 63k missing values in `tot_cur_bal` or the logic for joint applications). This aligns with the industry reality that a sophisticated model cannot fix broken data. We learned that data hygiene and consistent encoding were the true drivers of stability; inconsistent preprocessing would have dominated our model error more than the choice between Random Forest and XGBoost.

2. The Diagnostic Power of “Failed” Models: While our Linear Regression underperformed relative to ensemble methods, it proved to be our most valuable diagnostic tool. The Residuals vs Fitted plot revealed a systematic U-shaped curvature and a distinct “floor” at the lower interest rate bounds. This visual evidence demonstrated that the relationship between financial leverage and interest rates is structurally non-linear—for example, increasing revolving balance affects interest rates differently at low versus high utilization levels. We learned that relying solely on global metrics like RMSE would have masked these structural mismatches, whereas the residual diagnostics explicitly justified our shift to non-parametric tree-based methods.

3. The Limits of Additivity in Credit Scoring: Our analysis confirms that linear regression is fundamentally ill-suited for raw LendingClub data without extensive feature engineering. Linear models assume additive effects (e.g., “high DTI always adds X% risk”), but credit risk is inherently interactive—a high debt-to-income ratio carries different risks depending on verification status or income level. Our OLS baseline failed to capture these “risk multipliers” without manually specified interaction terms. This explains the performance gap between the OLS baseline and the Random Forest, confirming that in credit scoring, the predictive signal often lies in the interaction of variables rather than their individual additive contributions. Using these insights for feature engineering would likely optimize the model’s performance.

4. Critical Assessment of Feature Engineering: The “Conservative” Trap In our pursuit of data quality, we adopted a “conservative” approach to feature engineering, dropping variables like `earliest_cr_line` (due to parsing complexity) and `revol_util` (due to recalculation risks). In hindsight, this was likely too risk-averse. By not engineering a Credit History Length feature, we likely sacrificed significant predictive variance. We learned that there is a trade-off between data purity and predictive signal; future iterations would prioritize engineering these “difficult” features rather than discarding them to potentially lower our MSE below 8.78.

5. Boosting vs. Bagging and the “Informational Ceiling” Comparing: Random Forest and XGBoost revealed that while boosting effectively reduces bias (correcting the under-prediction of high interest rates), it cannot overcome intrinsic data limitations. Both models exhibited “regression to the mean” at the extremes of the distribution. This persistence suggests that a portion of the interest rate variance is driven by unobserved variables (such as FICO scores, which are likely used by LendingClub but not present in this dataset). We learned that marginal gains in algorithm complexity yield diminishing returns when the necessary features for extreme-case prediction are absent.

6. Operational Validity Over Theoretical Performance: A major strategic decision was the aggressive removal of variables that posed a “future leakage” risk, such as `issue_d` or `total_pymnt`. While retaining them might have artificially boosted our training R2, it would have caused a catastrophic failure in the “Reality Check” on new loan applications. We learned to prioritize operational validity over theoretical performance, ensuring our model is deployable in a real-world underwriting context.

7. Methodology Limitations Beyond the specific modeling steps, our iterative process revealed limitations in our approach:

- The Domain Knowledge Gap: We approached this problem with limited prior knowledge of US credit markets. While we utilized AI tools to assist with terminology and concept validation, they cannot replicate the intuition of a true subject matter expert. A deeper understanding of credit risk mechanics would likely have led to more sophisticated cleaning decisions (e.g., nuances in “missing” data) and richer feature engineering, unlocking performance gains that pure statistical optimization could not achieve.
- Path Dependence: Our decision to exclude Joint Applications (N=460) simplified the pipeline but limited the final model’s scope regarding this specific customer segment.
- Imputation Bias: Relying on median imputation for missing values in features like `dti` or `revol_util` potentially removed informative “missingness” signals that tree-based models could have utilized.
- Hyperparameter Search: Due to computational constraints, we employed a simplified grid search rather than Bayesian optimization. A more exhaustive search could arguably have improved performance marginally.
- AI Reliance: While AI tools accelerated code generation, we had to manually intervene to prevent standard “textbook” cleaning steps (like aggressive outlier removal) from deleting legitimate high-income data points.